GO

GO (GOLANG) · SDE2 & SSE

Go

> Goroutines, channels, select, interfaces, memory model – concurrency ని భాషలోనే కలిపిన డిజైన్.

JavaScript నుంచి వచ్చినవారికి: Go lo event loop లేదు, goroutines ఉన్నాయి. ఆ మార్పు ని ఇక్కడ అడుగడుగునా వివరించాం.

తెలుగు + ENGLISH

YASWANTH · REFERENCE

విషయ సూచిక (Table of Contents)

Part 1 – పునాదులు (Foundations)

1. Go అంటే ఏమిటి, ఎందుకు పుట్టింది (history, philosophy, ఎక్కడ వాడతారు, Go vs Java/Python/C++)
2. Setup & Toolchain (install, GOROOT/GOPATH, Go Modules, project layout, go commands)
3. మొదటి Program Structure (package, import, func main, exported vs unexported)
4. Variables, Constants, iota, Zero Values, Type Inference, Shadowing
5. Basic Types (int/uint/float/complex/bool), sizes, overflow, conversions
6. Strings, byte, rune, UTF-8 Deep Dive
7. Operators & Control Flow (if/for/switch/goto/labels)

Part 2 – Types & Data Structures

8. Arrays (fixed size, value semantics, comparison)
9. Slices – DEEP (header, append growth, aliasing, copy, nil vs empty, full slice expr)
10. Maps – internals, comma-ok, delete, nil map panic, iteration order, concurrency
11. Structs – fields, embedding, tags, anonymous, comparison, empty struct, memory layout
12. Pointers – & and *, new(), no arithmetic, nil, pointer vs value
13. Functions – multiple returns, variadic, closures, defer, panic & recover
14. Methods – value vs pointer receivers, method sets, method values & expressions
15. Interfaces – implicit satisfaction, any, type assertion/switch, internals, typed-nil trap
16. Embedding & Composition (struct/interface embedding, promotion, overriding)
17. Generics (Go 1.18+) – type parameters, constraints, type sets, inference, internals
18. Error Handling – error interface, sentinel, %w wrapping, errors.Is/As, custom types

Part 3 – Concurrency (అత్యంత ముఖ్యం)

19. Goroutines – go keyword, 2KB stack, vs OS threads, lifecycle, leaks
20. GMP Scheduler – G/M/P model, run queues, work-stealing, preemption, netpoller
21. Channels – unbuffered vs buffered, close, range, nil channel, directional, deadlock
22. select – multiplexing, default, timeout, for-select, done channel
23. sync package – Mutex, RWMutex, WaitGroup, Once, Cond, sync.Map, sync.Pool
24. sync/atomic – atomic ops, CAS, atomic.Value/Int64, atomic vs mutex
25. context – cancellation, WithCancel/Timeout/Deadline/Value, propagation, gotchas
26. Concurrency Patterns – worker pool, fan-in/out, pipeline, generator, semaphore, errgroup
27. Go Memory Model – happens-before, data races, -race, ordering
28. Common Concurrency Bugs – deadlock, leak, loop var capture, races, closed channel

Part 4 – Memory & Runtime Internals

29. Stack vs Heap + Escape Analysis (-gcflags='-m', escape triggers, optimizing)
30. Garbage Collector – tricolor mark-sweep, write barrier, pacing, GOGC, GOMEMLIMIT
31. Memory Allocator – mcache/mcentral/mheap, size classes, spans, tcmalloc heritage
32. Runtime Deep – defer/panic/recover internals, stack growth (copying), GODEBUG

Part 5 – Standard Library & Tooling

33. Packages & Modules Deep – semantic versioning, go.sum, vendoring, workspaces, init()
34. Essential stdlib tour – fmt, io/bufio, os, time, sort, slices/maps, encoding/json, regexp
35. Testing – table-driven, subtests, benchmarks, fuzzing, coverage, TestMain, mocking
36. Tooling – vet, gofmt/goimports, golangci-lint, pprof, trace, build tags, race detector
37. Reflection – reflect.Type/Value, struct tags, use cases, cost & when to avoid
38. unsafe – unsafe.Pointer, uintptr rules, when justified

Part 6 – Applied Go & Idioms

39. HTTP – net/http server & client, ServeMux, handlers, middleware, graceful shutdown
40. JSON & config, database/sql + connection pool, context in requests
41. Idiomatic Go (Effective Go highlights) – naming, errors, accept interfaces, zero-value
42. Gotchas & Pitfalls Cheat-sheet
43. Memory Tips + Common Mistakes + Interview Cheat-sheet

Part 1 – పునాదులు (Foundations)

> Go ని అర్థం చేసుకోవాలంటే మొదట దాని *తత్వం* (philosophy) అర్థం కావాలి: **simplicity, fast compilation, built-in concurrency.** ఈ Part లో language పుట్టుక నుండి, toolchain, మొదటి program, types, strings, control flow వరకు పునాది వేస్తాం.

---

1. Go అంటే ఏమిటి, ఎందుకు పుట్టింది

వివరణ

Go (అధికారికంగా *Golang* అని కూడా అంటారు, ఎందుకంటే `golang.org` domain) అనేది **Google** లో 2007లో మొదలై, 2009లో open-source అయిన ఒక **statically-typed, compiled** programming language. దీన్ని **Robert Griesemer, Rob Pike, Ken Thompson** (Unix, C, UTF-8 fame) తయారు చేశారు.

Go ఎందుకు పుట్టింది? Google లో C++ codebases **compile అవ్వడానికి గంటల సమయం** పట్టేది, dependency management గజిబిజిగా ఉండేది, మరియు multi-core hardware ని బాగా వాడుకోవడానికి

C++/Java concurrency చాలా క్లిష్టంగా ఉండేది. ఈ మూడు నొప్పులకి (slow builds, complex dependencies, poor concurrency) పరిష్కారంగా Go పుట్టింది.

Go యొక్క తత్వం (philosophy):

- **Simplicity (సరళత):** భాష చిన్నది – కేవలం **25 keywords** (C++ కి 90+, Java కి 50+). ఒక్క syntax చేయడానికి ఒకటే మార్గం. "There should be one obvious way."
- **Fast compilation:** పెద్ద codebases కూడా సెకన్లలో compile అవుతాయి. Dependency graph design వల్ల unused imports కూడా error.
- **Built-in concurrency:** `goroutine`, `channel` భాషలోనే భాగం – thread libraries అవసరం లేదు. "Don't communicate by sharing memory; share memory by communicating."
- **No inheritance, no exceptions, no generics (మొదట్లో):** classic OOP baggage తీసేశారు. Composition + interfaces + explicit error handling.
- **Batteries included:** గొప్ప standard library (HTTP server, JSON, crypto, testing అన్నీ built-in).
- **గొప్ప tooling:** `gofmt` (ఒకటే formatting style – arguments అనవసరం), `go test`, `go vet`, race detector భాషతోనే వస్తాయి.

Real-life Scenario

> **C++ = ఒక పూర్తి workshop నిండా powerful కానీ ప్రమాదకరమైన power tools.** ఏదైనా చేయవచ్చు, కానీ ఒక్క పొరపాటుకి వేలు తెగుతుంది (segfault, memory leak), నేర్చుకోవడానికి సంవత్సరాలు.
>
> **Go = IKEA furniture kit.** పరిమిత parts, ప్రతి దానికి ఒకటే సరైన స్థానం, Allen key (gofmt) ఒకటే. త్వరగా, safely, ఒక team అంతా ఒకేలా assemble చేయగలదు. "Boring" గా అనిపించవచ్చు – కానీ production లో boring అంటే **reliable.**

Go "smart" గా ఉండాలని ప్రయత్నించదు – **readable, predictable** గా ఉండాలని ప్రయత్నిస్తుంది. ఒక junior engineer రాసిన Go, ఒక senior రాసిన Go దాదాపు ఒకేలా ఉంటాయి. ఇదే దాని బలం.

ఎక్కడ వాడతారు (Real-world usage)

Go అనేది **cloud-native infrastructure** యొక్క భాష. ప్రపంచంలో అత్యంత ప్రసిద్ధ infra tools చాలా వరకు Go లోనే:

- **Docker** – containerization (Go లో రాశారు)
- **Kubernetes (K8s)** – container orchestration
- **etcd** – distributed key-value store (K8s కి backbone)
- **Prometheus, Grafana Loki** – monitoring
- **Terraform, Consul, Vault** (HashiCorp) – infra as code
- **CockroachDB, InfluxDB, TiDB** – databases
- **Cloudflare, Uber, Twitch, Dropbox, Netflix** – high-throughput backend services

ఎందుకు? ఎందుకంటే Go ఒకే binary గా compile అవుతుంది (no runtime/JVM needed), fast, low memory, మరియు concurrency సహజంగా వస్తుంది – ఇవన్నీ network services, CLI tools,

infrastructure కి perfect.

Go vs Java vs Python vs C++

అంశం	Go	Java	Python	C++
Typing	Static, compiled	Static, compiled (JVM)	Dynamic, interpreted	Static, compiled
Runtime	Single native binary	JVM అవసరం	Interpreter అవసరం	Native binary
Compile speed	చాలా వేగం	మధ్యస్థం	N/A (interpreted)	నెమ్మది
Memory mgmt	GC (low-latency)	GC (JVM)	GC (ref counting)	Manual / RAII
Concurrency	Goroutines + channels (built-in)	Threads, Executors, virtual threads (Loom)	GIL (thread-limited), asyncio	Threads, std::async
Inheritance	లేదు (composition + interfaces)	Class inheritance	Class inheritance	Multiple inheritance
Exceptions	లేదు (explicit `error` values)	try/catch	try/except	try/catch
Generics	ఉంది (1.18+)	ఉంది	Duck typing	Templates
Startup time	సూక్ష్మ (ms)	నెమ్మది (JVM warmup)	వేగం	సూక్ష్మ
నేర్చుకోవడం	సులభం (25 keywords)	మధ్యస్థం	సులభం	కష్టం
Best for	Cloud infra, network services, CLIs	Enterprise apps, Android	Scripting, ML, data	Systems, games, embedded

Key Points

- Go = Google (2007-09), by Griesemer/Pike/Thompson – Unix/C legacy.
- మూడు core goals: **fast compile, simple language, native concurrency.**
- Cloud-native ప్రపంచానికి de-facto భాష (Docker, K8s, etcd).
- "Boring on purpose" – smart కాదు, predictable & readable.
- Single static binary → deploy సులభం, no runtime dependency.

Interview దృష్టి

Q: Go ఎందుకు design చేశారు, ఏ problems solve చేయడానికి? A: Google లో మూడు నొప్పులు – (1) C++ యొక్క నెమ్మది builds, (2) క్లిష్టమైన dependency management, (3) multi-core hardware ని వాడుకోవడానికి కష్టమైన concurrency. Go వీటిని fast compilation, simple dependency graph, built-in

goroutines/channels తో solve చేస్తుంది. దానితో పాటు garbage collection తో memory safety, single static binary తో సులభమైన deployment ఇస్తుంది.

Q: Go ఎందుకు "opinionated"? A: gofmt ఒకటే formatting ఇస్తుంది (style wars లేవు), unused imports/variables compile errors, ఒక్క pattern చేయడానికి ఒకటే మార్గం – ఇవన్నీ large teams లో consistency & maintainability కోసం ఉద్దేశపూర్వకంగా చేసిన నిర్ణయాలు.

---

2. Setup & Toolchain

వివరణ

Go install చేయడం చాలా సులభం – `go.dev/dl` నుండి installer, లేదా Mac లో `brew install go`, Linux లో package manager. Install అయ్యాక `go version` verify చేయవచ్చు. Go తో పాటు మొత్తం **toolchain** (compiler, formatter, test runner, vet, doc, race detector) ఒకే `go` binary లో వస్తుంది – వేరే IDE లేదా build tool అవసరం లేదు.

రెండు కీలక environment concepts:

- **GOROOT** – Go install అయిన చోటు (ఉదా. `/usr/local/go`). Standard library ఇక్కడ ఉంటుంది. దీన్ని మనం సాధారణంగా touch చేయం.
- **GOPATH** – పాత (pre-1.11) workspace root (default `~/go`). పాత రోజుల్లో మొత్తం source ఇక్కడ `src/` లో ఉండాలి. ఇప్పుడు **Go Modules** వచ్చాక GOPATH అనవసరం అయింది; కానీ `$GOPATH/pkg/mod` ఇంకా **module cache** గా, `$GOPATH/bin` installed binaries కి వాడతారు.

Go Modules (go.mod / go.sum) – ఆధునిక dependency management

Go 1.11 లో **modules** వచ్చాయి. ఇప్పుడు project ఎక్కడైనా ఉండవచ్చు (GOPATH లో ఉండనవసరం లేదు). ఒక module = ఒక `go.mod` file ఉన్న directory.

```
# కొత్త project మొదలుపెట్టడం
mkdir myapp && cd myapp
go mod init github.com/surya/myapp   # go.mod create అవుతుంది

# dependency add చేయడం (import రాసి)
go get github.com/google/uuid@latest # go.mod + go.sum update

go mod tidy    # వాడని deps తీసేసి, missing deps add చేస్తుంది
go mod verify  # cache లోని modules tamper అవ్వలేదని check
go mod download # అన్ని deps download (build లేకుండా)
```

`go.mod` ఇలా ఉంటుంది:

```
module github.com/surya/myapp

go 1.22

require (
```

```
        github.com/google/uuid v1.6.0
        golang.org/x/sync v0.7.0 // indirect
)
```

- `go.mod` – module path, Go version, direct + indirect dependencies (semantic versions).
- `go.sum` – ప్రతి dependency యొక్క cryptographic **checksum** (hash). ఇది **supply-chain security** – cache లోని code tamper కాలేదని verify చేస్తుంది. దీన్ని ఎప్పుడూ commit చేయాలి.

Standard Project Layout

```
myapp/
├── go.mod
├── go.sum
├── main.go              # package main, func main
├── cmd/                 # multiple binaries (cmd/api, cmd/worker)
│   └── api/main.go
├── internal/            # ఈ module మాత్రమే import చేయగల private code
│   └── service/service.go
├── pkg/                  # బయటి projects import చేయదగిన public library code
│   └── util/util.go
└── api/, configs/, scripts/  # non-Go assets
```

- `internal/` – Go compiler enforce చేసే special directory. దీనిలోని packages ని ఆ module (parent) మాత్రమే import చేయగలదు; బయటి ఎవరూ చేయలేరు. Encapsulation కి గొప్ప tool.
- `cmd/` – ఒక repo లో అనేక binaries ఉంటే ఇక్కడ పెడతారు.

Essential `go` commands

Command	పని
`go run main.go`	Compile చేసి వెంటనే run (binary save చేయదు) – dev లో వాడతారు
`go build`	Binary compile చేసి save చేస్తుంది (deploy కి)
`go install`	Compile చేసి `$GOPATH/bin` లో పెడుతుంది (CLI tools install కి)
`go test ./...`	అన్ని packages test చేస్తుంది
`go vet ./...`	Suspicious code catch చేస్తుంది (static analysis)
`go fmt ./...`	Code ని canonical style కి format (gofmt)
`go get pkg@ver`	Dependency add/update
`go mod tidy`	Dependencies శుభ్రం చేస్తుంది
`go doc fmt.Println`	Documentation చూపిస్తుంది
`go env`	Go environment variables చూపిస్తుంది

`./...` అంటే "ఇక్కడి నుండి recursively అన్ని sub-packages" – Go లో చాలా common pattern.

Real-life Scenario

> **go.mod = ఒక recipe (వంటకం జాబితా):** ఏ ingredients (dependencies), ఏ versions కావాలో చెప్తుంది.
>
> **go.sum = ప్రతి ingredient packet మీద tamper-proof seal:** నువ్వు కొన్న పాల ప్యాకెట్ నిజంగా అదే, ఎవరూ మధ్యలో మార్చలేదని guarantee ఇస్తుంది. Seal మ్యాచ్ అవ్వకపోతే build ఆగిపోతుంది – ఇదే supply-chain attack నుండి రక్షణ.

Code – cross-compilation (Go యొక్క superpower)

```
# Mac లో కూర్చుని Linux ARM binary build చేయడం – toolchain ఏదీ install అవసరం లేదు!
GOOS=linux GOARCH=arm64 go build -o myapp-linux main.go

# Windows binary
GOOS=windows GOARCH=amd64 go build -o myapp.exe main.go

# ఇది static binary – ఆ machine లో Go install అవసరం లేదు, ఒక్క file copy చేస్తే చాలు
```

Gotchas (సాధారణ తప్పులు)

- **తప్పు:** పాత tutorials చూసి code ని `$GOPATH/src` లో పెట్టడం. **సరి:** Modules తో project ఎక్కడైనా ఉండవచ్చు; `go mod init` చాలు.
- **తప్పు:** `go.sum` ని `.gitignore` లో పెట్టడం. **సరి:** `go.sum` ఎప్పుడూ commit చేయాలి – reproducible + secure builds కి అవసరం.
- **తప్పు:** dependency తీసేసినా `go.mod` లో ఉండిపోవడం. **సరి:** `go mod tidy` regularly run చేయాలి.

Key Points

- మొత్తం toolchain ఒకే `go` binary – compiler, fmt, test, vet, race detector.
- **GOROOT** = Go install; **GOPATH** = పాత workspace, ఇప్పుడు module cache కి మాత్రమే.
- **Modules** (go.mod/go.sum) = ఆధునిక dependency mgmt; project GOPATH బయట ఉండవచ్చు.
- **go.sum** = security checksums – ఎప్పుడూ commit.
- **internal/** = compiler-enforced private packages.
- **Cross-compile** trivially – `GOOS` / `GOARCH` set చేస్తే చాలు.

Interview దృష్టి

Q: GOPATH vs Go Modules తేడా? A: GOPATH అనేది పాత model – మొత్తం code ఒకే global workspace (`~/go/src`) లో ఉండాలి, versioning లేదు. Modules (1.11+) project-local `go.mod` తో explicit versioned dependencies ఇస్తాయి, project ఎక్కడైనా ఉండవచ్చు, reproducible builds వస్తాయి. Modern Go = always modules.

Q: go.sum ఎందుకు? A: ప్రతి dependency version యొక్క cryptographic hash store చేస్తుంది. Download అయిన code నిజంగా అదే, tamper అవ్వలేదని verify చేస్తుంది – supply-chain security కి కీలకం.

3. మొదటి Program Structure

వివరణ

ప్రతి Go file ఒక **package declaration** తో మొదలవుతుంది. Executable program కి `package main` + `func main()` కావాలి — ఇదే program యొక్క entry point. Library కి `package main` అవసరం లేదు.

```
package main // ఈ file executable program లో భాగం

import "fmt" // standard library package import

// func main = program మొదలయ్యే చోటు (entry point)
func main() {
	fmt.Println("నమస్తే, Go!") // console కి print
}
```

`go run main.go` → `నమస్తే, Go!`

ప్రతి భాగం వివరణ

- **`package main`** — ఈ file ఏ package కి చెందుతుందో చెప్తుంది. `main` అనే మ్యాజిక్ పేరు ఉన్న package + `func main()` ఉంటేనే `go build` executable తయారు చేస్తుంది. మిగతా అన్ని package పేర్లు libraries.
- **`import "fmt"`** — వేరే package వాడాలంటే import చేయాలి. `fmt` = formatting/printing. **వాడని import compile error** — Go strict.
- **`func main()`** — arguments లేవు, return లేదు. Program ఇక్కడ మొదలై, ఇది return అయితే program ముగుస్తుంది.

Exported vs Unexported — Capitalization యే access modifier

Go లో `public` / `private` keywords **లేవు**. బదులుగా **మొదటి అక్షరం capital అయితే exported (public)**, small అయితే unexported (package-private).

```
package bank

// Exported (public) — capital B → వేరే packages వాడగలవు
func Balance() int { return balance }

// unexported (private) — small b → ఈ package లోపలే
var balance int

type Account struct {
	Owner  string // exported field — బయటి packages read/write చేయగలవు
	pin    int    // unexported field — package లోపలే
}
```

ఇది **types, functions, methods, struct fields, constants** – అన్నిటికీ వర్తిస్తుంది. `math.Pi` (capital P) వాడగలం; `math.pi` అంటూ ఏదైనా ఉంటే వాడలేం.

Real-life Scenario

> **Exported = ఇంటి బయటి board మీద పెద్ద అక్షరాలతో రాసిన "Reception".** ఎవరైనా చూడగలరు, రాగలరు.
>
> **Unexported = ఇంటి లోపలి బెడ్‌రూం.** కుటుంబ సభ్యులకే (అదే package లోని code కే) access. బయటి వాళ్ళు చూడలేరు.
>
> ఒక్క అక్షరం capital/small – అదే తలుపు తాళం. చాలా సరళం, కానీ powerful.

Code – ఒక చిన్న multi-file package

```
// file: greet/greet.go
package greet

import "fmt"

// SayHello — exported, బయటి packages వాడగలవు
func SayHello(name string) string {
	return fmt.Sprintf("నమస్తే, %s!", name)
}

// whisper — unexported helper, ఈ package లోపలే
func whisper(s string) string {
	return "(" + s + ")"
}
```

```
// file: main.go
package main

import (
	"fmt"

	"github.com/surya/myapp/greet"
)

func main() {
	fmt.Println(greet.SayHello("సూర్య")) // నమస్తే, సూర్య!
	// greet.whisper("hi")  // ❌ compile error: unexported
}
```

Gotchas (సాధారణ తప్పులు)

- **తప్పు:** వాడని import ఉంచడం → `imported and not used` compile error. **సరి:** `goimports` తో auto-manage, లేదా తొలగించడం.
- **తప్పు:** exported చేయాలనుకున్న function పేరు small letter తో మొదలుపెట్టడం → బయటి package వాడలేదు. **సరి:** capital తో మొదలుపెట్టు.

- **తప్పు:** ఒక folder లో రెండు వేర్వేరు package పేర్లు – compile error. ఒక directory = ఒక package.

Key Points

- ప్రతి file `package X` తో మొదలు; executable కి `package main` + `func main()`.
- వాడని import/variable = **compile error** (Go strict).
- Access control = **capitalization** (Capital = exported/public, small = unexported).
- Capitalization types/funcs/fields/constants అన్నిటికీ వర్తిస్తుంది.
- ఒక directory = ఒక package.

Interview దృష్టి

Q: Go లో public/private ఎలా? A: keywords లేవు; identifier మొదటి అక్షరం capital అయితే exported (బయటి packages access చేయగలవు), small అయితే unexported (package-local). Compiler enforce చేస్తుంది.

---

4. Variables, Constants, iota, Zero Values, Type Inference, Shadowing

వివరణ

Go లో variables declare చేయడానికి అనేక మార్గాలు. కీలకం: **ప్రతి variable కి ఒక type ఉంటుంది** (statically typed), కానీ Go చాలా సార్లు type ని *infer* చేయగలదు.

```
var a int = 10        // full form: name, type, value
var b = 20            // type inferred (int)
var c int             // value లేదు → zero value (0)
d := 30               // short form (:= ) — type inferred, function లోపలే
var (                 // grouped declaration
        name string = "Surya"
        age  int    = 30
)
```

- `var` – ఎక్కడైనా (package level లేదా function level) వాడవచ్చు.
- `:=` (short variable declaration) – **function లోపల మాత్రమే**. Type ని auto-infer చేస్తుంది. ఇదే Go లో అత్యంత common.
- **Package-level** variables కి `:=` వాడలేం; `var` మాత్రమే.

Zero Values – Go లో "null" అనే గజిబిజి లేదు

Go లో ప్రతి type కి ఒక **zero value** ఉంటుంది. Declare చేసి value ఇవ్వకపోతే, automatically zero value వస్తుంది. **Uninitialized garbage అనేదే లేదు** (C లా కాదు).

Type	Zero Value
`int, float, complex`	`0`
`bool`	`false`
`string`	`""` (empty, nil కాదు)
`pointer, slice, map, channel, func, interface`	`nil`
`struct`	ప్రతి field దాని zero value

ఇది **"zero value useful" idiom** కి పునాది – designed types ని initialize లేకుండానే వాడగలిగేలా చేయడం (ఉదా. `var mu sync.Mutex` వెంటనే ready).

Constants & iota

`const` = compile-time లోనే fix అయ్యే value. Runtime లో మారదు. Constants **untyped** గా ఉండగలవు (flexibility).

```
const Pi = 3.14159        // untyped — అవసరమైన చోట float32/float64 లా adapt
const MaxUsers int = 1000 // typed constant

// iota = auto-incrementing counter, const block లో 0 నుండి మొదలు
type Weekday int
const (
        Sunday    Weekday = iota // 0
        Monday                   // 1 (iota auto-increments)
        Tuesday                  // 2
        Wednesday                // 3
)

// iota తో bit flags (powers of 2)
const (
        Read    = 1 << iota // 1  (1<<0)
        Write               // 2  (1<<1)
        Execute             // 4  (1<<2)
)

// iota తో size units
const (
        _  = iota              // 0 ని skip (blank identifier)
        KB = 1 << (10 * iota) // 1<<10 = 1024
        MB                     // 1<<20
        GB                     // 1<<30
)
```

`iota` – const block లో ప్రతి line కి 0, 1, 2... గా పెరిగే counter. Enums, flags, size classes కి idiomatic.

Type Inference (`:=`)

```
count := 42          // int
price := 9.99        // float64
```

```
name := "Go"        // string
ok := true          // bool
nums := []int{1, 2}  // []int

// multiple assignment
x, y := 1, 2
x, y = y, x         // swap — temp అవసరం లేదు
```

Go **default** గా integers ని `int`, decimals ని `float64` గా infer చేస్తుంది.

Shadowing (నీడ) – ఒక tricky gotcha

లోపలి scope లో అదే పేరుతో కొత్త variable declare చేస్తే, అది బయటి variable ని *shadow* (కప్పి) చేస్తుంది.

```
func main() {
	x := 10
	fmt.Println(x) // 10
	{
		x := 20        // కొత్త x — బయటి x ని shadow చేస్తుంది
		fmt.Println(x) // 20
	}
	fmt.Println(x) // 10 (బయటి x మారలేదు)
}
```

Real-life Scenario

> **Zero value = కొత్త బ్యాంకు account.** ఎవరైనా account తెరిస్తే balance ఏదో garbage కాదు – always ₹0. Go లో కూడా `var n int` అంటే n = 0, ఏదో random విలువ కాదు. ఇది bugs ని చాలా తగ్గిస్తుంది.
>
> **Shadowing = ఇంట్లో "బాబు" అనే పేరు.** హాల్లో "బాబు" అంటే మీ కొడుకు; పక్కింట్లో వాళ్ళ "బాబు" వేరు. అదే పేరు, వేరే వ్యక్తి, వేరే గది (scope). పొరపాటున తప్పు "బాబు" ని పిలిస్తే గందరగోళం – అదే shadowing bug.

Gotchas (సాధారణ తప్పులు)

- **Shadowing bug (అత్యంత ప్రమాదకరం):** error handling లో `:=` వాడి outer `err` ని shadow చేయడం:

```
func process() (err error) {
	x, err := step1()      // err declared
	if cond {
		y, err := step2()  // ❌ కొత్త err! outer err update కాదు
		_ = y
		if err != nil { return } // ఇది inner err
	}
	_ = x
	return err // outer err ఎప్పటికీ set కాకపోవచ్చు
}
```

Fix: `go vet -vettool=shadow` లేదా `y, err = step2()` (`=` వాడు, `:=` కాదు).

- **తప్పు:** `:=` ని package level లో వాడటం → compile error. **సరి:** package level లో `var`.
- **తప్పు:** declare చేసి వాడని local variable → `declared and not used` error. **సరి:** `_` (blank) కి assign లేదా తీసేయి.

Key Points

- `var` (ఎక్కడైనా) vs `:=` (function లోపలే, inferred).
- **Zero values** – ప్రతి type initialize అవుతుంది; నో garbage. `""`, `0`, `false`, `nil`.
- `const` = compile-time; **untyped constants** flexible.
- `iota` = const block counter – enums, flags, sizes కి idiomatic.
- **Shadowing** with `:=` = classic silent bug – `go vet` వాడు.
- Unused local variables = compile error.

Interview దృష్టి

Q: Go లో zero value అంటే ఏమిటి, ఎందుకు ముఖ్యం? A: Declare చేసి initialize చేయని ప్రతి variable కి type-specific default వస్తుంది (0, "", false, nil). C లాంటి uninitialized garbage ఉండదు – safety. ఇది "make the zero value useful" idiom ని enable చేస్తుంది (ఉదా. `var mu sync.Mutex` వెంటనే usable).

Q: `var x = 0` vs `x := 0` తేడా? A: రెండూ int x=0 ఇస్తాయి, కానీ `:=` function లోపలే పనిచేస్తుంది; package-level కి `var` కావాలి. `:=` కి కనీసం ఒక కొత్త variable ఉండాలి left side లో.

---

5. Basic Types (int/uint/float/complex/bool)

వివరణ

Go statically-typed – ప్రతి value కి ఒక precise type. Numeric types size ని పేరులోనే చెప్తాయి. కీలక rule: **Go లో implicit conversion లేదు** – int ని float కి కలపాలంటే explicitly convert చేయాలి. ఇది "surprises" ని తగ్గించే design decision.

Numeric Types & Sizes

Type	Size	Range / గమనిక
`int8 / uint8`	1 byte	-128..127 / 0..255 (`byte` = `uint8`)
`int16 / uint16`	2 bytes	~±32K / 0..65535
`int32 / uint32`	4 bytes	~±2.1B (`rune` = `int32`)
`int64 / uint64`	8 bytes	~$\pm 9.2 \times 10^{18}$
`int / uint`	4 లేదా 8 bytes	**platform-dependent** (64-bit OS లో 8 bytes)

Type	Size	Range / గమనిక
`float32`	4 bytes	~7 decimal digits precision
`float64`	8 bytes	~15 digits – **default** decimal type
`complex64/128`	8/16 bytes	real + imaginary parts
`bool`	1 byte	true / false
`uintptr`	pointer-size	pointer ని integer గా (unsafe)

`int` vs `int64`: `int` platform word size – 64-bit machines లో 64 bits. General counting కి `int` వాడు; wire formats/serialization కి explicit `int32` / `int64` వాడు.

Code – types, conversions, overflow

```
package main

import (
	"fmt"
	"math"
)

func main() {
	var i int = 10
	var f float64 = 3.5

	// ❌ fmt.Println(i + f)  // compile error: mismatched types
	// ✅ explicit conversion అవసరం
	sum := float64(i) + f
	fmt.Println(sum) // 13.5

	// float → int: దశాంశం truncate అవుతుంది (round కాదు)
	f2 := 3.99
	fmt.Println(int(f2)) // 3
	// ⚠️ గమనిక: నేరుగా int(3.99) రాయలేం — అది compile error.
	// constant లో దశాంశం (fractional part) ఉంటే int కి convert కాదు;
	// variable ని మాత్రమే truncate చేయగలం.

	// Overflow — wrap around అవుతుంది (panic కాదు!)
	var b uint8 = 255
	b++
	fmt.Println(b) // 0  (255+1 wraps to 0)

	var x int8 = 127
	x++
	fmt.Println(x) // -128  (overflow → wrap)

	// Max/Min limits
	fmt.Println(math.MaxInt64) // 9223372036854775807
	fmt.Println(math.MaxInt8)  // 127

	// float precision gotcha
	fmt.Println(0.1 + 0.2)         // 0.30000000000000004
```

```
        fmt.Println(0.1+0.2 == 0.3)    // false!
}
```

Overflow behavior – silent wrap

Go integer overflow **panic కాదు, wrap around అవుతుంది** (two's complement). `uint8` 255 + 1 = 0. ఇది C లాంటిదే – performance కోసం. కాబట్టి counters, IDs కి తగిన size ఎంచుకోవాలి. Constant overflow మాత్రం compile-time లో catch అవుతుంది.

Real-life Scenario

> **Implicit conversion లేకపోవడం = airport లో currency exchange.** రూపాయలు, డాలర్లు నేరుగా కలపలేవు – మొదట explicitly exchange చేయాలి. Go కూడా `int` + `float64` ని నేరుగా కలపనివ్వదు; నువ్వు `float64(i)` అని explicit convert చేయాలి. చిరాకుగా అనిపించవచ్చు, కానీ "అనుకోకుండా తప్పు currency కలిపేశా" లాంటి bugs రావు.
>
> **Overflow wrap = car odometer.** 999999 దాటితే 000000 కి తిరిగి వస్తుంది – ఆగిపోదు, wrap అవుతుంది. `uint8` కూడా అంతే.

Gotchas (సాధారణ తప్పులు)

- **Float equality:** `0.1 + 0.2 == 0.3` → **false** (IEEE-754 rounding). **Fix:** `math.Abs(a-b) < epsilon` తో compare.
- **Money ని float లో store చేయడం** → rounding errors. **Fix:** integer cents (`int64`) లేదా decimal library వాడు.
- `int` **size assume చేయడం:** 32-bit vs 64-bit platform లో వేరు. Portable code కి `int64` explicit వాడు.
- **Overflow silent** – పెద్ద counters కి తగిన size ఎంచుకో లేదా `math/bits` overflow checks.

Key Points

- Numeric types size ని పేరులోనే చెప్తాయి; `int` / `uint` = platform word size.
- **No implicit conversion** – always explicit `T(value)` .
- Overflow = **silent wrap** (panic కాదు); constant overflow మాత్రం compile error.
- `float64` = default decimal; float equality ప్రమాదకరం.
- `byte` = `uint8` , `rune` = `int32` (aliases).
- Money = never float; integer cents వాడు.

Interview దృష్టి

Q: Go implicit type conversion ఎందుకు allow చేయదు? A: Silent bugs తగ్గించడానికి. `int` + `float` లాంటి mixed arithmetic లో precision loss లేదా sign errors అనుకోకుండా జరగకుండా, programmer explicitly convert చేయాలి – intent స్పష్టం అవుతుంది.

Q: Integer overflow అయితే Go ఏం చేస్తుంది? A: Panic కాదు; two's-complement wrap around అవుతుంది (uint8 255+1=0). కాబట్టి తగిన integer size ఎంచుకోవాలి. Constant expressions overflow

మాత్రం compile-time లో caught.

Named Types vs Type Aliases – `type X Y` vs `type X = Y`

Go లో `type` keyword కి **రెండు వేర్వేరు అర్థాలు** ఉన్నాయి – ఇది చాలా మంది గందరగోళపడే subtle concept.

```
// 1) DEFINED type (named type) — కొత్త, ప్రత్యేకమైన type పుడుతుంది
type Celsius float64      // Celsius కి underlying type float64, కానీ float64 కాదు
type UserID int           // UserID కి methods జోడించవచ్చు, type safety వస్తుంది

// 2) type ALIAS — అదే type కి కేవలం రెండో పేరు (కొత్త type కాదు)
type Temp = float64       // Temp అంటే అక్షరాలా float64 — identical
```

అంశం	**Defined type** `type X Y`	**Alias** `type X = Y`
కొత్త type పుడుతుందా?	అవును (distinct)	కాదు (అదే type, రెండో పేరు)
Method జోడించగలమా?	అవును (`func (c Celsius) ...`)	కాదు (alias మీద method నిషేధం)
Auto-convert అవుతుందా?	కాదు – explicit `float64(c)` కావాలి	అవును (identical types)
ఎప్పుడు	Domain types, type safety, methods	Refactoring/migration లో package మార్చేటప్పుడు compatibility

```
type Celsius float64
func (c Celsius) ToF() Celsius { return c*9/5 + 32 }   // method on named type

var c Celsius = 36.6
var f float64 = 36.6
// c = f           // ❌ compile error — Celsius ≠ float64 (type safety!)
c = Celsius(f)     // ✅ explicit conversion
fmt.Println(c.ToF())

// byte = uint8, rune = int32, any = interface{} — ఇవి stdlib లో నిజమైన ALIASES
```

గుర్తు: `=` ఉంటే **alias** (అదే type), `=` లేకపోతే **కొత్త defined type**. Method attach చేయాలంటే defined type కావాలి. `byte` / `rune` / `any` నిజమైన aliases (అందుకే `byte` == `uint8`). Alias యొక్క అసలు use case: పెద్ద codebase లో ఒక type ని కొత్త package కి క్రమంగా తరలించేటప్పుడు (`type OldName = newpkg.NewName`) పాత code విరగకుండా ఉంచడం.

6. Strings, byte, rune, UTF-8 Deep Dive

వివరణ

Go లో **string = read-only (immutable) slice of bytes.** అది Java లా UTF-16 కాదు – **UTF-8 encoded bytes.** ఇది Go designers (Ken Thompson & Rob Pike UTF-8 ని కనిపెట్టారు!) చేసిన ఉద్దేశపూర్వక నిర్ణయం. దీని వల్ల రెండు కీలక concepts:

- `byte` (= `uint8`) – ఒక్క byte. `len(s)` bytes సంఖ్య ఇస్తుంది, characters కాదు.
- `rune` (= `int32`) – ఒక Unicode **code point** (ఒక "character"). తెలుగు/emoji అక్షరాలు multiple bytes కావచ్చు.

```
s := "నమస్తే"
fmt.Println(len(s))                // 18 (bytes — తెలుగు అక్షరం 3 bytes చొప్పున)
fmt.Println(utf8.RuneCountInString(s)) // 6 (actual runes/characters)
```

string internal layout

String internally రెండు words: **{pointer to bytes, length}.** ఇది immutable – modify చేయలేం. `s[i]` ఒక **byte** ఇస్తుంది (rune కాదు!).

```
s := "hello"
fmt.Println(s[0])          // 104  (byte value of 'h', కాదు "h")
fmt.Println(string(s[0])) // h
// s[0] = 'H'             // ❌ compile error: strings immutable
```

range over string = runes (bytes కాదు!)

`for i, r := range s` – index `i` **byte offset**, `r` ఒక **rune**. Range automatically UTF-8 decode చేస్తుంది.

```
s := "నమస్"
for i, r := range s {
        fmt.Printf("byte %d: rune %c (%d)\n", i, r, r)
}
// byte offsets 0, 3, 6, 9,... — ఒక్కో rune 3 bytes
// కానీ s[1] అంటే మధ్య byte — invalid character!
```

Bytes vs Runes conversion

```
s := "café"
b := []byte(s)  // UTF-8 bytes: [99 97 102 195 169] — len 5
r := []rune(s)  // runes: [99 97 102 233] — len 4 (é = 1 rune)

fmt.Println(len(b), len(r)) // 5 4
fmt.Println(string(r[3]))   // é
```

రూల్: **byte-level ops** (raw data, I/O) → `[]byte`. **Character-level ops** (i-th character, reverse) → `[]rune`.

Strings ని modify చేయడం – []byte లేదా strings.Builder

Strings immutable కాబట్టి "modify" అంటే కొత్తది build చేయడం. Loop లో `+=` **O(n²)** (ప్రతిసారి కొత్త string). బదులు `strings.Builder`:

```
var sb strings.Builder
for i := 0; i < 1000; i++ {
        sb.WriteString("x")   // internal []byte కి append — O(n) total
}
result := sb.String()     // చివర్లో ఒక్కసారి string
```

Essential string packages

```
strings.Contains("golang", "go")       // true
strings.Split("a,b,c", ",")            // ["a" "b" "c"]
strings.Join([]string{"a", "b"}, "-")  // "a-b"
strings.ReplaceAll("aaa", "a", "b")    // "bbb"
strings.ToUpper("go")                  // "GO"
strings.TrimSpace("  hi  ")            // "hi"
strings.HasPrefix("golang", "go")      // true

n, _ := strconv.Atoi("42")             // string → int
s := strconv.Itoa(42)                  // int → string
f, _ := strconv.ParseFloat("3.14", 64) // string → float64
b, _ := strconv.ParseBool("true")      // string → bool

utf8.RuneCountInString("నమస్తే")       // 6
utf8.ValidString("...")                // valid UTF-8?
unicode.IsLetter('A')                  // true
unicode.IsDigit('5')                   // true
```

Real-life Scenario

> **string = ఒక పొడవాటి రైలు, byte = ఒక్క wheel, rune = ఒక్క bogie (compartment).** ఇంగ్లీష్ అక్షరాలకి ఒక్క bogie కి ఒక్క wheel (1 byte = 1 char). కానీ తెలుగు/emoji bogies పెద్దవి – ఒక్క bogie కి 3-4 wheels (bytes). నువ్వు "5వ wheel" (`s[5]` byte) అడిగితే అది ఏదో bogie మధ్యలో ఉండవచ్చు – అర్థంలేని byte. "5వ bogie" (rune) కావాలంటే `[]rune` లేదా `range` వాడాలి.

Gotchas (సాధారణ తప్పులు)

- `len(s)` **= bytes, characters కాదు!** Multi-byte scripts లో `len("నమస్తే")` = 18. **Fix:** `utf8.RuneCountInString`.
- `s[i]` **= byte, rune కాదు.** Non-ASCII string ని `s[i]` తో iterate చేస్తే garbage. **Fix:** `range` లేదా `[]rune`.
- **Loop లో** `+=` **string concat = O(n²). Fix:** `strings.Builder`.

- `string(65)` = **"A"** (rune→string), కానీ `string(someInt)` బహుశా నువ్వు అనుకున్నది కాదు – `strconv.Itoa` వాడు number→string కి.
- **Byte offset లో string slice చేస్తే** multi-byte character మధ్యలో కట్ అవ్వవచ్చు → invalid UTF-8.

Key Points

- string = **immutable UTF-8 bytes**; internal `{ptr, len}`.
- `byte` = uint8 (raw byte); `rune` = int32 (Unicode code point).
- `len(s)` = **bytes**; `utf8.RuneCountInString` = characters.
- `s[i]` = byte; `range s` = runes (auto UTF-8 decode).
- Character ops → `[]rune`; byte/IO ops → `[]byte`.
- Concat in loop → **strings.Builder** (O(n) కాదు O(n^2)).

Interview దృష్టి

Q: Go string internally ఎలా store అవుతుంది? `len` ఏం return చేస్తుంది? A: String = immutable, `{pointer, length}` header ఉన్న UTF-8 byte slice. `len(s)` bytes సంఖ్య ఇస్తుంది, characters కాదు. Multi-byte characters (తెలుగు, emoji) కి rune count కావాలంటే `utf8.RuneCountInString`.

Q: byte vs rune? A: `byte` = uint8 = ఒక raw byte. `rune` = int32 = ఒక Unicode code point (ఒక logical character, 1-4 bytes UTF-8). `range` string runes ఇస్తుంది; index `s[i]` byte ఇస్తుంది.

---

7. Operators & Control Flow

వివరణ

Go control flow చాలా minimal – `if`, `for`, `switch` మాత్రమే (while/do-while లేవు; అన్నీ `for` తోనే). ఇదే Go simplicity. కొన్ని ప్రత్యేకతలు: **braces తప్పనిసరి**, condition చుట్టూ parentheses **లేవు**, opening brace **అదే line** లో ఉండాలి (gofmt enforce).

Operators

```
// Arithmetic:  + - * / %          (% integers మాత్రమే)
// Comparison:  == != < <= > >=
// Logical:     && || !            (short-circuit)
// Bitwise:     & | ^ &^ << >>     (^ = XOR/NOT, &^ = AND NOT/bit clear)
// Assignment:  = += -= *= ... <<= &^=
// ++ / --      statements మాత్రమే (expressions కాదు): i++ ✓, x = i++ ❌
```

Go లో `++` / `--` **statements**, expressions కాదు – `j = i++` invalid. `&^` (AND NOT) = ఒక ప్రత్యేక bit-clear operator.

if – with optional init statement

```
if x > 10 {
	fmt.Println("పెద్దది")
} else if x > 5 {
	fmt.Println("మధ్యస్థం")
} else {
	fmt.Println("చిన్నది")
}

// init statement తో — err ని if scope కే పరిమితం (idiomatic!)
if err := doSomething(); err != nil {
	return err
}
// ఇక్కడ err scope బయట లేదు — శుభ్రం

// map lookup తో common pattern
if val, ok := m["key"]; ok {
	fmt.Println(val)
}
```

init statement (`if x := f(); cond`) – variable ని `if` / `else` scope కే పరిమితం చేస్తుంది. Error handling లో అత్యంత idiomatic.

for – Go యొక్క ఏకైక loop (4 రూపాలు)

```
// 1. Classic C-style
for i := 0; i < 5; i++ {
	fmt.Println(i)
}

// 2. While-style (condition only)
n := 10
for n > 0 {
	n--
}

// 3. Infinite loop
for {
	// break తో బయటకు
	break
}

// 4. range — slices, maps, strings, channels
nums := []int{10, 20, 30}
for i, v := range nums {   // index, value
	fmt.Println(i, v)
}
for _, v := range nums {   // index skip
	fmt.Println(v)
}

m := map[string]int{"a": 1, "b": 2}
for k, v := range m {      // key, value (random order!)
	fmt.Println(k, v)
}
```

```
for i := range 5 {          // Go 1.22+ : 0,1,2,3,4 (range over int)
        fmt.Println(i)
}
```

switch – powerful & clean

Go `switch` C కంటే powerful. **Auto-break** (fallthrough default కాదు), conditions అవసరం లేదు.

```
// 1. Expression switch (auto-break — fallthrough అవసరం లేదు)
switch day {
case "Sat", "Sun":       // multiple values ఒక్క case లో
        fmt.Println("weekend")
case "Mon":
        fmt.Println("Monday")
default:
        fmt.Println("weekday")
}

// 2. Tagless switch (if-else chain ని శుభ్రంగా)
switch {
case score >= 90:
        grade = "A"
case score >= 80:
        grade = "B"
default:
        grade = "C"
}

// 3. switch with init
switch os := runtime.GOOS; os {
case "darwin":
        fmt.Println("macOS")
case "linux":
        fmt.Println("Linux")
}

// 4. fallthrough — తర్వాతి case కి explicitly పోవడం
switch n {
case 1:
        fmt.Println("one")
        fallthrough        // కింది case కూడా execute
case 2:
        fmt.Println("two")
}
// n=1 → "one" మరియు "two"

// 5. type switch (interfaces తో — Topic 15 లో deep)
switch v := i.(type) {
case int:
        fmt.Println("int:", v)
case string:
        fmt.Println("string:", v)
}
```

goto, labels, labeled break/continue

`goto` ఉంది కానీ అరుదు. **Labels** nested loops నుండి బయటకు రావడానికి ఉపయోగం.

```
outer:
for i := 0; i < 3; i++ {
	for j := 0; j < 3; j++ {
		if i*j > 2 {
			break outer      // బయటి loop నుండి కూడా బయటకు
		}
		if j == 1 {
			continue outer   // బయటి loop యొక్క తర్వాతి iteration
		}
	}
}

// goto — cleanup patterns లో అరుదుగా
	i := 0
loop:
	if i < 3 {
		fmt.Println(i)
		i++
		goto loop
	}
```

Real-life Scenario

> `switch` **fallthrough = దారిలో టోల్ గేట్లు.** సాధారణంగా ఒక గేటు దాటాక ఆగిపోతావు (auto-break). కానీ `fallthrough` అంటే "next గేటు కూడా దాటిపో" అని explicit permission. Go C కి రివర్స్: C లో default fallthrough (అందరూ మర్చిపోయి bugs), Go లో default stop (safe), fallthrough కావాలంటే అడగాలి.
>
> **Labeled break = బహుళ అంతస్తుల భవనం నుండి fire exit.** సాధారణ break ఒక్క గదిలోంచి బయటకు (inner loop). `break outer` మొత్తం భవనం (nested loops) నుండి నేరుగా బయటకు.

Gotchas (సాధారణ తప్పులు)

- **తప్పు:** C అనుకుని `switch` case చివర `break` రాయడం అనవసరం — Go auto-break. `fallthrough` explicit కావాలి.
- **తప్పు:** `if (x > 5)` parentheses రాయడం — gofmt తీసేస్తుంది; condition parens Go లో లేవు.
- **తప్పు:** opening brace ని కొత్త line లో పెట్టడం (`func f()\n{`) → compile error (auto semicolon insertion). Brace అదే line లో ఉండాలి.
- **తప్పు:** map range order stable అనుకోవడం — **random**. Sorted కావాలంటే keys sort చేయాలి.

Key Points

- ఒక్క loop keyword: `for` (while/do-while లేవు) — 4 forms.
- `if` / `switch` **init statement** తో scoped variables (idiomatic error handling).
- Condition parens లేవు; braces తప్పనిసరి; brace అదే line.

- `switch` **auto-breaks**; `fallthrough` explicit; tagless switch = if-else chain.
- **Labeled break/continue** – nested loops control.
- `for i := range n` (Go 1.22+) integers మీద range.

Interview దృష్టి

Q: Go `switch` C `switch` కంటే ఎలా వేరు? A: (1) Auto-break – ప్రతి case తర్వాత automatic break, fall-through default కాదు (`fallthrough` explicit). (2) Cases integers మాత్రమే కాదు, ఏ type అయినా, strings కూడా. (3) Tagless switch = శుభ్రమైన if-else chain. (4) Type switch interfaces తో పని చేస్తుంది.

Q: init statement in if ఎందుకు idiomatic? A: `if err := f(); err != nil` – `err` scope ని ఆ if block కే పరిమితం చేస్తుంది, outer scope pollute చేయదు, shadowing bugs తగ్గిస్తుంది.

Part 2 – Types & Data Structures

> ఈ Part Go యొక్క **గుండె.** Arrays, slices, maps, structs, pointers, functions, methods, interfaces, generics, errors – వీటిని లోతుగా అర్థం చేసుకుంటే Go నీకు వశమవుతుంది. ముఖ్యంగా **slices, interfaces** – interview లో అత్యధికంగా అడిగేవి.

---

8. Arrays

వివరణ

Array = fixed-size, same-type elements యొక్క వరుస. Go లో array యొక్క **size type లో భాగం** – `[3]int` మరియు `[5]int` **వేర్వేరు types!** ఇదే array ని rigid గా చేస్తుంది; అందుకే practice లో slices ఎక్కువ వాడతారు. కానీ arrays ని అర్థం చేసుకోవడం slices అర్థం చేసుకోవడానికి పునాది.

```
var a [3]int             // [0 0 0] — zero values
b := [3]int{1, 2, 3}     // explicit
c := [...]int{1, 2, 3, 4} // [...] → compiler size లెక్కిస్తుంది (4)
d := [5]int{1: 10, 3: 30} // index-based: [0 10 0 30 0]

fmt.Println(len(a))      // 3
```

Value semantics – అత్యంత కీలకం

Go arrays **value types** – assign చేస్తే లేదా function కి pass చేస్తే **మొత్తం array copy అవుతుంది** (Java/C లా reference కాదు!). ఇది Go beginners కి పెద్ద surprise.

```
a := [3]int{1, 2, 3}
b := a          // మొత్తం copy!
b[0] = 99
fmt.Println(a)  // [1 2 3] — a మారలేదు
fmt.Println(b)  // [99 2 3]

func modify(arr [3]int) { arr[0] = 100 } // copy మీద పని — original safe
modify(a)
fmt.Println(a)  // [1 2 3] — మారలేదు!
```

పెద్ద array function కి pass చేస్తే ప్రతిసారి copy – costly. అందుకే slices (or `*[N]T` pointer) వాడతారు.

Comparison

Same-type arrays `==` తో compare చేయవచ్చు (elements comparable అయితే). Slices compare చేయలేం (compile error) – ఇది array యొక్క ఒక ప్రత్యేకత.

```
a := [3]int{1, 2, 3}
b := [3]int{1, 2, 3}
fmt.Println(a == b)  // true — element-wise
```

Multi-dimensional

```
var grid [3][4]int          // 3 rows, 4 cols
grid[1][2] = 5
matrix := [2][2]int{{1, 2}, {3, 4}}
```

Real-life Scenario

> **Array = ఒక అపార్ట్‌మెంట్ భవనంలో fixed floor plan (ఖచ్చితంగా 3 అంతస్తులు).** అంతస్తుల సంఖ్య design లోనే fixed – తర్వాత పెంచలేవు. అలాగే `[3]int` కి 3 slots fixed.
>
> **Value copy = ఆ భవనం యొక్క photocopy blueprint ఇవ్వడం.** ఎవరైనా blueprint మీద rooms మార్చినా, అసలు భవనం మారదు. అందుకే Go array ని function కి పంపితే original safe – కానీ blueprint పెద్దదైతే photocopy ఖరీదు (memory copy).

Gotchas (సాధారణ తప్పులు)

- **తప్పు:** array function కి pass చేసి, function లోని మార్పులు caller లో కనిపిస్తాయని అనుకోవడం. **సరి:** value copy – మార్పులు కావాలంటే `*[N]T` pointer లేదా slice వాడు.
- **తప్పు:** `[3]int` కి 4 elements ఇవ్వడం → compile error. Size fixed.
- **తప్పు:** వేర్వేరు size arrays ని compare/assign చేయడం → వేర్వేరు types, compile error.

Key Points

- Array size = **type లో భాగం**; `[3]int` ≠ `[5]int`.
- **Value semantics** – assign/pass = full copy (reference కాదు).
- `[...]T{}` = compiler size లెక్కిస్తుంది.
- Same-type arrays `==` compare చేయవచ్చు (slices చేయలేం).
- Practice లో **slices** ఎక్కువ; arrays fixed-size cases (crypto hashes, lookup tables) లో.

Interview దృష్టి

Q: Go arrays value types నా reference types నా? A: Value types. Assign చేసినా, function కి pass చేసినా మొత్తం array copy అవుతుంది. Function లోని మార్పులు original ని affect చేయవు. పెద్ద arrays కి copy ఖరీదు కాబట్టి slices లేదా pointers వాడతారు.

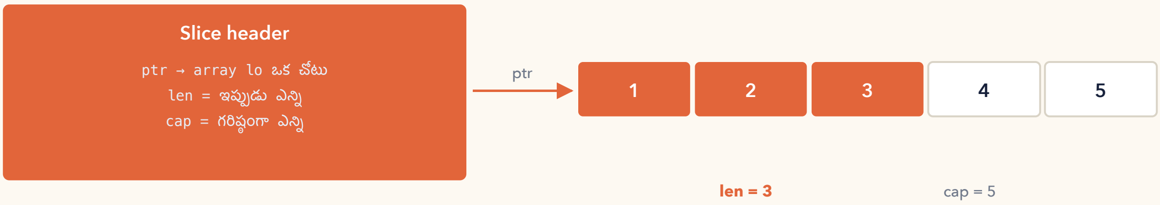
Q: Array vs slice – ఎప్పుడు array? A: Size compile-time లో fixed & known అయినప్పుడు (ఉదా. `[32]byte` SHA-256 hash, [256]int lookup table). లేకపోతే almost always slice. Arrays comparable, stack-allocatable, కానీ inflexible.

---

9. Slices – DEEP Dive (అత్యంత ముఖ్యం)

SLICE · HEADER, BACKING ARRAY, మరియు APPEND యొక్క ఉచ్చు

SLICE = ఒక HEADER – మూడు fields మాత్రమే



ఎందుకు slices ప్రమాదకరం కావొచ్చు

s2 := s1[:2] – రెండూ **ఒకే array** ని చూపుతాయి. s2 lo మార్చితే s1 lo కూడా మారుతుంది.
append() – cap మిగిలి ఉంటే అదే array lo రాస్తుంది (ఇతరులకి కనిపిస్తుంది);
cap నిండితే కొత్త array కి copy చేస్తుంది (ఇక కనిపించదు). ఈ అస్థిరతే bugs కి మూలం.

సురక్షితమైన అలవాటు

నిజంగా copy కావాలంటే: **copy(dst, src)** లేదా three-index slice **s[a:b:b]** (cap ని కూడా కత్తిరించడం).
Function కి slice ఇచ్చేటప్పుడు – "ఇది మార్చొచ్చా?" అని ఆలోచించడం.

వివరణ

Slice = ఒక underlying array మీద dynamic, resizable "window." Go లో ఇది అత్యధికంగా వాడే data structure. Slice ఒక **3-word header:**

```
slice header = { ptr → underlying array , len , cap }
```

- `ptr` – underlying array యొక్క ఒక element కి pointer.
- `len` – ఇప్పుడు slice లో ఎన్ని elements కనిపిస్తాయి (`len(s)`).
- `cap` – underlying array లో ptr నుండి చివరి వరకు ఎన్ని slots ఉన్నాయి (`cap(s)`).

Array value type; **slice reference-like** (header చిన్నది copy అవుతుంది, కానీ అది అదే underlying array కి point చేస్తుంది). ఇదే slice యొక్క శక్తి మరియు అనేక bugs కి మూలం.

```
s := []int{10, 20, 30}          // len=3, cap=3
s2 := make([]int, 3, 10)        // len=3, cap=10 (pre-allocated)
var s3 []int                     // nil slice: ptr=nil, len=0, cap=0
```

make & the three fields

```
s := make([]int, 2, 5)
fmt.Println(len(s), cap(s)) // 2 5
// underlying array: [0 0 _ _ _], slice చూపేది మొదటి 2
```

append & growth (doubling) – internals

`append` slice చివర element జోడిస్తుంది. `len < cap` అయితే అదే array లో పెడుతుంది (fast). `len == cap` అయితే **కొత్త, పెద్ద array allocate చేసి, పాత data copy చేసి,** కొత్త header return చేస్తుంది.

```
s := make([]int, 0, 2)
fmt.Printf("len=%d cap=%d\n", len(s), cap(s)) // 0 2
s = append(s, 1)  // len=1 cap=2
s = append(s, 2)  // len=2 cap=2 (full)
s = append(s, 3)  // len=3 cap=4 → కొత్త array! (grew)
```

Growth strategy: cap చిన్నది (< 256) అయితే **doubling** (2×); పెద్దది అయితే ~1.25× (memory వృథా తగ్గించడానికి). ఖచ్చితమైన factor Go version మీద ఆధారపడుతుంది కానీ amortized O(1) append guarantee.

> **అందుకే `append` return value ని ఎప్పుడూ తిరిగి assign చేయాలి:** `s = append(s, x)`. లేకపోతే grow అయినప్పుడు కొత్త header మిస్ అవుతుంది.

Aliasing / shared backing array — the #1 gotcha

రెండు slices ఒకే underlying array ని share చేయగలవు. ఒకదాంట్లో మార్పు మరొకదాంట్లో కనిపిస్తుంది!

```
a := []int{1, 2, 3, 4, 5}
b := a[1:3]        // b = [2 3], కానీ a తో అదే array share
b[0] = 99
fmt.Println(a)     // [1 99 3 4 5] — a కూడా మారింది!
fmt.Println(b)     // [99 3]
fmt.Println(len(b), cap(b)) // 2 4 (b index 1 నుండి array చివరి వరకు)
```

Slice expressions

```
s := []int{0, 1, 2, 3, 4, 5}
s[1:4]    // [1 2 3] — index 1..3
s[:3]     // [0 1 2]
s[2:]     // [2 3 4 5]
s[:]      // మొత్తం

// full slice expression: s[low:high:max] — cap ని limit చేస్తుంది!
t := s[1:3:3]  // len=2, cap=2 (max index 3 వరకే)
// ఇది append aliasing bug ని ఆపుతుంది — కింద చూడు
```

copy() — safe duplication

Aliasing తప్పించాలంటే `copy` తో deep copy:

```
src := []int{1, 2, 3}
dst := make([]int, len(src))
n := copy(dst, src)   // n = copied elements count (min of lens)
dst[0] = 99
fmt.Println(src)      // [1 2 3] — safe, independent
```

nil vs empty slice

```
var a []int          // nil slice: a == nil → true, len 0, cap 0
b := []int{}         // empty (non-nil): b == nil → false, len 0
c := make([]int, 0)  // empty (non-nil)

// రెండింటిపైనా append, len, range పని చేస్తాయి — behavior ఒకటే
a = append(a, 1)     // nil slice కి append safe!
```

రూల్: function నుండి return చేసేటప్పుడు `nil` slice idiomatic (empty కి extra allocation అవసరం లేదు). కానీ JSON marshal లో nil → `null`, empty → `[]` — తేడా ఉంటుంది.

GOTCHA 1: append aliasing bug (silent data corruption)

```
a := []int{1, 2, 3, 4, 5}
b := a[:2]                 // len=2, cap=5 (అదే array!)
b = append(b, 99)         // cap ఉంది కాబట్టి అదే array లో index 2 overwrite
fmt.Println(a)            // [1 2 99 4 5] — a[2] corrupted!
```

Fix: full slice expression తో cap limit — `b := a[:2:2]`. ఇప్పుడు append కొత్త array allocate చేస్తుంది, `a` safe.

GOTCHA 2: loop variable capture (Go 1.22 కి ముందు)

```
// Go 1.21 & earlier — BUG:
funcs := []func(){}
for _, v := range []int{1, 2, 3} {
        funcs = append(funcs, func() { fmt.Println(v) })
}
for _, f := range funcs { f() }
// 1.21-: 3 3 3 (అందరూ చివరి v ని share)
// 1.22+: 1 2 3 (per-iteration variable — fixed!)
```

Go 1.22 లో loop variable ప్రతి iteration కి కొత్తది అయింది. పాత Go లో `v := v` shadow trick కావాలి.

Real-life Scenario

> **Slice = ఒక పొడవాటి పార్కింగ్ లాట్ (underlying array) మీద ఒక "reserved zone" boundary tape.** Tape (slice header) చిన్నది — start pointer, ఎన్ని slots (len), zone చివరి వరకు total (cap). రెండు జట్లు ఒకే lot మీద overlapping tapes వేస్తే (aliasing), ఒక జట్టు car మార్చితే మరో జట్టుకి కనిపిస్తుంది. `copy` = వేరే lot లో సొంత cars పెట్టుకోవడం — independent.
>
> **append growth = zone నిండిపోతే** valet పెద్ద lot కి అన్ని cars ని move చేసి, కొత్త address ఇస్తాడు (కొత్త array). పాత address (`s`) ని update చేసుకోకపోతే నీ cars ఎక్కడున్నాయో మర్చిపోతావు — అందుకే `s = append(...)`.

Slice comparison table

అంశం	Array	Slice
Size	Fixed (type లో భాగం)	Dynamic (grow అవుతుంది)
Header	Data నే	`{ptr, len, cap}`
Assign/pass	Full copy (value)	Header copy (అదే array share)
`==` compare	✅ (comparable elems)	❌ compile error (nil తో మాత్రమే)
`nil`	సాధ్యం కాదు	సాధ్యం (`var s []T`)
append	❌	✅

Gotchas (సారాంశం)

- `append` **return ని re-assign** చేయకపోతే grow అయినప్పుడు data మిస్.
- **Sub-slice append** shared array ని corrupt చేయవచ్చు → full slice expr `a[i:j:j]`.
- **Slicing పెద్ద array ని retain చేస్తుంది** – పెద్ద file చదివి ఒక్క byte slice ఉంచుకుంటే మొత్తం array GC అవ్వదు (memory leak). Fix: `copy` తో fresh slice.
- **nil vs empty:** logic లో ఒకటే కానీ JSON లో `null` vs `[]`.

Key Points

- Slice = `{ptr, len, cap}` header మీద underlying array window.
- `append`: cap ఉంటే in-place; లేకపోతే కొత్త array (double/1.25×) + copy. **ఎప్పుడూ re-assign.**
- Slices **share backing arrays** → aliasing bugs. `copy` = independent.
- **Full slice expr** `s[i:j:k]` cap limit → append aliasing ని ఆపుతుంది.
- `nil` slice = usable (append/len/range OK); `== nil` true.
- Loop var capture: **Go 1.22+** లో fixed (per-iteration).

Interview దృష్టి

Q: Slice internally ఎలా ఉంటుంది? Array కంటే ఎలా వేరు? A: Slice = 3-word header `{pointer, len, cap}` ఒక underlying array కి. Array value type (full copy); slice header copy అవుతుంది కానీ అదే array share చేస్తుంది (reference-like). Slice grow అవుతుంది (append), array కాదు.

Q: `append` ఎలా పని చేస్తుంది? Growth? A: `len < cap` అయితే in-place, O(1). `len == cap` అయితే కొత్త పెద్ద array allocate (చిన్నప్పుడు 2×, పెద్దప్పుడు ~1.25×), పాత data copy, కొత్త header return. అందుకే `s = append(s, x)` గా re-assign చేయాలి. Amortized O(1).

Q: ఈ code output ఏమిటి?

```
a := []int{1,2,3,4}; b := a[:2]; b = append(b, 99); fmt.Println(a)
```

A: `[1 2 99 4]`. `b` కి cap=4 (a తో share), append index 2 ని in-place overwrite చేసింది. Fix: `b := a[:2:2]`.

---

10. Maps

వివరణ

Map = hash table – key → value associations. Go లో built-in `map[KeyType]ValueType`. Keys **comparable** అయి ఉండాలి (`==` support చేసేవి – ints, strings, structs of comparables, pointers; slices/maps/functions keys కాలేవు).

```
m := map[string]int{"apple": 3, "banana": 5}  // literal
m2 := make(map[string]int)                     // empty, ready
var m3 map[string]int                          // nil map — read OK, write PANIC!

m["cherry"] = 7          // insert/update
val := m["apple"]        // read (లేకపోతే zero value)
delete(m, "banana")      // remove
fmt.Println(len(m))      // entries count
```

comma-ok idiom – "ఉందా, లేదా zero?"

Missing key కి map **zero value** ఇస్తుంది – key లేదని vs value 0 అని తేడా తెలియదు. `comma-ok` దీన్ని పరిష్కరిస్తుంది:

```
val, ok := m["apple"]
if ok {
        fmt.Println("ఉంది:", val)
} else {
        fmt.Println("లేదు")
}

// count map లో common
if _, exists := seen[x]; !exists {
        seen[x] = true
}
```

Iteration – random order (ఉద్దేశపూర్వకం!)

Map range **ప్రతిసారి వేర్వేరు order** ఇస్తుంది – Go దీన్ని **ఉద్దేశపూర్వకంగా randomize** చేస్తుంది, developers order మీద ఆధారపడకుండా. Sorted output కావాలంటే keys తీసి sort చేయాలి.

```
for k, v := range m {   // random order ప్రతి run కి!
        fmt.Println(k, v)
}

// sorted iteration
keys := make([]string, 0, len(m))
for k := range m {
```

```
        keys = append(keys, k)
}
sort.Strings(keys)
for _, k := range keys {
        fmt.Println(k, m[k])
}
```

Internals – లోపల ఏం జరుగుతుంది

Go map = **hash table with buckets.** ప్రతి bucket 8 key-value pairs దాచుతుంది (`bmap`). Key ని hash చేసి, hash యొక్క low bits bucket ని ఎంచుకుంటాయి, high bits (top hash) bucket లోపల fast match కి. Bucket నిండితే **overflow bucket** chain అవుతుంది. Load factor (~6.5 avg per bucket) దాటితే map **grows** – buckets రెట్టింపు అవుతాయి, entries క్రమంగా (incrementally) కొత్త buckets కి rehash/evacuate అవుతాయి.

- Map value ఒక `*hmap` (pointer to header) – అందుకే map ని function కి pass చేస్తే **అదే map** share అవుతుంది (slice header లా కాదు, ఇది నిజంగా reference-like).
- Random iteration = range మొదలయ్యేటప్పుడు random bucket + offset నుండి మొదలుపెడతారు.

Concurrency – maps NOT safe

Concurrent read + write panic చేస్తుంది ("concurrent map writes"/"read and write"). Go runtime దీన్ని detect చేసి crash చేస్తుంది (silent corruption కంటే మంచిది). Fix: `sync.Mutex` / `sync.RWMutex` లేదా `sync.Map` (Topic 23).

```
var mu sync.Mutex
mu.Lock()
m[key] = value
mu.Unlock()
```

Map of struct – addressability gotcha

Map values **addressable కాదు** – struct field ని నేరుగా modify చేయలేం:

```
type P struct{ X int }
m := map[string]P{"a": {X: 1}}
// m["a"].X = 5   // ❌ compile error: cannot assign
m["a"] = P{X: 5}  // ✅ మొత్తం value replace
// లేదా map[string]*P వాడు
mp := map[string]*P{"a": {X: 1}}
mp["a"].X = 5     // ✅ pointer ద్వారా
```

Real-life Scenario

> **Map = ఒక భారీ లైబ్రరీ లో పుస్తకాల catalog.** పుస్తకం పేరు (key) ఇస్తే, అది ఏ అరలో (bucket) ఉందో hash చెప్తుంది – వెతకనవసరం లేదు, O(1). ప్రతి అరలో 8 పుస్తకాలు; అర నిండితే పక్కన overflow అర. లైబ్రరీ చాలా నిండితే (load factor), staff రెట్టింపు అరలున్న కొత్త విభాగానికి పుస్తకాలు క్రమంగా మారుస్తారు (incremental grow).
>
> **Random iteration = ప్రతిసారి catalog ని వేరే అర నుండి చదవడం** – నువ్వు "అక్షర క్రమం" మీద ఆధారపడకూడదని staff ఉద్దేశపూర్వకంగా చేస్తారు.

Gotchas (సాధారణ తప్పులు)

- **nil map కి write → panic!** `var m map[string]int; m["x"]=1` crash. **Fix:** `make` లేదా literal తో initialize. (Read from nil OK – zero value.)
- **Missing key = zero value, error కాదు.** ఉందో లేదో తెలియాలంటే **comma-ok**.
- **Iteration order random** – దాని మీద ఆధారపడకు; sort చేయి.
- **Concurrent access = panic.** Mutex/sync.Map వాడు.
- **Map value struct field నేరుగా assign చేయలేం** – pointer values లేదా full replace.
- **Map element యొక్క address తీయలేం** (`&m[k]` ❌) – grow అయితే memory move అవుతుంది.

Key Points

- Map = hash table, buckets (8 pairs each) + overflow; grows at load factor.
- Value = `*hmap` → function కి pass చేస్తే **నిజంగా shared** (reference-like).
- **nil map read OK, write panic.** `make` /literal తో init.
- Missing key → zero value; **comma-ok** to distinguish.
- **Random iteration order** (intentional).
- **Not concurrency-safe** – mutex/sync.Map.
- Map values not addressable – pointer values for mutation.

Interview దృష్టి

Q: Go map internally ఎలా? Grow ఎప్పుడు? A: Hash table with buckets; ప్రతి bucket 8 key-value pairs + overflow chain. Key hash యొక్క low bits bucket ఎంచుకుంటాయి, high bits bucket లోపల fast compare. Load factor (~6.5) దాటితే buckets రెట్టింపు + incremental evacuation. Value ఒక pointer కాబట్టి maps reference-like.

Q: nil map తో ఏం జరుగుతుంది? A: Read from nil map = zero value (safe). Write to nil map = **panic**. అందుకే వాడేముందు `make` లేదా literal తో initialize చేయాలి.

Q: Map iteration order guaranteed నా? A: లేదు – ఉద్దేశపూర్వకంగా randomized (ప్రతి range random bucket నుండి మొదలు). Deterministic order కావాలంటే keys collect చేసి sort చేయాలి.

`clear()` builtin – map/slice ని ఖాళీ చేయడం (Go 1.21+)

Go 1.21 లో వచ్చిన `clear()` builtin – map లోని **అన్ని entries ని తీసేస్తుంది** (అదే map ని reuse చేస్తూ, కొత్తది allocate చేయకుండా), లేదా slice లోని అన్ని elements ని zero value కి set చేస్తుంది.

```
m := map[string]int{"a": 1, "b": 2, "c": 3}
clear(m)                  // అన్ని keys తీసేస్తుంది
fmt.Println(len(m))       // 0  (అదే map, re-allocate కాదు – memory reuse)

// పాత విధానం: loop తో delete (ఇప్పటికీ valid, కానీ clear చిన్నది)
// for k := range m { delete(m, k) }

s := []int{1, 2, 3}
clear(s)                  // slice లో అన్ని zero → [0 0 0]  (len మారదు!)
fmt.Println(s, len(s))    // [0 0 0] 3
```

Target	`clear()` ఏం చేస్తుంది
Map	అన్ని entries delete (len → 0), అదే backing map reuse
Slice	అన్ని elements ని zero value కి (len/cap **మారవు**)

> **Gotcha:** slice కి `clear(s)` elements ని sunna చేస్తుంది కానీ length తగ్గించదు – length 0 చేయాలంటే `s = s[:0]`. Map ను reuse చేయాలనుకున్నప్పుడు (hot loop లో re-allocation తప్పించడానికి) `clear(m)` ఉత్తమం. NaN keys ఉన్న maps ని loop-delete తో పూర్తిగా clear చేయలేము – `clear()` వాటిని కూడా తీసేస్తుంది (ఇది clear() ఉనికికి ఒక అసలు కారణం).

---

11. Structs

వివరణ

Struct = named fields యొక్క సముదాయం – Go లో custom data types కి పునాది. Classes లేవు; structs + methods + interfaces = Go's OOP. Struct value type (array లా copy semantics).

```
type User struct {
        ID    int
        Name  string
        Email string
        Age   int
}

// అనేక initialization మార్గాలు
u1 := User{1, "Surya", "s@x.com", 30}        // positional (fragile – avoid)
u2 := User{ID: 1, Name: "Surya"}             // named (idiomatic; missing → zero)
u3 := User{}                                  // అన్ని zero values
var u4 User                                   // అదే – zero struct
p := &User{Name: "Ravi"}                      // pointer to struct
```

```
fmt.Println(u2.Name)   // field access
u2.Age = 31            // mutation
```

Embedding (composition – inheritance కి బదులు)

Go లో inheritance లేదు; **embedding** (anonymous field) తో composition. Embedded struct యొక్క fields/methods **promoted** అవుతాయి (నేరుగా access).

```
type Address struct {
	City, Zip string
}
type Employee struct {
	Name    string
	Address // embedded (anonymous) — Address fields promoted
	Salary  int
}

e := Employee{Name: "Ravi", Address: Address{City: "Hyd", Zip: "500001"}, Salary:
100000}
fmt.Println(e.City)         // promoted! (e.Address.City కూడా works)
fmt.Println(e.Address.Zip)  // explicit path కూడా
```

(Embedding ని Topic 16 లో లోతుగా చూస్తాం.)

Struct tags – metadata for reflection

Field తర్వాత backtick string = **tag** – runtime లో reflection చదువుతుంది. JSON, DB, validation libraries వీటిని వాడతాయి.

```
type User struct {
	ID    int    `json:"id"`
	Name  string `json:"name"`
	Email string `json:"email,omitempty"` // empty అయితే skip
	pass  string `json:"-"`               // ఎప్పటికీ serialize కాదు
}
// encoding/json ఈ tags చదివి JSON keys నిర్ణయిస్తుంది
```

Anonymous structs

Type పేరు లేకుండా one-off structs – tests, config, JSON parsing లో ఉపయోగం.

```
point := struct {
	X, Y int
}{X: 1, Y: 2}

// table-driven tests లో common
tests := []struct {
	name string
	in   int
	want int
}{
	{"double", 2, 4},
```

```
        {"zero", 0, 0},
}
```

Comparison

Struct fields **అన్నీ comparable** అయితే struct `==` compare చేయవచ్చు (field-wise). Slice/map/func field ఉంటే comparable కాదు.

```
type Point struct{ X, Y int }
a := Point{1, 2}
b := Point{1, 2}
fmt.Println(a == b)  // true — field-wise
// struct with slice field → == compile error
```

empty struct{} – zero-byte

`struct{}` **0 bytes** occupy చేస్తుంది – memory లేకుండా "presence" signal. Set (map[T]struct{}), channels లో signaling కి idiomatic.

```
// Set implementation — value memory వృథా లేదు
set := map[string]struct{}{}
set["a"] = struct{}{}
_, exists := set["a"]

// Channel signaling — data లేదు, signal మాత్రమే
done := make(chan struct{})
close(done) // "పని అయిపోయింది" signal
```

Internals – memory layout, alignment, padding

Struct fields memory లో వరుసగా ఉంటాయి, కానీ CPU **alignment** అవసరాల వల్ల మధ్యలో **padding** bytes చేరవచ్చు. ప్రతి field దాని size కి align అవ్వాలి (int64 → 8-byte boundary). Poor field order = వృథా memory.

```
// చెడ్డ order — 24 bytes (padding వల్ల)
type Bad struct {
        a bool   // 1 byte + 7 padding
        b int64  // 8 bytes
        c bool   // 1 byte + 7 padding
}
// మంచి order — 16 bytes (పెద్దవి ముందు, చిన్నవి కలిపి)
type Good struct {
        b int64  // 8 bytes
        a bool   // 1 byte
        c bool   // 1 byte + 6 padding
}
// unsafe.Sizeof(Bad{}) = 24, unsafe.Sizeof(Good{}) = 16
```

రూల్: fields ని **size descending** (పెద్దవి ముందు) order చేస్తే padding తగ్గుతుంది. Millions of structs ఉంటే ఇది గణనీయమైన memory పొదుపు.

Real-life Scenario

> **Struct = ఒక ఉద్యోగి ID కార్డ్** – పేరు, ID, department అన్నీ ఒక చోట, ఒక entity గా. Value copy = ఆ కార్డ్ యొక్క photocopy.
>
> **Padding = సూట్‌కేస్ లో వస్తువులు అమర్చడం.** పెద్ద వస్తువులు ముందు పెట్టి, చిన్నవి ఖాళీల్లో సర్దితే స్థలం ఆదా. ఇష్టం వచ్చినట్లు పెడితే మధ్యలో ఖాళీ (padding) వృథా – అదే struct field ordering.
>
> **empty struct{} = "హాజరు" register లో టిక్ మార్క్.** ఏ information లేదు, కేవలం "ఉన్నాడు" అని. 0 bytes.

Gotchas (సాధారణ తప్పులు)

- **Positional init (`User{1, "x"}`) fragile** – field order మారితే silently break. **Named init వాడు.**
- **Value copy surprise:** struct ని function కి pass చేస్తే copy; మార్పులు కావాలంటే `*T`.
- **Slice/map field ఉంటే struct copy shallow** – inner slice అదే array share.
- **Poor field ordering = memory waste** (alignment padding).
- **Struct with slice/map field is not comparable** – `==` compile error.

Key Points

- Struct = named fields; Go's building block (classes లేవు).
- **Named init** idiomatic; positional fragile.
- **Embedding** = composition; fields/methods promoted.
- **Tags** = reflection metadata (JSON/DB).
- `struct{}` = 0 bytes – sets, signaling channels.
- Comparable if all fields comparable.
- **Field order matters** – size-descending తో padding తగ్గించు.

Interview దృష్టి

Q: Go లో inheritance ఎలా? Classes లేకుండా? A: Inheritance లేదు; **embedding** (composition) వాడతారు. Struct లో మరో struct ని anonymous field గా embed చేస్తే దాని fields/methods promoted అవుతాయి. "Composition over inheritance."

Q: empty struct{} ఎందుకు? A: 0 bytes occupy చేస్తుంది. Set (`map[T]struct{}`) లో value memory వృథా చేయకుండా, లేదా channels లో pure signaling (`chan struct{}`) కి. Data కాదు, presence/signal మాత్రమే కావాలప్పుడు.

Q: Struct memory layout optimize ఎలా? A: Fields ని size-descending order చేయాలి (int64/pointers ముందు, bool/byte చివర) – alignment padding తగ్గి struct size తగ్గుతుంది. `unsafe.Sizeof` తో verify. Hot paths / millions of instances కి ముఖ్యం.

---

12. Pointers

వివరణ

Pointer = ఒక value యొక్క memory address ని పట్టుకునే variable. Go లో pointers ఉన్నాయి కానీ **pointer arithmetic లేదు** (C లా `p++` చేయలేం – safety). రెండు operators:

- **`&x`** – x యొక్క address తీసుకో ("address of").
- **`*p`** – p point చేసే value ("dereference").

```
x := 10
p := &x            // p = *int (x యొక్క address)
fmt.Println(*p)    // 10 (dereference)
*p = 20            // p ద్వారా x ని మార్చడం
fmt.Println(x)     // 20

var q *int         // nil pointer (zero value)
// fmt.Println(*q) // ❌ panic: nil pointer dereference
```

new() – allocate & get pointer

`new(T)` ఒక T ని zero-value తో allocate చేసి **దాని pointer** return చేస్తుంది.

```
p := new(int)      // *int, *p == 0
*p = 42

u := new(User)     // *User, అన్ని fields zero
u.Name = "Ravi"    // (*u).Name కి syntactic sugar — Go auto-derefs
```

`new(T)` vs `&T{}`: structs కి `&User{Name: "x"}` ఎక్కువ idiomatic (fields set చేయవచ్చు). `new` ఎక్కువగా basic types కి.

Pointer vs value – function semantics

Value pass = copy (మార్పులు caller కి కనిపించవు). Pointer pass = అదే memory (మార్పులు కనిపిస్తాయి).

```
func incVal(n int)   { n++ }        // copy — no effect
func incPtr(n *int)  { *n++ }       // original మారుతుంది

x := 5
incVal(x); fmt.Println(x) // 5
incPtr(&x); fmt.Println(x) // 6

// పెద్ద struct copy తప్పించడానికి కూడా pointer
func process(u *User) { u.Age++ }   // copy లేదు, mutation కనిపిస్తుంది
```

ఎప్పుడు pointer వాడాలి

వాడు pointer అయితే...	**వాడు value అయితే...**
Caller లో mutation కనిపించాలి	Immutable / read-only data
పెద్ద struct (copy ఖరీదు)	చిన్న struct (int, small structs)
"not set" ని nil తో సూచించాలి	Zero value సరిపోతుంది
Method receiver mutate చేస్తే	Concurrency: value copy safer (no sharing)

Note: slices, maps, channels అంతర్గతంగా pointer-like – వాటిని ఎక్కువసార్లు value గా pass చేసినా underlying data share అవుతుంది.

Go లో pointer arithmetic లేదు (& no dangling)

C లో `p + 1` చేయవచ్చు (danger). Go arithmetic ని నిషేధిస్తుంది – memory safety. అవసరమైతే `unsafe` package (Topic 38), కానీ చాలా అరుదు. Go GC వల్ల **dangling pointers లేవు** – pointer ఉన్నంత వరకు object GC అవ్వదు (escape analysis heap కి move చేస్తుంది, Topic 29).

```
func newUser() *User {
	u := User{Name: "local"} // local variable
	return &u                // C లో danger! Go లో safe –
}                           // escape analysis u ని heap కి move చేస్తుంది
```

Real-life Scenario

> **Pointer = ఒక ఇంటి చిరునామా (address), value = ఇల్లు యొక్క photocopy.**
>
> నువ్వు స్నేహితుడికి ఇంటి **photocopy** (value) ఇస్తే, అతను దానిపై గోడ రంగు మార్చినా నీ అసలు ఇల్లు మారదు. కానీ ఇంటి **చిరునామా** (pointer) ఇస్తే, అతను వెళ్ళి నిజంగా గోడ రంగు మారుస్తాడు – నీ ఇంటిపై కనిపిస్తుంది.
>
> **nil pointer = ఖాళీ చిరునామా చీటీ.** ఆ చిరునామాకి వెళ్తే ఇల్లే లేదు – crash (nil dereference panic). **Pointer arithmetic లేకపోవడం = "చిరునామా + 5 ఇళ్ళు" అని ఊహించి వెళ్ళడాన్ని** Go నిషేధిస్తుంది – పక్కింటి వాళ్ళ property లోకి తప్పుగా చొరబడకుండా.

Gotchas (సాధారణ తప్పులు)

- **nil pointer dereference = panic.** `var p *T; *p` crash. వాడేముందు nil check.
- **Loop లో local variable యొక్క address:** పాత Go (1.21-) లో `for _, v := range xs { ptrs = append(ptrs, &v) }` అన్నీ ఒకే address! (1.22+ fixed). ఖచ్చితత్వం కోసం `v := v`.
- **అనవసర pointers = GC pressure + heap allocations.** చిన్న values కి value semantics ఎక్కువ efficient.
- `*p` **ముందు nil interface / nil map field** – nested nil dereference.

Key Points

- `&x` = address, `*p` = dereference. **No pointer arithmetic** (safety).
- `new(T)` = zero-value T యొక్క pointer; structs కి `&T{}` idiomatic.
- Pointer = mutation కనిపిస్తుంది + పెద్ద struct copy తప్పిస్తుంది.
- **No dangling pointers** – GC + escape analysis (local address return safe).
- nil pointer dereference = **panic**; వాడేముందు check.
- slices/maps/channels already pointer-like internally.

Interview దృష్టి

Q: Local variable యొక్క address return చేయడం Go లో safe నా? A: అవును. C లో అది dangling pointer (stack unwound), కానీ Go **escape analysis** ఆ variable heap కి "escapes" అని గుర్తించి heap లో allocate చేస్తుంది. GC pointer ఉన్నంతవరకు దాన్ని ఉంచుతుంది. అందుకే `return &localVar` idiomatic మరియు safe.

Q: Go లో pointer arithmetic ఎందుకు లేదు? A: Memory safety – buffer overruns, dangling pointers, arbitrary memory access తప్పించడానికి. అవసరమైన rare cases కి `unsafe.Pointer` ఉంది కానీ GC guarantees కోల్పోతాం.

Q: ఎప్పుడు pointer receiver/param, ఎప్పుడు value? A: Mutation కావాలంటే, లేదా struct పెద్దదైతే (copy ఖరీదు) pointer. Immutable/చిన్న values కి, లేదా concurrency లో sharing తప్పించాలంటే value. ఒక type కి consistency కోసం అన్ని methods ఒకే receiver kind వాడటం మంచిది.

---

13. Functions – Multiple Returns, Variadic, Closures, defer, panic & recover

వివరణ

Go functions **first-class citizens** – variables కి assign, arguments గా pass, return చేయవచ్చు. కొన్ని ప్రత్యేకతలు: **multiple return values** (error handling కి పునాది), **named returns**, **variadic**, **closures**, మరియు అత్యంత idiomatic `defer`.

Multiple returns – Go's signature feature

```
func divide(a, b int) (int, error) {
	if b == 0 {
		return 0, errors.New("division by zero")
	}
	return a / b, nil
}

q, err := divide(10, 2)
if err != nil {
	// handle
}
```

```
// blank identifier తో ignore
_, err = divide(10, 0)
```

ఇదే Go యొక్క **error handling foundation** – exceptions కి బదులు `(result, error)` return చేయడం.

Named returns

Return values కి పేర్లు ఇవ్వవచ్చు – pre-declared variables లా. `return` (naked) వాటిని return చేస్తుంది. `defer` వీటిని modify చేయగలదు (కీలకం!).

```
func split(sum int) (x, y int) {  // x, y already declared, zero
	x = sum * 4 / 9
	y = sum - x
	return              // naked return — x, y
}

// deferred function లో named return modify చేయడం (error wrapping లో common)
func doWork() (err error) {
	defer func() {
		if r := recover(); r != nil {
			err = fmt.Errorf("recovered: %v", r) // return value
మారుస్తుంది!
		}
	}()
	panic("boom")
}
```

Variadic functions

`...T` = ఎన్ని arguments అయినా. లోపల slice గా వస్తుంది.

```
func sum(nums ...int) int {
	total := 0
	for _, n := range nums {
		total += n
	}
	return total
}
sum(1, 2, 3)          // 6
sum()                 // 0
nums := []int{1, 2, 3}
sum(nums...)          // slice ని spread — నోట్ చేయు "..."
```

Closures – first-class functions

Function లోపల function; బయటి variables ని "capture" (close over) చేస్తుంది. State ని encapsulate చేయడానికి powerful.

```
func counter() func() int {
	count := 0
	return func() int {   // count ని capture (closure)
```

```
                count++
                return count
        }
}
c := counter()
fmt.Println(c(), c(), c()) // 1 2 3 — state persists

// Higher-order function
func apply(nums []int, f func(int) int) []int {
        out := make([]int, len(nums))
        for i, n := range nums {
                out[i] = f(n)
        }
        return out
}
doubled := apply([]int{1, 2, 3}, func(n int) int { return n * 2 })
```

defer — LIFO cleanup (idiomatic!)

`defer` ఒక function call ని **surrounding function return అయ్యేవరకు వాయిదా** వేస్తుంది. అనేక defers ఉంటే **LIFO** (చివరిది మొదట) order లో run అవుతాయి. Resource cleanup కి Go's answer (finally కి బదులు).

```
func readFile(path string) error {
        f, err := os.Open(path)
        if err != nil {
                return err
        }
        defer f.Close()   // function ఎలా return అయినా (error/panic/normal) close

        // ... f వాడు; return ఎక్కడైనా, f.Close() guaranteed
        return nil
}

// LIFO order
func main() {
        defer fmt.Println("1")
        defer fmt.Println("2")
        defer fmt.Println("3")
        // output: 3 2 1
}
```

KEY: defer arguments defer అయిన క్షణంలోనే evaluate అవుతాయి (call మాత్రం వాయిదా):

```
func f() {
        i := 0
        defer fmt.Println(i)  // i=0 ఇప్పుడే capture! output: 0
        i = 10
        // defer run అయ్యేటప్పుడు కూడా 0 print చేస్తుంది
}
```

defer in loop — gotcha

Loop లో defer అన్నీ function చివర్లో run — resources పేరుకుపోతాయి.

```go
// ❌ BUG: files అన్నీ function చివరిదాకా open ఉంటాయి
func processAll(paths []string) {
	for _, p := range paths {
		f, _ := os.Open(p)
		defer f.Close()   // function చివర్లోనే! వేలాది files open
	}
}
// ✅ FIX: loop body ని function లో wrap
func processAll2(paths []string) {
	for _, p := range paths {
		func() {
			f, _ := os.Open(p)
			defer f.Close() // ఈ inner func చివర్లో — వెంటనే
			// process f
		}()
	}
}
```

panic & recover — Go యొక్క "exceptions" (అరుదుగా వాడాలి)

- `panic` – program ని unwind చేయడం మొదలుపెడుతుంది; deferred functions run అవుతాయి; recover లేకపోతే crash.
- `recover` – deferred function లో మాత్రమే పని చేస్తుంది; panic ని ఆపి, program ని కొనసాగిస్తుంది.

```go
func safeDivide(a, b int) (result int, err error) {
	defer func() {
		if r := recover(); r != nil {
			err = fmt.Errorf("recovered: %v", r)
		}
	}()
	result = a / b  // b==0 → runtime panic
	return
}
r, err := safeDivide(10, 0)
fmt.Println(r, err) // 0 recovered: runtime error: integer divide by zero
```

రూల్: panic/recover ని **normal error handling కి వాడవద్దు** – errors ని return చేయి. panic ని truly exceptional cases (programmer bug, unrecoverable state) కి, లేదా goroutine/server ని crash నుండి కాపాడటానికి (top-level recover in HTTP handler) మాత్రమే.

Internals – defer/panic/recover ఎలా

Prior Go versions లో `defer` heap-allocated linked list – నెమ్మది. Go 1.13/1.14 లో **open-coded defers** – compiler direct గా function epilogue లో inline చేస్తుంది (దాదాపు zero cost) simple cases లో. `panic` runtime లో deferred call chain ని unwind చేస్తూ walk చేస్తుంది; `recover` current goroutine యొక్క panic ని `_panic` struct నుండి తీసి stop చేస్తుంది. (Topic 32 లో deeper.)

Real-life Scenario

> **defer = హోటల్ గదిలో "checkout అయ్యేటప్పుడు లైట్లు ఆర్పు" reminder.** గదిలో ఏం చేసినా (త్వరగా వెళ్ళినా, argument అయినా = panic), వెళ్ళేటప్పుడు reminder execute అవుతుంది. LIFO = చివర తెరిచిన తలుపు ముందు మూసేయడం.
>
> **defer args early eval = గదిలో అడుగుపెట్టిన క్షణంలో "22° AC పెట్టు" అని write చేయడం.** తర్వాత గదిలో వేడి పెరిగినా (i=10), reminder మీద original 22° (i=0) నే ఉంటుంది.
>
> **panic/recover = fire alarm.** Building అంతా evacuate (unwind) అవుతున్నప్పుడు, ఒక floor manager (deferred recover) "ఆగండి, ఇది false alarm, నేను handle చేస్తా" అని ఆపగలడు. కానీ దీన్ని రోజూ door lock కి వాడకూడదు – errors కి normal return వాడు.

Gotchas (సాధారణ తప్పులు)

- **defer args early evaluation** – `defer f(i)` current i ని capture; closure `defer func(){f(i)}()` latest i.
- **Loop లో defer** = resources పేరుకుపోతాయి → inner function లో wrap.
- **recover deferred function లో మాత్రమే** పని చేస్తుంది – నేరుగా main flow లో `recover()` nil.
- **panic ని error handling కి వాడటం** = non-idiomatic. Errors return చేయి.
- **Naked returns పెద్ద functions లో** గందరగోళం – చిన్న functions కే.

Key Points

- **Multiple returns** → `(result, error)` = Go error handling foundation.
- **Named returns** – defer వాటిని modify చేయగలదు (error wrapping).
- **Variadic** `...T`; slice spread `slice...`.
- **Closures** capture outer variables – stateful functions.
- **defer** – LIFO cleanup; **args evaluate immediately**, call deferred.
- **defer in loop** = leak; wrap in function.
- **panic/recover** = last resort, not error handling; recover only in defer.

Interview దృష్టి

Q: defer arguments ఎప్పుడు evaluate అవుతాయి? A: `defer` statement execute అయిన క్షణంలోనే arguments evaluate అవుతాయి; function call మాత్రమే return వరకు వాయిదా. కాబట్టి `i:=0; defer fmt.Println(i); i=10` → 0 print చేస్తుంది. Latest value కావాలంటే closure వాడు.

Q: panic/recover vs error – ఎప్పుడు ఏది? A: 99% cases errors return చేయాలి (explicit, expected failures). panic/recover ని truly exceptional/unrecoverable situations కి, లేదా boundaries లో (goroutine/HTTP handler crash ని ఆపడానికి) మాత్రమే. recover deferred function లోనే పని చేస్తుంది.

Q: Multiple defers order? A: LIFO – చివరిగా defer చేసినది మొదట run. Nested resource cleanup (open A, open B → close B, close A) కి సరిగ్గా సరిపోతుంది.

---

14. Methods – Value vs Pointer Receivers, Method Sets

VALUE VS POINTER RECEIVER · METHOD SETS

VALUE vs POINTER RECEIVER

func (c Counter) Inc()	func (c *Counter) Inc()
Copy మీద పని చేస్తుంది అసలు struct మారదు చిన్న, immutable types కి	అసలు struct మీద పని మార్పులు నిలుస్తాయి పెద్ద structs కి (copy ఖరీదు)

Method set – ఇదే గందరగోళానికి మూలం

*T యొక్క method set lo **value + pointer** రెండు receivers ఉంటాయి.
T యొక్క method set lo **value receivers మాత్రమే**.
అందుకే: pointer receiver ఉన్న method interface ని satisfy చేయాలంటే – &x ఇవ్వాలి, x కాదు.

ఆచరణాత్మక నియమం

ఒక type కి ఏ ఒక్క method pointer receiver వాడితే – **అన్ని** pointer receiver వాడాలి.
కలగలిపితే – method set గందరగోళం, మరియు చదివేవారికి "ఇది mutate చేస్తుందా?" అని అస్పష్టత.

వివరణ

Method = ఒక type కి attach అయిన function. Go లో methods classes లో కాదు – receiver ద్వారా type కి bind అవుతాయి. Receiver = function పేరుకి ముందు `(r T)` లేదా `(r *T)`.

```
type Rectangle struct{ Width, Height float64 }

// Value receiver — copy మీద పని చేస్తుంది
func (r Rectangle) Area() float64 {
	return r.Width * r.Height
}

// Pointer receiver — original ని modify చేయగలదు
func (r *Rectangle) Scale(factor float64) {
	r.Width *= factor
	r.Height *= factor
}

rect := Rectangle{Width: 3, Height: 4}
fmt.Println(rect.Area())  // 12
rect.Scale(2)             // Go auto (&rect).Scale(2)
fmt.Println(rect.Area())  // 48
```

Methods ని struct లకే కాదు, **ఏ named type కి అయినా** (int-based, slice-based) define చేయవచ్చు:

```
type Celsius float64
func (c Celsius) ToFahrenheit() Celsius { return c*9/5 + 32 }

type IntList []int
func (l IntList) Sum() int { s := 0; for _, v := range l { s += v }; return s }
```

Value vs Pointer Receiver – ఎప్పుడు ఏది

Value receiver `(r T)`	**Pointer receiver** `(r *T)`
Copy మీద పని – original మారదు	Original ని mutate చేస్తుంది
చిన్న types, immutable ops	పెద్ద structs (copy తప్పించడం)
Read-only (getters)	State మార్చే methods (setters)
Concurrency-safe (no shared state)	Mutation కావాలప్పుడు

రూల్ 1: Method original ని modify చేయాలంటే **pointer receiver. రూల్ 2:** పెద్ద struct → pointer (copy ఖరీదు తప్పించడానికి). **రూల్ 3 (consistency):** ఒక type కి ఒక method pointer receiver వాడితే, **అన్ని methods pointer receiver** వాడు (mix చేయకు) – method set consistency కి.

Method Sets – అత్యంత కీలకం (interface satisfaction కి)

Method set = ఒక type value మీద ఏ methods call చేయవచ్చో నిర్ణయిస్తుంది. ఇది **interface satisfaction** కి direct impact:

- Type `T` యొక్క method set = **value receiver methods మాత్రమే.**
- Type `*T` యొక్క method set = **value + pointer receiver methods** (రెండూ).

అంటే: **pointer receiver method ఉన్న interface ని value satisfy చేయదు; pointer మాత్రమే చేస్తుంది.**

```
type Stringer interface{ String() string }

type Point struct{ X, Y int }
func (p *Point) String() string {  // POINTER receiver
        return fmt.Sprintf("(%d,%d)", p.X, p.Y)
}

var s Stringer
p := Point{1, 2}
// s = p    // ❌ compile error! Point (value) method set లో String లేదు
s = &p      // ✅ *Point method set లో ఉంది
fmt.Println(s.String()) // (1,2)
```

ఎందుకు? Value ని interface లో పెట్టినప్పుడు అది copy. ఆ copy addressable కాదు, కాబట్టి pointer receiver (mutating) method ని call చేయలేం – Go దీన్ని compile time లో నిషేధిస్తుంది.

> **గమనిక:** నేరుగా `p.String()` (addressable variable మీద) పని చేస్తుంది – Go auto `(&p).String()`. కానీ **interface లో పెట్టేటప్పుడు** ఈ auto-address-of జరగదు – method set rule strict.

Method Values & Method Expressions

Method value – receiver ని bind చేసిన function value (closure లా):

```
rect := Rectangle{3, 4}
areaFn := rect.Area   // method value — rect bound
fmt.Println(areaFn()) // 12 (rect అవసరం లేదు)
```

Method expression – receiver ని explicit argument గా తీసుకునే function:

```
areaExpr := Rectangle.Area   // func(Rectangle) float64
fmt.Println(areaExpr(rect))  // 12 — receiver ని pass చేయాలి

scaleExpr := (*Rectangle).Scale // func(*Rectangle, float64)
scaleExpr(&rect, 2)
```

Real-life Scenario

> **Value receiver = ఫారం యొక్క photocopy మీద సంతకం** – original ఫారం మారదు.
> **Pointer receiver = అసలు ఫారం మీద సంతకం** – శాశ్వతం.
>
> **Method set rule = "ఈ document మార్చే అధికారం ఎవరికి?"** photocopy (value) పట్టుకున్నవాడు చదవగలడు (value methods) కానీ మార్చలేడు. అసలు document ఉన్నవాడు (pointer) చదవగలడు + మార్చగలడు. అందుకే mutating interface కి pointer తప్పనిసరి – photocopy సరిపోదు.

Gotchas (సాధారణ తప్పులు)

- **Pointer receiver method ఉన్న interface ని value తో satisfy చేయలేం** – `&value` పంపాలి. (అత్యధిక interview + real bugs.)
- **Value receiver మీద mutation lost** – `func (r T) Set(x)` original ని మార్చదు (copy). Pointer వాడు.
- **Receiver types mix చేయడం** – కొన్ని value, కొన్ని pointer → method set గందరగోళం. Consistent గా ఉండు.
- **nil pointer receiver** – pointer receiver method ని nil pointer మీద call చేయవచ్చు (method లోపల nil handle చేస్తే); కానీ field access panic.
- **Value receiver + goroutine లో pass** – copy safe కానీ mutation share కాదు.

Key Points

- Method = receiver ద్వారా type కి bound function; ఏ named type కి అయినా.
- **Value receiver** = copy (read-only, small); **pointer receiver** = mutate/large.
- **Method set:** `T` → value methods; `*T` → value + pointer methods.
- **Pointer receiver method ఉన్న interface → value satisfy చేయదు** (కీలకం!).
- Consistency: ఒక type కి అన్ని methods ఒకే receiver kind.
- Method values (bound) vs method expressions (receiver as arg).

Interview దృష్టి

Q: Value vs pointer receiver – ఎలా ఎంచుకుంటావు? A: (1) Method state modify చేయాలంటే pointer. (2) Struct పెద్దదైతే pointer (copy తప్పించడం). (3) Consistency: ఒక method pointer అయితే అన్నీ pointer. చిన్న immutable types కి value OK. Default: doubt ఉంటే pointer.

Q: ఈ code ఎందుకు compile అవ్వదు?

```
type I interface{ M() }
type T struct{}
func (t *T) M() {}
var i I = T{}  // error
```

A: `M` pointer receiver. `T` (value) యొక్క method set లో `M` లేదు – `*T` లోనే ఉంది. కాబట్టి value `T{}` interface `I` ని satisfy చేయదు. `var i I = &T{}` పని చేస్తుంది.

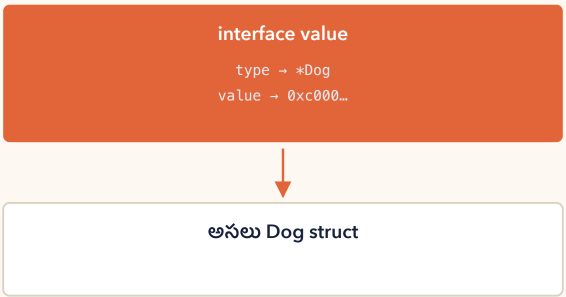
Q: Method value vs method expression? A: Method value = `x.M` – receiver `x` bound అయిన function. Method expression = `T.M` – receiver ని explicit మొదటి argument గా తీసుకునే function (`func(T, ...)`).

---

15. Interfaces – Implicit Satisfaction, Internals, typed-nil Trap

INTERFACE INTERNALS · TYPE + VALUE జత

INTERFACE లోపల – రెండు pointers



క్లాసిక్ nil interface ఉచ్చు

var p *Dog = nil; var a Animal = p; a == nil → **false**!
ఎందుకంటే interface lo **type** ఉంది (*Dog), value మాత్రమే nil.
అందుకే error return చేసేటప్పుడు – nil pointer ని error interface lo ఎప్పుడూ wrap చేయకూడదు.

నివారణ

Error return చేయాలంటే:

`return nil` అని స్పష్టంగా రాయడం –concrete nil pointer ని return చేయకపోవడం. ఇది Go lo అత్యంత ప్రసిద్ధమైన ఉచ్చు.

వివరణ

Interface = method signatures యొక్క సముదాయం (behavior contract). Go interfaces యొక్క superpower: **implicit satisfaction** – ఒక type interface ని satisfy చేయడానికి `implements` అని

declare చేయనవసరం లేదు. అవసరమైన methods ఉంటే చాలు, automatically satisfy అవుతుంది ("duck typing, but static").

```
type Writer interface {
        Write(p []byte) (n int, err error)
}

// File "implements" అని ఎక్కడా రాయలేదు — Write method ఉంది చాలు
type File struct{ name string }
func (f *File) Write(p []byte) (int, error) {
        fmt.Printf("writing %d bytes to %s\n", len(p), f.name)
        return len(p), nil
}

var w Writer = &File{name: "log.txt"}  // automatic satisfaction
w.Write([]byte("hello"))
```

ఎందుకు implicit? – decoupling

Java లో `class File implements Writer` – File, Writer కి coupled. Go లో interface ని **consumer** వైపు define చేయవచ్చు (producer కి తెలియకుండా). `io.Writer` ని Go team define చేసింది; నీ type దానికి తెలియకుండానే satisfy అవుతుంది. ఇది **"accept interfaces, return structs"** idiom కి పునాది.

Empty interface – `any`

`interface{}` (= `any`, Go 1.18+ alias) – **సున్నా methods → ప్రతి type satisfy చేస్తుంది.** "ఏదైనా value" store చేయగలదు (Java Object లా).

```
var x any        // any = interface{}
x = 42
x = "hello"
x = []int{1, 2}

func describe(i any) {
        fmt.Printf("%v (%T)\n", i, i)  // value మరియు type
}
```

`any` ని అతిగా వాడకూడదు – type safety కోల్పోతాం. Generics (Topic 17) చాలా cases లో మంచి ప్రత్యామ్నాయం.

Type Assertion – value ని బయటకు తీయడం

Interface లో ఉన్న concrete type ని extract చేయడం. **comma-ok form safe** (panic లేదు):

```
var i any = "hello"

s := i.(string)       // assertion – తప్పైతే panic
fmt.Println(s)        // hello

s, ok := i.(string)   // comma-ok – safe
if ok { fmt.Println(s) }
```

```
n, ok := i.(int)      // ok=false, n=0 (panic లేదు)
```

Type Switch

అనేక types కి branch చేయడం:

```
func process(i any) {
	switch v := i.(type) {
	case int:
		fmt.Println("int:", v*2)
	case string:
		fmt.Println("string:", len(v))
	case []int:
		fmt.Println("slice len:", len(v))
	case nil:
		fmt.Println("nil")
	default:
		fmt.Printf("unknown: %T\n", v)
	}
}
```

Internals – iface vs eface, itab

Interface value internally **రెండు words:**

- `eface` (empty interface, `any`): `{*_type, data}` – type descriptor + data pointer.
- `iface` (non-empty interface): `{*itab, data}` – **itab** (interface table) + data pointer.

`itab` = interface type + concrete type + **method pointers array** (concrete type యొక్క implementations). ఇది method dispatch ని enable చేస్తుంది. itab lazily compute అయి cached – మొదటిసారి కొంచెం cost, తర్వాత fast.

```
Interface value:
┌──────────────┬──────────────┐
│  *itab       │   data ptr   │
│ (type+method │ (concrete    │
│  table)      │  value)      │
└──────────────┴──────────────┘
```

`data` ఎప్పుడూ pointer-like – concrete value పెద్దదైతే heap కి escape అవుతుంది (interface లో పెట్టడం తరచుగా allocation). చిన్న values కూడా boxing.

⚠️ The typed-nil trap (అత్యంత famous Go bug)

Interface `nil` ఐతే type మరియు value రెండూ nil అయినప్పుడే. ఒక typed nil pointer ని interface లో పెడితే, interface **nil కాదు!** (type non-nil).

```
type MyError struct{}
func (e *MyError) Error() string { return "oops" }
```

```
func doWork() error {
        var e *MyError = nil    // typed nil pointer
        return e                // ⚠️ interface కి (*MyError, nil) — nil కాదు!
}

func main() {
        err := doWork()
        if err != nil {
                fmt.Println("error occurred!")  // ఇది print అవుతుంది! 😱
                // err != nil ఎందుకంటే type = *MyError (non-nil), value = nil మాత్రమే
        }
}
```

ఎందుకు? interface = `{type, value}`. ఇక్కడ `{*MyError, nil}` – type slot fill అయింది, కాబట్టి interface ≠ nil. Interface nil అవ్వాలంటే **రెండూ** nil అవ్వాలి.

Fix: typed nil pointer ని ఎప్పుడూ interface గా return చేయకు; explicit `return nil`:

```
func doWork() error {
        var e *MyError = nil
        if someCondition {
                e = &MyError{}
        }
        if e == nil {
                return nil   // ✅ explicit untyped nil
        }
        return e
}
```

"Accept interfaces, return structs"

Go idiom: functions **arguments** గా interfaces తీసుకోవాలి (flexible – ఏ implementation అయినా), కానీ **concrete structs return** చేయాలి (caller కి full type, methods అందుబాటులో).

```
// మంచిది: interface accept (flexible input)
func Save(w io.Writer, data []byte) error {
        _, err := w.Write(data)
        return err
}
// File, buffer, network — ఏదైనా io.Writer పంపవచ్చు
```

Real-life Scenario

> **Interface = ఉద్యోగ జాబితా (job description).** "Write() చేయగలవాడు కావాలి" అని మాత్రమే అడుగుతుంది – వ్యక్తి ఎవరైనా (File, Network, Buffer) పర్వాలేదు, ఆ పని చేస్తే చాలు. Java "నేను Writer" అని badge వేసుకోవాలి; Go "నువ్వు Write చేయగలవా? అయితే నువ్వే Writer" అంటుంది.
>
> **typed-nil trap = ఖాళీ కవరు, కానీ మీద address రాసి ఉంది.** నువ్వు "కవరు లోపల ఏమీ లేదా?" (value nil?) అని అడిగితే "అవును ఖాళీ." కానీ "కవరు ఏదీ లేదా?" (interface nil?) అంటే "లేదు, కవరు ఉంది (type ఉంది), లోపల మాత్రం ఖాళీ." ఈ తేడాయే bug – postman ఖాళీ కవరుని కూడా "మెయిల్ వచ్చింది" అని లెక్కిస్తాడు.

Gotchas (సాధారణ తప్పులు)

- **typed-nil trap** – typed nil pointer ని error/interface గా return చేయడం. **Fix:** explicit `return nil`.
- **Pointer receiver + value = no satisfaction** (Topic 14) – `&value` పంపు.
- `any` **overuse** – type safety loss; generics/concrete types ప్రాధాన్యం.
- **Type assertion without comma-ok** → wrong type అయితే panic. Safe form వాడు.
- **Interface holding large value** = heap allocation (boxing) – hot paths లో గమనించు.
- **nil interface method call** → panic (`{nil, nil}` మీద method).

Key Points

- Interface = behavior contract; **implicit satisfaction** (no `implements`).
- **Consumer** interface ని define చేయవచ్చు → decoupling.
- `any` / `interface{}` = 0 methods = everything.
- Type assertion (comma-ok safe), type switch.
- Internals: `iface {*itab, data}` , `eface {*_type, data}` ; itab = method table.
- **typed-nil trap:** interface nil ఐతేనే type & value రెండూ nil.
- "Accept interfaces, return structs"; keep interfaces small.

Interview దృష్టి

Q: Go interfaces Java కంటే ఎలా వేరు? A: Implicit satisfaction – type "implements" అని declare చేయదు, అవసరమైన methods ఉంటే చాలు (structural typing). దీనివల్ల interface ని consumer వైపు define చేయవచ్చు, producer తెలియకుండా. Decoupling చాలా బలం.

Q: typed-nil trap వివరించు. A: Interface = `{type, value}` pair. Typed nil pointer (`var p *T = nil`) ని interface గా assign చేస్తే, interface = `{*T, nil}` – type slot non-nil, కాబట్టి `interface != nil` . Interface nil అవ్వాలంటే type & value రెండూ nil. Fix: explicit `return nil` వాడు, typed nil pointer ని కాదు.

Q: Interface internally ఎలా store అవుతుంది? A: రెండు words – `iface` కి `{*itab, data ptr}` , `eface` (any) కి `{*_type, data ptr}` . itab లో interface type, concrete type, method pointers

array ఉంటాయి (dynamic dispatch). itab cached. Value boxing ద్వారా data pointer.

---

16. Embedding & Composition

వివరణ

Go లో inheritance లేదు. బదులుగా **embedding** – ఒక struct లో మరో struct/interface ని **anonymous field** గా చేర్చడం. దాని fields/methods **promoted** అవుతాయి – outer type మీద నేరుగా access. ఇది "composition over inheritance" ని native గా చేస్తుంది.

Struct Embedding

```
type Animal struct {
        Name string
}
func (a Animal) Eat() { fmt.Println(a.Name, "తింటోంది") }

type Dog struct {
        Animal  // embedded (anonymous) — Name, Eat() promoted
        Breed string
}

d := Dog{Animal: Animal{Name: "Tommy"}, Breed: "Labrador"}
d.Eat()           // promoted method — d.Animal.Eat() కి sugar
fmt.Println(d.Name) // promoted field
fmt.Println(d.Breed)
```

Embedded field పేరు = type పేరు (`d.Animal`). కానీ promotion వల్ల నేరుగా `d.Name` , `d.Eat()` వాడవచ్చు.

Overriding (method shadowing)

Outer type అదే పేరుతో method define చేస్తే, అది embedded method ని **shadow** చేస్తుంది (override లా). Embedded ని explicitly కూడా call చేయవచ్చు.

```
func (d Dog) Eat() {                // Animal.Eat ని shadow
        fmt.Println(d.Name, "ఎముక కొరుకుతోంది")
        d.Animal.Eat()                 // parent method explicit call (super లా)
}
d.Eat()
// Tommy ఎముక కొరుకుతోంది
// Tommy తింటోంది
```

కీలక తేడా (inheritance కాదు): Embedded `Animal` యొక్క method లోపల `a.Name` వాడితే, అది **Animal యొక్క view** – polymorphic dispatch కి Dog కి తిరిగి రాదు (Go లో virtual methods లేవు; embedding ≠ subtyping).

Interface Embedding

Interfaces కూడా embed అవుతాయి – చిన్న interfaces కలిపి పెద్దది. stdlib లో common:

```
type Reader interface{ Read(p []byte) (int, error) }
type Writer interface{ Write(p []byte) (int, error) }

// io.ReadWriter = రెండింటి కలయిక
type ReadWriter interface {
        Reader
        Writer
}
// ReadWriter satisfy చేయాలంటే Read + Write రెండూ కావాలి
```

`io.ReadWriteCloser`, `io.ReadWriter` వంటివి ఇలాగే compose చేయబడ్డాయి.

Embedding pointers & interfaces in structs

```
// Interface ని struct లో embed — decorator/middleware pattern
type LoggingReader struct {
        io.Reader   // ఏ Reader అయినా wrap చేయవచ్చు
}
func (lr LoggingReader) Read(p []byte) (int, error) {
        n, err := lr.Reader.Read(p)  // delegate
        fmt.Printf("read %d bytes\n", n)
        return n, err
}
```

Diamond-free – Go's approach

C++ diamond problem (multiple inheritance ambiguity) Go లో లేదు, ఎందుకంటే embedding = subtyping కాదు. కానీ **ambiguous promotion** సాధ్యం – రెండు embedded types లో అదే method name ఉంటే, promotion జరగదు; explicitly qualify చేయాలి.

```
type A struct{}
func (A) Hello() { fmt.Println("A") }
type B struct{}
func (B) Hello() { fmt.Println("B") }

type C struct {
        A
        B
}
c := C{}
// c.Hello()  // ❌ ambiguous selector — compile error
c.A.Hello()   // ✅ explicit
c.B.Hello()   // ✅ explicit
```

ఇది diamond ని compile error గా చేస్తుంది – silent ambiguity లేదు.

Composition table

అంశం	Inheritance (Java)	Embedding (Go)
Relationship	"is-a" (subtype)	"has-a" (composition)
Polymorphism	Virtual dispatch (base→derived)	లేదు (embedded method Go's view)
Multiple	Single (Java classes)	Multiple embeds OK
Diamond	Ambiguity problem	Compile error (explicit qualify)
Override	Virtual override	Method shadowing
Coupling	Tight	Loose (delegation)

Real-life Scenario

> **Embedding = ఒక కంపెనీ లో "అకౌంటెంట్‌ని hire చేయడం" (has-a), "నేనే అకౌంటెంట్ అవ్వడం" (is-a) కాదు.** Manager కి accountant embed చేస్తే, "manager.FileTaxes()" అనగానే accountant పని చేస్తాడు (promotion/delegation). Manager కావాలంటే తనే override చేయవచ్చు, లేదా "accountant, నువ్వు చెయ్యి" (`m.Accountant.FileTaxes()`) అనవచ్చు. ఇద్దరు accountants (A, B) ఉంటే "FileTaxes చెయ్యి" అంటే ఎవరు? – Go "ఎవరో స్పష్టంగా చెప్పు" అంటుంది (ambiguity compile error).

Gotchas (సాధారణ తప్పులు)

- **Embedding ≠ inheritance** – embedded method Go's own view; outer override virtual dispatch ఇవ్వదు. Polymorphism కి **interfaces** వాడు.
- **Ambiguous promotion** – రెండు embeds లో అదే method → explicit qualify.
- **Embedded interface nil** – struct లో interface embed చేసి nil ఉంచి method call → nil panic.
- **JSON marshaling** embedded fields ని flatten చేస్తుంది (కొన్నిసార్లు surprise).

Key Points

- **Embedding** = anonymous field → fields/methods **promoted**.
- Composition ("has-a"), inheritance (subtype) కాదు – no virtual dispatch.
- **Method shadowing** = outer method wins; `outer.Embedded.M()` = super call.
- **Interface embedding** composes small interfaces (io.ReadWriter).
- **Diamond** → **compile error** (explicit qualify), silent ambiguity లేదు.
- Polymorphism కి interfaces; code reuse కి embedding.

Interview దృష్టి

Q: Go embedding inheritance కాదని ఎందుకు అంటారు? A: Embedding fields/methods ని promote చేస్తుంది (delegation), కానీ subtype relationship సృష్టించదు. Embedded type యొక్క

method లోపల outer type యొక్క overridden method కి polymorphic dispatch జరగదు (no virtual methods). అది "has-a" composition; Java "is-a" inheritance కాదు.

Q: Go లో diamond problem ఎలా handle అవుతుంది? A: Embedding subtyping కాదు కాబట్టి classic diamond లేదు. కానీ రెండు embedded types లో అదే method/field name ఉంటే promotion ambiguous → **compile error**; explicitly `x.A.M()` గా qualify చేయాలి. Silent ambiguity ఉండదు.

---

17. Generics (Go 1.18+)

వివరణ

Go 1.18 లో **generics** (type parameters) వచ్చాయి – 10+ years డిబేట్ తర్వాత. దీనికి ముందు type-agnostic code కి `interface{}` + type assertions (unsafe, slow) లేదా code duplication తప్పనిసరి. Generics **type-safe, reusable** code ని ఇస్తాయి.

```
// Type parameter T — comparable constraint (== support చేసేవి)
func Contains[T comparable](slice []T, target T) bool {
	for _, v := range slice {
		if v == target {
			return true
		}
	}
	return false
}

Contains([]int{1, 2, 3}, 2)          // true — T=int inferred
Contains([]string{"a", "b"}, "c")    // false — T=string
```

`[T comparable]` = **type parameter** T, constraint `comparable`. Call లో Go type ని **infer** చేస్తుంది (`Contains[int](...)` explicit కూడా వ్రాయవచ్చు).

Constraints – type sets

Constraint = ఒక type parameter ఏ types అంగీకరించగలదో నిర్వచించే interface. Generics కి interfaces ని **type sets** గా విస్తరించారు.

```
// built-in constraints (constraints package లో + builtin)
// any        — ఏ type అయినా
// comparable — == != support (map keys, equality)

// custom constraint — union of types
type Number interface {
	~int | ~int64 | ~float64   // ~ = "underlying type" ఈ types అయిన అన్నీ
}

func Sum[T Number](nums []T) T {
	var total T
	for _, n := range nums {
		total += n
```

```
        }
        return total
}
Sum([]int{1, 2, 3})       // 6
Sum([]float64{1.5, 2.5})  // 4.0
```

`~int` (tilde) = "underlying type int ఉన్న అన్ని types" – ఉదా. `type Celsius float64` కూడా `~float64` constraint ని satisfy చేస్తుంది. `~` లేకుండా `int` అయితే exact `int` మాత్రమే.

Generic types (structs)

```
type Stack[T any] struct {
        items []T
}
func (s *Stack[T]) Push(item T) { s.items = append(s.items, item) }
func (s *Stack[T]) Pop() (T, bool) {
        var zero T
        if len(s.items) == 0 {
                return zero, false
        }
        item := s.items[len(s.items)-1]
        s.items = s.items[:len(s.items)-1]
        return item, true
}

s := &Stack[int]{}
s.Push(1); s.Push(2)
val, ok := s.Pop()  // 2, true
```

Type inference

Go చాలా cases లో type args ని infer చేస్తుంది – explicit `[int]` అవసరం లేదు:

```
func Map[T, U any](s []T, f func(T) U) []U {
        out := make([]U, len(s))
        for i, v := range s {
                out[i] = f(v)
        }
        return out
}
// T=int inferred; U=string function return నుండి inferred
strs := Map([]int{1, 2, 3}, func(n int) string { return strconv.Itoa(n) })
```

ఎప్పుడు వాడాలి / ఎప్పుడు వాడకూడదు

వాడు:

- General-purpose data structures (Stack, Tree, LinkedList, Set).
- Container algorithms (Map, Filter, Reduce, Min, Max, Sort).
- ఒకే logic multiple types కి (type-safe గా).

వాడకు:

- కేవలం ఒక type కి పని చేస్తే – over-engineering.
- Interface (behavior) సరిపోయే చోట – generics data కి, interfaces behavior కి.
- Methods కి type parameters లేవు (only functions/types) – దాని బదులు interface.

> **Rule of thumb:** "ఇది multiple types కి ఒకే **implementation** కావాలా?" → generics. "Multiple types కి వేర్వేరు **behavior** కావాలా?" → interface.

Internals – GCShape stenciling + dictionaries

C++ templates ప్రతి type కి పూర్తి copy generate చేస్తాయి (code bloat). Go **hybrid approach: "GCShape stenciling with dictionaries."**

- **GCShape** = memory layout దృష్ట్యా ఒకేలా ఉండే types ఒక group. అన్ని pointer types ఒకే GCShape (అన్నీ word-sized pointers). `int`, `float64` వేర్వేరు shapes.
- ఒక GCShape కి **ఒక్క compiled version (stencil)** generate అవుతుంది – అన్ని pointer-based instantiations ఆ ఒక్క copy share చేస్తాయి (code size తగ్గుతుంది).
- **Dictionary** = runtime లో pass అయ్యే hidden argument – నిర్దిష్ట type యొక్క metadata (type descriptors, method tables, size). Stencil ఈ dictionary నుండి type-specific info తీసుకుంటుంది.

Trade-off: C++ కంటే code bloat తక్కువ, కానీ dictionary lookups వల్ల monomorphized C++ కంటే కొంచెం slower కావచ్చు (pointer-shape generics లో indirect). Value types (int) కి better specialization.

Real-life Scenario

> **Generics = ఒక universal remote control template.** TV, AC, ఫ్యాన్ – అన్నిటికీ "power on/off" button అవసరం. ప్రతి దానికి వేరే remote (code duplication) చేయకుండా, ఒక template remote – పరికరం type ని "slot" లో పెడితే (type parameter) సరిపోతుంది.
>
> **GCShape + dictionary = ఒకే remote circuit board, కానీ పరికర-నిర్దిష్ట codes ఒక చిన్న chip (dictionary) లో.** అన్ని "pointer-లాంటి" పరికరాలు ఒకే board share చేస్తాయి, chip మాత్రం device-specific.

Gotchas (సాధారణ తప్పులు)

- **Methods కి type parameters లేవు** – `func (s S) M[T any]()` invalid. Type-level parameters మాత్రమే.
- `~` **మర్చిపోవడం** – `int | float64` custom types (type Celsius float64) ని reject చేస్తుంది; `~int | ~float64` accept చేస్తుంది.
- **Over-generalization** – ఒక type కి generics = అనవసర complexity.
- **comparable ≠ ordered** – `<`, `>` కి `comparable` సరిపోదు; `constraints.Ordered` (`golang.org/x/exp/constraints`) కావాలి.
- **Zero value** – `var zero T` generic zero కి; T-specific zero తెలియదు.

Key Points

- Generics (1.18+) = type-safe reuse; `[T constraint]`.
- **Constraints** = interfaces as type sets; `any`, `comparable`, custom unions.
- `~T` = underlying type match (named types కూడా).
- Type **inference** చాలా cases లో explicit args తీసేస్తుంది.
- Data structures/algorithms కి generics; behavior కి interfaces.
- Internals: **GCShape stenciling + dictionaries** (code bloat తక్కువ, slight indirection).
- **Methods కి type params లేవు.**

Interview దృష్టి

Q: Generics vs interface{} – ఎప్పుడు generics? A: `interface{}` / `any` type safety కోల్పోతుంది (runtime assertions, boxing, slow). Generics compile-time type safety + performance ఇస్తాయి. Multiple types కి ఒకే implementation (containers, algorithms) కావాలంటే generics; runtime లో vary అయ్యే behavior కి interfaces.

Q: Go generics internally ఎలా? C++ templates తో పోలిక? A: Go "GCShape stenciling with dictionaries" వాడుతుంది – memory-layout దృష్ట్యా ఒకేలా ఉండే types (అన్ని pointers ఒక shape) ఒక్క compiled stencil share చేస్తాయి, type-specific info runtime dictionary నుండి. C++ ప్రతి type కి పూర్తి copy (monomorphization) – వేగం ఎక్కువ కానీ code bloat ఎక్కువ. Go trade-off: తక్కువ code size, slight runtime indirection.

Q: `~int` లో `~` అర్థం? A: "Underlying type int ఉన్న అన్ని types" – `type MyInt int` కూడా satisfy చేస్తుంది. `~` లేకపోతే exact `int` type మాత్రమే.

---

18. Error Handling

ERROR HANDLING · VALUES, WRAPPING, %W

ERROR HANDLING – exceptions లేవు, values ఉన్నాయి

ఇతర భాషలు	Go
try { … } catch (e) { … } Error "పైకి ఎగురుతుంది" ఎక్కడ పడుతుందో కనిపించదు	if err != nil { return err } Error ఒక సాధారణ value ప్రతి చోటా స్పష్టంగా కనిపిస్తుంది

WRAPPING – context కలుపుతూ

```
fmt.Errorf("saving user %d: %w", id, err)
```

%w తో	%v తో
errors.Is(err, ErrNotFound) పని చేస్తుంది errors.As(err, &myErr) పని చేస్తుంది మూలం దాక chain ఉంటుంది	కేవలం string – chain తెగిపోతుంది errors.Is ఎప్పటికీ match కాదు Debug చేయడం కష్టం

వివరణ

Go లో **exceptions లేవు.** బదులుగా errors **ordinary values** – functions `error` ని చివరి return value గా ఇస్తాయి, caller దాన్ని explicitly check చేస్తాడు. ఇది Go యొక్క అత్యంత విభిన్న (కొంతమందికి చిరాకు) design decision. తత్వం: **"errors are values"** – errors ని hide చేయకుండా, first-class గా handle చేయడం.

```
// error అనేది ఒక built-in interface – ఒక్క method
type error interface {
        Error() string
}

func readConfig(path string) (Config, error) {
        data, err := os.ReadFile(path)
        if err != nil {
                return Config{}, err   // error ని పైకి propagate
        }
        // ...
        return cfg, nil             // success → nil error
}

// idiomatic caller pattern
cfg, err := readConfig("app.yaml")
if err != nil {
        log.Fatal(err)
}
```

Errors ని create చేయడం

```
err1 := errors.New("something failed")            // simple
err2 := fmt.Errorf("failed to read %s: %w", path, err) // formatted + wrap
```

Sentinel errors – predefined error values

Known error conditions ని package-level variables గా. Callers `errors.Is` తో compare చేస్తారు.

```
var ErrNotFound = errors.New("not found")
var ErrPermission = errors.New("permission denied")

func lookup(id int) (User, error) {
        if id < 0 {
                return User{}, ErrNotFound
        }
        // ...
}

// caller
u, err := lookup(-1)
if errors.Is(err, ErrNotFound) {   // == కాదు, errors.Is (wrapping-aware)
        // handle 404
}
```

stdlib examples: `io.EOF`, `sql.ErrNoRows`, `os.ErrNotExist`.

Error wrapping – `%w` (Go 1.13+)

`fmt.Errorf` తో `%w` verb = error ని **wrap** చేస్తుంది – context జోడిస్తూ original ని preserve చేస్తుంది. దీన్ని `errors.Unwrap`, `errors.Is`, `errors.As` తో పొరలుగా చూడవచ్చు.

```
func processOrder(id int) error {
	if err := chargeCard(id); err != nil {
		return fmt.Errorf("processOrder %d: %w", id, err)  // wrap
	}
	return nil
}
// error chain: "processOrder 42: charge failed: insufficient funds"
```

errors.Is / errors.As / errors.Unwrap

Function	పని
`errors.Is(err, target)`	Chain లో ఎక్కడైనా `target` sentinel ఉందా? (`==` కాదు, unwraps)
`errors.As(err, &target)`	Chain లో నిర్దిష్ట **type** error ఉందా? దాన్ని extract చేస్తుంది
`errors.Unwrap(err)`	ఒక పొర తీసేస్తుంది (wrapped error)

```
// errors.Is — sentinel comparison (wrap-aware)
if errors.Is(err, sql.ErrNoRows) {
	// row లేదు
}

// errors.As — custom type extraction
type ValidationError struct{ Field string }
func (e *ValidationError) Error() string { return "invalid " + e.Field }

var ve *ValidationError
if errors.As(err, &ve) {
	fmt.Println("bad field:", ve.Field)  // wrapped అయినా extract
}
```

Custom error types – extra context

```
type HTTPError struct {
	Code    int
	Message string
	Err     error   // wrapped underlying error
}
func (e *HTTPError) Error() string {
	return fmt.Sprintf("HTTP %d: %s", e.Code, e.Message)
}
func (e *HTTPError) Unwrap() error { return e.Err }  // errors.Is/As కి enable

// వాడకం
return &HTTPError{Code: 404, Message: "user not found", Err: ErrNotFound}
```

`Unwrap()` method define చేస్తే నీ custom error `errors.Is/As` chain లో పాల్గొంటుంది.

errors.Join – బహుళ errors ని ఒకేసారి (Go 1.20+)

కొన్నిసార్లు ఒకే operation లో **చాలా errors** వస్తాయి (ఉదా: batch validation – అన్ని fields check చేసి అన్ని తప్పులు ఒకేసారి report చేయాలి). వాటిని ఒకే error గా కలపడానికి `errors.Join`.

```
import "errors"

func validate(u User) error {
	var errs []error
	if u.Name == "" {
		errs = append(errs, errors.New("name ఖాళీ"))
	}
	if u.Age < 0 {
		errs = append(errs, errors.New("age negative"))
	}
	// errors.Join(nil, nil) == nil — ఏ error లేకపోతే nil return అవుతుంది
	return errors.Join(errs...)
}

func main() {
	err := validate(User{Name: "", Age: -5})
	fmt.Println(err)
	// name ఖాళీ
	// age negative   (ప్రతి error కొత్త line లో)

	// joined error కూడా errors.Is తో wrap-aware — ప్రతి sub-error ని match చేస్తుంది
	fmt.Println(errors.Is(err, err)) // true
}
```

Wrapping విధానం	ఎప్పుడు వాడాలి
`fmt.Errorf("...: %w", err)`	**ఒక** underlying error కి context జోడించడానికి (single chain)
`errors.Join(e1, e2, ...)`	**బహుళ** independent errors ని కలపడానికి (multi-error). `%w` ని రెండుసార్లు `fmt.Errorf` లో వాడి కూడా multi-wrap చేయవచ్చు (1.20+)

> **గమనిక:** `errors.Join` returned error లో `nil` entries automatically skip అవుతాయి; అన్నీ nil అయితే మొత్తం `nil`. కాబట్టి పైన `return errors.Join(errs...)` – errors లేకపోతే safe గా nil ఇస్తుంది.

panic vs error – ఎప్పుడు ఏది

error return	panic
Expected, recoverable failures	Programmer bugs, impossible states
File లేదు, network timeout, bad input	Nil dereference, array out of bounds

error return	panic
Caller handle చేయాలి	Unrecoverable, program bug
99% cases	Rare; boundaries లో recover

రూల్: library code errors return చేయాలి, panic కాదు. panic ని truly unrecoverable situations (nil config at startup, invariant violation) కి; server boundaries లో recover.

Best practices

```
// ✅ Wrap చేస్తూ context జోడించు
return fmt.Errorf("connect to db: %w", err)

// ✅ ఒకే విషయం రెండుసార్లు handle చేయకు (log + return కాదు)
// log OR return — రెండూ కాదు (double logging)

// ✅ Error messages: lowercase, punctuation లేకుండా (wrap చేయబడతాయి)
errors.New("connection refused")  // "failed to connect: connection refused"

// ✅ nil error return మర్చిపోకు success path లో
```

Real-life Scenario

> **Go error handling = ప్రతి తలుపు దగ్గర "ఇది తెరుచుకుందా?" అని చెక్ చేయడం.** Exception languages = "ఏదైనా తప్పైతే fire alarm మోగుతుంది, ఎవరో పైన catch చేస్తారు" – convenient కానీ ఎక్కడ, ఎందుకు fail అయిందో అస్పష్టం. Go = ప్రతి అడుగు దగ్గర "OK నా?" – verbose కానీ **error path స్పష్టంగా code లో కనిపిస్తుంది.**
>
> **Error wrapping (%w) = courier tracking.** Package fail అయితే, ప్రతి hub (function) "నా దగ్గర ఇలా జరిగింది: [+ కింది hub కారణం]" అని sticker వేస్తుంది. చివర్లో పూర్తి trail: "delivery failed: hub2 delay: hub1 truck breakdown" – root cause + full path.

Gotchas (సాధారణ తప్పులు)

- **Error ignore చేయడం** – `val, _ := f()` errors మింగేయడం. `go vet` /linters catch చేస్తాయి. Explicit handle చేయి.
- `err ==` **bad** wrapped errors తో – `errors.Is` వాడు.
- **typed-nil trap** (Topic 15) – custom error pointer nil ని error గా return చేస్తే `!= nil`.
- **Double handling** – log చేసి return కూడా → duplicate logs. ఒకటే చేయి.
- `%w` **vs** `%v` – `%v` string మాత్రమే (unwrap కాదు); `%w` wrap (Is/As పని చేస్తాయి). Chain కావాలంటే `%w`.
- **Sentinel over-use** – public API లో చాలా sentinels = tight coupling. Behavior-based errors (interfaces) కొన్నిసార్లు మంచివి.

Key Points

- **Errors are values** – `error` interface, explicit `if err != nil`.
- `errors.New`, `fmt.Errorf` (`%w` = wrap).
- **Sentinel errors** (`ErrNotFound`) + `errors.Is` (wrap-aware, `==` కాదు).
- **Custom types** + `Unwrap()` + `errors.As` (type extraction).
- **Wrap with context** (`%w`) but don't double-handle.
- **panic ≠ error** – panic bugs కి, error expected failures కి.

Interview దృష్టి

Q: Go exceptions ఎందుకు వాడదు? Trade-offs? A: Errors as values – explicit, visible in code, no hidden control flow. Trade-off: verbose (`if err != nil` పదేపదే). లాభం: error paths స్పష్టం, ఎక్కడ fail అవుతుందో స్పష్టం, forgotten error handling ని linters catch చేస్తాయి. Exceptions convenient కానీ invisible flow + తరచుగా unhandled.

Q: errors.Is vs errors.As? A: `errors.Is(err, ErrX)` – chain లో ఎక్కడైనా నిర్దిష్ట sentinel **value** ఉందా (unwraps). `errors.As(err, &target)` – chain లో నిర్దిష్ట **type** ఉందా, దాన్ని `target` లోకి extract చేస్తుంది. Value comparison → Is; type + fields కావాలంటే → As.

Q: `%w` వల్ల ఏం లాభం? A: Error ని wrap చేస్తూ original ని preserve చేస్తుంది – context జోడించి కూడా `errors.Is/As` chain లో original error ని కనుగొనగలం. `%v` కేవలం string, unwrap కాదు.

Part 3 — Concurrency (అత్యంత ముఖ్యం)

> ఇది Go యొక్క **అత్యంత గొప్ప feature** మరియు interviews లో అత్యధికంగా అడిగే section. Goroutines, GMP scheduler, channels, select, sync primitives, context, patterns, memory model — వీటిని లోతుగా చూద్దాం. తత్వం: **"Don't communicate by sharing memory; share memory by communicating"** (channels via) — కానీ mutex కూడా అవసరమే.
>
> **Concurrency vs Parallelism:** Concurrency = అనేక పనులని *deal* చేయగల structure (ఒకే core మీద కూడా — interleaving). Parallelism = అనేక పనులని *ఏకకాలంలో* execute చేయడం (multiple cores). Go concurrency ఇస్తుంది; hardware + scheduler parallelism ని enable చేస్తాయి.

---

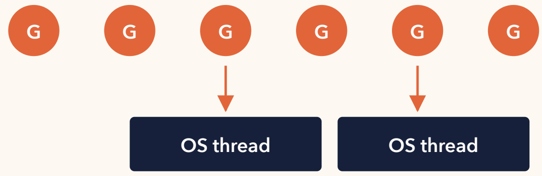
19. Goroutines

GOROUTINES · M:N SCHEDULING

GOROUTINE — OS thread కాదు, చాలా చౌక

OS Thread	Goroutine
~1 MB stack · kernel నిర్వహిస్తుంది Context switch ఖరీదు (kernel lo) వేలు సృష్టిస్తే machine కూలుతుంది	~2 KB stack — అవసరమైతే పెరుగుతుంది Go runtime నిర్వహిస్తుంది (user space) లక్షలు సృష్టించొచ్చు

M:N SCHEDULING — చాలా goroutines, కొన్ని threads



Runtime scheduler goroutines ని threads మీద పంచుతుంది

ఒక goroutine block అయితే — ఆ thread మీద **వేరే goroutine నడుస్తుంది**

JavaScript తో పోలిక

JS: ఒకే thread + event loop — async పని మాత్రమే సమాంతరం, CPU పని కాదు.
Go: నిజమైన సమాంతరత — goroutines వేర్వేరు CPU cores మీద ఏకకాలంలో నడుస్తాయి.

వివరణ

Goroutine = Go runtime manage చేసే ఒక lightweight thread. `go` keyword ఒక function ముందు పెడితే, అది కొత్త goroutine లో concurrently run అవుతుంది. దీని magic: ***చాలా చవక*** — initial stack కేవలం **2KB** (OS thread ~1-2MB కంటే వెయ్యి రెట్లు తక్కువ), అవసరమైతే dynamically grow అవుతుంది. అందుకే ఒక Go program **లక్షలాది** goroutines run చేయగలదు.

```go
func sayHello(name string) {
	fmt.Println("Hello,", name)
}

func main() {
	go sayHello("Surya")   // కొత్త goroutine లో — concurrent
	go sayHello("Ravi")

	time.Sleep(time.Millisecond) // main goroutine wait (లేకపోతే program exit)
	fmt.Println("main done")
}
```

కీలకం: `main` కూడా ఒక goroutine (main goroutine). **main return అయితే, program మొత్తం exit** – మిగతా goroutines పూర్తయినా కాకపోయినా చంపబడతాయి. అందుకే పైన `time.Sleep` (production లో WaitGroup వాడతాం, Topic 23).

Goroutine vs OS Thread

అంశం	Goroutine	OS Thread
Managed by	Go runtime (user-space)	OS kernel
Initial stack	2KB (grows/shrinks)	1-2MB (fixed)
Creation cost	చాలా చవక (~μs)	ఖరీదు (syscall)
Context switch	Fast (user-space, no kernel)	నెమ్మది (kernel trap)
Scheduling	Cooperative + preemptive (Go scheduler)	Preemptive (OS)
సంఖ్య	లక్షలు సాధ్యం	వేలు (memory limited)
Identity	No thread ID (intentional)	Has TID
Communication	Channels (idiomatic)	Shared memory + locks

కీలక insight: Go **N goroutines ని M OS threads మీద multiplex చేస్తుంది** (M:N scheduling – Topic 20). 10 lakh goroutines ఉన్నా, అవి కొన్ని OS threads (GOMAXPROCS ~ CPU cores) మీద run అవుతాయి.

Goroutine lifecycle

1. `go f()` → goroutine create, run queue లో పెట్టబడుతుంది.
2. Scheduler దాన్ని ఒక P (logical processor) కి assign, M (OS thread) మీద run.
3. Blocks (channel/syscall/sleep) → parked, thread free అవుతుంది.
4. Ready అయితే మళ్ళీ queue లో.
5. Function return → goroutine terminates, stack freed.

Stack growth – 2KB నుండి

Goroutine stack fixed కాదు. Function call deep అయి 2KB చాలకపోతే, runtime **పెద్ద stack (2×) allocate చేసి, పాత stack ని copy చేసి**, pointers ని adjust చేస్తుంది (contiguous/copying stacks). అవసరం తగ్గితే shrink కూడా. అందుకే recursion deep అయినా crash కాదు (limit ~1GB వరకు).

Goroutine Leaks – silent killer

Goroutine ఎప్పటికీ terminate అవ్వకపోతే (blocked forever) = **leak.** Memory + resources వృథా. Common cause: channel మీద forever block, లేదా cancellation లేకపోవడం.

```
// ❌ LEAK: ఈ goroutine ch మీద forever block అవుతుంది (ఎవరూ send చేయరు)
func leak() {
	ch := make(chan int)
	go func() {
		val := <-ch   // forever blocks — leak!
		fmt.Println(val)
	}()
	// ch కి ఎవరూ send చేయరు, goroutine ఎప్పటికీ ముగియదు
}

// ✅ FIX: context/done channel తో cancellation
func noLeak(ctx context.Context) {
	ch := make(chan int)
	go func() {
		select {
		case val := <-ch:
			fmt.Println(val)
		case <-ctx.Done():   // cancel → goroutine exits
			return
		}
	}()
}
```

Real-life Scenario

> **OS thread = ఒక పూర్తి ప్రొఫెషనల్ chef ని hire చేయడం** – జీతం ఎక్కువ (1-2MB), hire/fire ఖరీదు, restaurant లో 20-30 మంది మాత్రమే పెట్టగలవు.
>
> **Goroutine = ఒక చిన్న task-card.** ఒక్కో card చవక (2KB), వేలాది cards రాయగలవు. కొద్దిమంది chefs (OS threads) ఈ వేలాది cards ని చకచకా తీసుకుని పని చేస్తారు (M:N scheduling). ఒక card "ingredient రావాలి" అని wait చేస్తే (blocked), chef ఆ card పక్కన పెట్టి వేరే card తీసుకుంటాడు – chef idle గా ఉండడు.
>
> **Goroutine leak = ఎప్పటికీ complete అవ్వని task-card** – "పండు రావాలి" అని రాసి ఉంది, కానీ ఆ పండు ఎప్పటికీ రాదు. Card stack పేరుకుపోతుంది, memory నిండిపోతుంది.

Gotchas (సాధారణ తప్పులు)

- **main exit → అన్ని goroutines చస్తాయి.** WaitGroup/channel తో wait చేయి.
- **Goroutine leaks** – cancellation (context/done) లేకుండా blocking goroutines. `runtime.NumGoroutine()` monitor.

- `go` **loop variable capture** (Go 1.21-) – `for _, v := range xs { go func(){ use(v) }() }` అన్నీ చివరి v. (1.22+ fixed; పాత Go లో `go func(v int){}(v)`).
- **Unhandled panic in goroutine → మొత్తం program crash** (recover ఆ goroutine లోనే ఉండాలి).
- **Goroutine result ని ignore చేయడం** – error/panic silently lost.

Key Points

- Goroutine = lightweight thread, **2KB stack** (grows), Go runtime managed.
- `go f()` → concurrent; **main goroutine exit = program exit.**
- **M:N** – లక్షల goroutines కొన్ని OS threads మీద multiplex.
- Stack **copying growth** (2× when needed); no fixed limit (~1GB).
- **Goroutine leaks** = blocked-forever goroutines → cancellation తో fix (context/done).
- Panic in goroutine → crash unless recovered *in that goroutine*.

Interview దృష్టి

Q: Goroutine vs OS thread? A: Goroutine = Go runtime managed user-space lightweight thread, 2KB initial stack (grows), μs creation, fast user-space context switch. OS thread = kernel managed, 1-2MB fixed stack, ఖరీదైన creation/switch. Go N goroutines ని M threads మీద multiplex (M:N) చేస్తుంది – లక్షల goroutines సాధ్యం.

Q: Goroutine leak అంటే? ఎలా తప్పించాలి? A: ఒక goroutine ఎప్పటికీ terminate అవ్వకపోవడం (సాధారణంగా channel మీద forever block లేదా cancellation లేకపోవడం) – memory/resource వృథా. Fix: `context` లేదా done channel తో cancellation, `select` లో `<-ctx.Done()`, buffered channels సరిగ్గా వాడటం. `runtime.NumGoroutine()` తో monitor.

Q: main goroutine return అయితే? A: మొత్తం program వెంటనే exit – మిగతా goroutines పూర్తయినా కాకపోయినా చంపబడతాయి (graceful shutdown లేదు). అందుకే WaitGroup/channel తో వాటి completion కోసం wait చేయాలి.

---

20. GMP Scheduler

GMP SCHEDULER · GOROUTINE, MACHINE, PROCESSOR

GMP SCHEDULER

G	Goroutine	మన code · లక్షలు ఉండొచ్చు
M	Machine	OS thread · పరిమితం
P	Processor	context · GOMAXPROCS కి సమానం

Work stealing

ప్రతి P కి తన సొంత run queue. తన queue ఖాళీ అయితే – **వేరే P నుంచి దొంగిలిస్తుంది**.
దీంతో load ఆటోమేటిక్ గా సమానమవుతుంది, central lock అవసరం లేకుండా.

Blocking syscall జరిగితే

G ఒక syscall lo ఇరుక్కుంటే – ఆ M కూడా block అవుతుంది.
Runtime ఆ P ని **విడదీసి వేరే M కి** అతికిస్తుంది, మిగతా goroutines ఆగవు.
అందుకే Go lo blocking I/O రాసినా – మొత్తం program స్తంభించదు.

వివరణ

Go runtime లో goroutines ని OS threads మీద ఎలా run చేయాలో నిర్ణయించేది **GMP scheduler** (aka goroutine scheduler). ఇది Go concurrency యొక్క గుండె. మూడు అక్షరాలు:

- **G (Goroutine)** – ఒక goroutine: దాని stack, instruction pointer, state.
- **M (Machine)** – ఒక OS thread (kernel thread). నిజంగా code execute చేసేది.
- **P (Processor)** – ఒక **logical processor / scheduling context.** ప్రతి P కి ఒక **local run queue** (goroutines) ఉంటుంది. **P సంఖ్య = GOMAXPROCS** (default = CPU cores).

రూల్: ఒక G run అవ్వాలంటే, ఒక M కి ఒక P attach అయి ఉండాలి. **G ని run చేయడానికి M కి P కావాలి.** P సంఖ్య fixed (GOMAXPROCS) – అందుకే ఏకకాలంలో run అయ్యే goroutines సంఖ్య ~ GOMAXPROCS.

ASCII diagram

```
        ┌──────────── Global Run Queue (GRQ) ────────────┐
        │   G   G   G   G   (overflow / fairness)         │
        └─────────────────────────────────────────────────┘
                │              │              │
   ┌────────────┴──┐  ┌────────┴──────┐  ┌────┴──────────┐
   │  P0           │  │  P1           │  │  P2           │  (GOMAXPROCS = 3)
   │ LRQ: G G G    │  │ LRQ: G G      │  │ LRQ: (empty)  │  Local Run Queues
   └───────┬───────┘  └───────┬───────┘  └──────┬────────┘
           │ attached         │                 │ steals from P0!
      ┌────┴────┐        ┌────┴────┐       ┌────┴────┐
      │  M0     │        │  M1     │       │  M2     │     OS threads
      │(running │        │(running │       │(running │
      │  a G)   │        │  a G)   │       │  a G)   │
      └─────────┘        └─────────┘       └─────────┘
```

Run queues – local + global

- **Local Run Queue (LRQ)** – ప్రతి P కి own queue (~256 goroutines capacity). Lock-free (సాధారణంగా), fast. కొత్త goroutine సాధారణంగా current P యొక్క LRQ లో పడుతుంది.
- **Global Run Queue (GRQ)** – అన్ని Ps share చేసే queue. LRQ overflow అయినప్పుడు, లేదా fairness కోసం (ప్రతి 61 schedules కి ఒకసారి GRQ check).

Work-stealing – idle P ఖాళీగా ఉండదు

ఒక P యొక్క LRQ ఖాళీ అయితే (పని లేదు), అది idle గా కూర్చోదు – **work-stealing:** వేరే P యొక్క LRQ నుండి సగం goroutines దొంగిలిస్తుంది (steal). GRQ నుండి కూడా తీసుకుంటుంది. ఇది load balancing ని automatic గా చేస్తుంది – ఏ core idle గా వృథా అవ్వదు.

GOMAXPROCS

`runtime.GOMAXPROCS(n)` = ఏకకాలంలో ఎన్ని Ps (అంటే ఎన్ని goroutines actively running). Default = `runtime.NumCPU()`. CPU-bound work కి cores సంఖ్య optimal. (Go 1.5+ default = all cores.)

```
runtime.GOMAXPROCS(4)  // 4 Ps
fmt.Println(runtime.NumCPU())      // machine cores
fmt.Println(runtime.NumGoroutine()) // live goroutines
```

Preemption – 1.14 నుండి asynchronous

మొదట్లో Go **cooperative scheduling** – goroutine function call, channel op, లేదా allocation దగ్గరే yield చేసేది. దీనివల్ల **tight loop** (function call లేని `for {}`) ఒక P ని forever hog చేయగలదు (starvation). **Go 1.14 లో asynchronous preemption** వచ్చింది – runtime signal (SIGURG) పంపి, ఏ safe point లోనైనా goroutine ని preempt చేయగలదు. ఇప్పుడు 10ms కంటే ఎక్కువ run అయ్యే goroutine ని preempt చేస్తారు (fairness).

Syscalls & handoff

Goroutine ఒక **blocking syscall** (file read, network – kernel level) చేస్తే, ఆ M block అవుతుంది. ఆ M యొక్క **P ని detach** చేసి వేరే M (లేదా కొత్త M) కి ఇస్తారు (**handoff**) – తద్వారా ఆ P యొక్క మిగతా goroutines ఆగవు. Syscall పూర్తయితే, ఆ goroutine మళ్ళీ ఒక P కోసం queue అవుతుంది. అందుకే blocking syscalls goroutines ని block చేసినా, మిగతా work కొనసాగుతుంది.

Netpoller integration – network I/O magic

Network I/O కి Go **netpoller** వాడుతుంది (epoll/kqueue/IOCP OS-specific). Goroutine network read/write చేస్తే block అయితే, ఆ M block అవ్వదు – goroutine netpoller కి "park" అవుతుంది, M free అయి వేరే goroutines run చేస్తుంది. Data ready అయితే (epoll notify), netpoller ఆ goroutine ని runnable గా mark చేస్తుంది. **అందుకే Go 1 lakh concurrent network connections ని కొన్ని threads తో handle చేయగలదు** – thread-per-connection అవసరం లేదు. ఇదే Go ని network servers కి perfect చేస్తుంది.

Real-life Scenario

> **P = ఒక వంట స్టేషన్ (కటింగ్ బోర్డ్ + అవసరమైన tools), M = ఒక chef, G = ఒక order ticket.**
>
> - Chef (M) ఒక స్టేషన్ (P) దగ్గర నిలబడి, ఆ స్టేషన్ యొక్క ticket-holder (LRQ) నుండి orders తీసి వండుతాడు.
> - స్టేషన్ల సంఖ్య fixed (GOMAXPROCS = 3 స్టేషన్లు) – ఒకేసారి 3 orders active.
> - ఒక స్టేషన్ tickets అయిపోతే, ఆ chef పక్క స్టేషన్ నుండి సగం tickets దొంగిలిస్తాడు (**work-stealing**) – ఖాళీగా ఉండడు.
> - ఒక order "oven లో 20 నిమి కాలాలి" (blocking syscall) అంటే, chef అక్కడే waste అవ్వకుండా, ఆ స్టేషన్ ని వేరే chef కి handoff చేసి, తను oven monitor చేస్తాడు.
> - ఒక order పదార్థం రావాలని wait (network I/O) చేస్తే, waiter (netpoller) ఆ ticket పక్కన పెట్టి, "పదార్థం వచ్చాక చెప్తా" అంటాడు – chef వేరే orders వండుతాడు.

GMP terms table

అక్షరం	పూర్తి పేరు	ఏమిటి
G	Goroutine	పని unit (stack, PC, state)
M	Machine	OS thread – నిజంగా execute చేస్తుంది
P	Processor	scheduling context + LRQ; సంఖ్య = GOMAXPROCS
LRQ	Local Run Queue	per-P goroutine queue (fast, ~256)
GRQ	Global Run Queue	shared queue (overflow, fairness)

Gotchas (సాధారణ తప్పులు)

- **Tight loop without preemption points** (pre-1.14) ఒక P ని hog చేస్తుంది. 1.14+ async preemption దీన్ని పరిష్కరించింది, కానీ ఇంకా CPU-bound loops ని `runtime.Gosched()` తో yield చేయడం సహాయపడవచ్చు.
- **GOMAXPROCS ని అతిగా పెంచడం** CPU-bound work లో context-switch overhead పెంచుతుంది. I/O-bound కి goroutines ఎక్కువ, GOMAXPROCS cores.
- **CGo calls M ని block చేస్తాయి** (native code) – P handoff, thread pool పెరుగుతుంది.
- `GOMAXPROCS=1` అనుకుని concurrency bugs దాగవు – ఒక్క P అయినా preemption వల్ల interleaving.

Key Points

- **G/M/P:** G=goroutine, M=OS thread, P=scheduling context (GOMAXPROCS).
- G run అవ్వాలంటే M+P కావాలి; parallel goroutines ~ **GOMAXPROCS**.
- **Local (per-P) + Global** run queues; **work-stealing** = load balance.
- **Async preemption** (1.14+) – tight loops కూడా preempt (fairness).
- **Syscall handoff** – blocking syscall లో P detach → work కొనసాగుతుంది.

- **Netpoller** (epoll/kqueue) – network I/O block అయినా M free → lakhs of connections.

Interview దృష్టి

Q: GMP model వివరించు. A: G = goroutine (work unit), M = OS thread (executes), P = logical processor with local run queue (scheduling context, count = GOMAXPROCS). G run అవ్వాలంటే M ఒక P ని hold చేయాలి. Goroutines Ps యొక్క local run queues లో ఉంటాయి; idle P work-stealing చేస్తుంది. ఇది M:N scheduling – లక్షల goroutines కొన్ని threads మీద.

Q: Blocking syscall / network I/O లో scheduler ఏం చేస్తుంది? A: Blocking syscall – M block అవుతుంది కానీ దాని P ని వేరే M కి handoff చేస్తారు, ఇతర goroutines కొనసాగుతాయి. Network I/O – goroutine ని netpoller (epoll/kqueue) కి park చేస్తారు, M block అవ్వదు, data ready అయితే netpoller goroutine ని runnable చేస్తుంది. అందుకే Go network-heavy workloads ని efficiently handle చేస్తుంది.

Q: Work-stealing అంటే? GOMAXPROCS? A: ఒక P యొక్క local run queue ఖాళీ అయితే, వేరే P నుండి సగం goroutines steal చేసి load balance చేస్తుంది. GOMAXPROCS = active Ps సంఖ్య = ఏకకాలంలో run అయ్యే goroutines గరిష్ఠం, default = CPU cores.

---

21. Channels

CHANNELS · BUFFERED VS UNBUFFERED

CHANNEL – goroutines మధ్య సురక్షితమైన పైపు

UNBUFFERED (cap 0)

Sender receiver కోసం **వేచి ఉంటుంది**
ఇద్దరూ ఒకేసారి కలుసుకోవాలి (rendezvous)
ఇది ఒక synchronisation సాధనం కూడా

BUFFERED (cap n)

Buffer నిండేదాకా sender ఆగడు
నిండాక మాత్రమే block అవుతుంది
Producer-consumer కి ఇదే సరైనది

మూడు ఉచ్చులు

1 · మూసిన channel కి పంపితే → **panic**. మూసేది ఎప్పుడూ sender మాత్రమే.
2 · nil channel మీద పని → శాశ్వతంగా block.
3 · Receiver లేని unbuffered channel → deadlock. Go దీన్ని runtime lo పట్టుకుంటుంది.

Go యొక్క నినాదం: **"Do not communicate by sharing memory; share memory by communicating."** – mutex తో shared variable ని కాపాడటం కంటే, ownership ని channel ద్వారా బదిలీ చేయడం సురక్షితం.

వివరణ

Channel = goroutines మధ్య typed values పంపే pipe. Go concurrency తత్వం: **"Don't communicate by sharing memory; share memory by communicating."** Channels ద్వారా data pass చేస్తే, ownership transfer అవుతుంది – locks అవసరం తగ్గుతుంది.

```
ch := make(chan int)      // unbuffered channel of int
ch <- 42                  // send (arrow channel లోకి)
```

```
val := <-ch                // receive (arrow channel నుండి)
close(ch)                  // channel మూసేయడం
```

Unbuffered vs Buffered – కీలక తేడా

అంశం	**Unbuffered** `make(chan T)`	**Buffered** `make(chan T, n)`
Capacity	0	n
Send blocks	receiver ready అయ్యేవరకు	buffer నిండేవరకు మాత్రమే
Receive blocks	sender ready అయ్యేవరకు	buffer ఖాళీ అయ్యేవరకు
Semantics	**Synchronous** (rendezvous/handshake)	**Asynchronous** (up to n)
వాడకం	Synchronization, guarantee delivery	Decoupling, bursts absorb

Unbuffered = handshake: send మరియు receive ఏకకాలంలో జరగాలి – sender receiver కోసం wait చేస్తాడు, vice versa. ఇది strong synchronization + happens-before guarantee ఇస్తుంది.

Buffered = mailbox: buffer లో n slots ఉంటే, receiver లేకున్నా n values పంపవచ్చు (block కాకుండా). Buffer నిండితే send blocks.

```
// Unbuffered — synchronous handshake
ch := make(chan string)
go func() { ch <- "data" }()  // receiver వచ్చేవరకు block
msg := <-ch                    // ఇక్కడ handshake పూర్తి
fmt.Println(msg)

// Buffered — 2 sends block కాకుండా
buf := make(chan int, 2)
buf <- 1  // OK (slot 1)
buf <- 2  // OK (slot 2)
// buf <- 3  // ఇది block అవుతుంది (buffer full)
```

close & range over channel

`close(ch)` = "ఇక values రావు" signal. Closed channel నుండి receive:

- మిగిలిన buffered values వస్తాయి, తర్వాత **zero value + ok=false.**
- `for v := range ch` closed అయ్యేవరకు receive చేసి, ఆగుతుంది.

```
ch := make(chan int, 3)
ch <- 1; ch <- 2; ch <- 3
close(ch)

for v := range ch {    // 1 2 3 తర్వాత loop ముగుస్తుంది (closed)
        fmt.Println(v)
}
```

```
// comma-ok — closed detect
v, ok := <-ch          // ok=false అంటే closed & empty
```

రూల్: sender close చేయాలి, receiver కాదు. Closed channel కి send = panic.

nil channel – deadlock trap

`nil` channel మీద send/receive **forever blocks** (deadlock). ఇది bug కావచ్చు, కానీ `select` లో ఉద్దేశపూర్వకంగా case ని disable చేయడానికి కూడా వాడతారు (Topic 22).

```
var ch chan int   // nil channel
// <-ch      // forever blocks (deadlock)
// ch <- 1   // forever blocks
```

Directional channels – type safety

Function signatures లో channel direction restrict చేయవచ్చు – send-only లేదా receive-only. Misuse ని compile-time లో ఆపుతుంది.

```
func producer(out chan<- int) {   // send-only (chan<-)
        for i := 0; i < 3; i++ {
                out <- i
        }
        close(out)
}
func consumer(in <-chan int) {    // receive-only (<-chan)
        for v := range in {
                fmt.Println(v)
        }
}

ch := make(chan int)
go producer(ch)  // bidirectional → send-only auto-convert
consumer(ch)     // → receive-only
```

Deadlock – Go detects it

అన్ని goroutines blocked అయి progress లేకపోతే, Go runtime **"fatal error: all goroutines are asleep - deadlock!"** తో crash చేస్తుంది.

```
func main() {
        ch := make(chan int)
        ch <- 1   // ❌ deadlock! receiver లేడు, unbuffered → forever block
        // fatal error: all goroutines are asleep - deadlock!
}
```

Channel state cheat sheet

Operation	nil channel	open channel	closed channel
Send `ch <-`	forever block	block/proceed	**panic**
Receive `<-ch`	forever block	block/proceed	zero value, ok=false
Close `close(ch)`	**panic**	OK	**panic** (double close)

Internals – hchan structure

Channel internally `hchan` struct (heap-allocated): buffer (ring buffer for buffered), send/receive queues (waiting goroutines – `sudog`), a mutex, closed flag. Send/receive lock తీసుకుని operate చేస్తాయి. Unbuffered లో value నేరుగా sender stack నుండి receiver కి copy (buffer bypass) – optimization.

Real-life Scenario

> **Unbuffered channel = ఇద్దరి మధ్య చేతికి చెయ్యి baton pass (relay race).** ఇద్దరూ ఏకకాలంలో సిద్ధంగా ఉండాలి – ఇచ్చేవాడు, తీసుకునేవాడు. లేకపోతే ఒకరు wait. Baton pass అయిన క్షణంలో "handshake" – ఖచ్చితంగా synchronized.
>
> **Buffered channel = post box (n ఉత్తరాలు పట్టేది).** Postman (receiver) లేకున్నా, box లో n ఉత్తరాలు వేయవచ్చు (block కాకుండా). Box నిండితే, ఖాళీ అయ్యేవరకు wait. Sender, receiver decoupled.
>
> **close = "ఇక ఉత్తరాలు రావు" board.** Receiver box లోని మిగతా ఉత్తరాలు తీసుకుని, board చూసి ఆగుతాడు.

Gotchas (సాధారణ తప్పులు)

- **Closed channel కి send → panic.** Sender close చేయాలి, ownership స్పష్టం.
- **Double close → panic.** Once/sync తో guard.
- **nil channel op → forever block** (deadlock) – initialize మర్చిపోవద్దు.
- **Unbuffered channel + no receiver → deadlock.** main లో common.
- **Range over never-closed channel → forever block** (goroutine leak).
- **Buffered channel size ని performance fix గా అతిగా వాడటం** – deadlock ని దాచవచ్చు, root cause hide.

Key Points

- Channel = typed pipe; "share memory by communicating."
- **Unbuffered** = synchronous handshake (send↔receive rendezvous).
- **Buffered** (`make(chan T, n)`) = async up to n; full→send blocks.
- **Sender closes**; closed → remaining values then zero/ok=false; `range` stops.
- Send to closed / double close = **panic**; nil channel = forever block.

- **Directional** (`chan<-` , `<-chan`) = compile-time safety.
- Deadlock (all blocked) → runtime fatal error.

Interview దృష్టి

Q: Unbuffered vs buffered channel? A: Unbuffered (`make(chan T)`) – synchronous: send receiver ready అయ్యేవరకు block (rendezvous, strong sync + happens-before). Buffered (`make(chan T, n)`) – up to n values buffer లో, receiver లేకున్నా send అవుతుంది (async decoupling); buffer full అయితే send blocks. Sync guarantee కావాలంటే unbuffered; bursts absorb / decouple కి buffered.

Q: Closed channel మీద ఏం జరుగుతుంది? A: Receive – remaining buffered values, తర్వాత zero value + `ok=false` (`range` ఆగుతుంది). Send to closed channel = **panic**. Double close = panic. Nil channel send/receive = forever block. Sender close చేయాలి (receiver కాదు).

Q: nil channel వాడకం ఉందా? A: nil channel op forever block చేస్తుంది. `select` లో ఒక case యొక్క channel ని nil చేస్తే, ఆ case శాశ్వతంగా disable అవుతుంది – dynamic case enable/disable కి useful pattern (ఉదా. producer done అయ్యాక ఆ channel ని nil).

---

22. select

వివరణ

`select` **= అనేక channel operations మీద ఒకేసారి wait చేయడం.** ఇది switch లా కనిపిస్తుంది కానీ cases channel ops. ఏ case ready అయితే అది run అవుతుంది; అనేకం ready అయితే **randomly ఒకటి** ఎంచుకుంటుంది (fairness). ఇది Go concurrency యొక్క multiplexer.

```
select {
case msg1 := <-ch1:
        fmt.Println("ch1:", msg1)
case msg2 := <-ch2:
        fmt.Println("ch2:", msg2)
case ch3 <- 42:
        fmt.Println("sent to ch3")
}
// ఏ ఒక్క case ready అయ్యేవరకు block; ready case run
```

default – non-blocking select

`default` case ఉంటే, ఏ channel ready కాకపోతే వెంటనే `default` run – **block అవ్వదు.** Polling కి ఉపయోగం.

```
select {
case msg := <-ch:
        fmt.Println("received:", msg)
default:
```

```
        fmt.Println("no message (non-blocking)")
}
```

Timeout – time.After

`time.After(d)` ఒక channel return చేస్తుంది, d తర్వాత value వస్తుంది. select తో timeout patterns:

```
select {
case result := <-workCh:
        fmt.Println("got:", result)
case <-time.After(2 * time.Second):
        fmt.Println("timeout! పని 2s లో పూర్తి కాలేదు")
}
```

Gotcha: loop లో `time.After` ప్రతిసారి కొత్త timer allocate చేస్తుంది – hot loops లో `time.NewTimer` + reset లేదా context deadline వాడు.

for-select loop – event loop pattern

Long-running goroutines కి అత్యంత common pattern – multiple channels ని continuously handle చేయడం.

```
func worker(jobs <-chan int, done <-chan struct{}) {
        for {
                select {
                case job := <-jobs:
                        fmt.Println("processing:", job)
                case <-done:
                        fmt.Println("shutting down")
                        return   // goroutine exit – leak లేదు
                }
        }
}
```

done channel – cancellation signal

`chan struct{}` (0 bytes) ని "stop" signal గా వాడటం idiomatic. Close చేస్తే అన్ని receivers ఏకకాలంలో unblock అవుతాయి (broadcast).

```
done := make(chan struct{})
go func() {
        for {
                select {
                case <-done:
                        return    // cancel
                default:
                        // పని చేయి (non-blocking)
                }
        }
}()
close(done)  // అన్ని goroutines కి "ఆగండి" broadcast
```

కీలకం: `close(done)` ఒకసారి, అన్ని receivers ని unblock చేస్తుంది. Value పంపడం (`done <- struct{}{}`) ఒక్క receiver కి మాత్రమే – broadcast కాదు.

empty select – forever block

```
select {}   // forever blocks — అన్ని goroutines పని చేస్తుంటే main ని alive గా ఉంచడానికి
(అరుదు)
```

Real-life Scenario

> **select = ఒక receptionist అనేక ఫోన్ లైన్ల ముందు.** ఏ లైన్ మోగితే దాన్ని ఎత్తుతాడు (ready case). రెండు ఏకకాలంలో మోగితే ఒకదాన్ని randomly. ఏదీ మోగకపోతే wait (block), కానీ "default" ఉంటే "ప్రస్తుతం కాల్ లేదు" అని వేరే పని చేస్తాడు (non-blocking).
>
> **timeout = "2 నిమిషాల్లో ఎవరూ కాల్ చేయకపోతే lunch కి వెళ్తా"** (`time.After`). **done channel close = ఆఫీస్ లో "అందరూ ఇంటికి వెళ్ళండి" అనౌన్స్‌మెంట్** – ఒక్కసారి చెప్తే అందరూ (all receivers) వింటారు.

Gotchas (సాధారణ తప్పులు)

- **`time.After` in loop = timer leak** – ప్రతి iteration కొత్త timer (GC అయ్యేవరకు live). `context.WithTimeout` లేదా reusable `time.Timer` వాడు.
- **default లేని select + no ready case = block** (deadlock if forever).
- **Empty select `select{}`** = forever block (main hang) – ఉద్దేశపూర్వకంగానే వాడు.
- **Value send done కి** ఒక్క receiver ని unblock చేస్తుంది; **broadcast కి close(done)** వాడు.
- **select randomness** – multiple ready cases fair random; order మీద ఆధారపడకు.

Key Points

- `select` = multiplex multiple channel ops; ready case runs.
- Multiple ready → **random** (fairness).
- **`default`** = non-blocking select (polling).
- **`time.After`** = timeout pattern (loop లో leak – గమనించు).
- **for-select** = event loop; **`<-ctx.Done()` /done channel** = cancellation.
- **`close(done)`** = broadcast unblock (all receivers); send = one receiver.

Interview దృష్టి

Q: select ఏం చేస్తుంది? Multiple cases ready అయితే? A: అనేక channel operations మీద ఏకకాలంలో wait చేస్తుంది; ఏ case ready అయితే అది execute. Multiple ready అయితే **uniformly random** ఒకటి ఎంచుకుంటుంది (starvation తప్పించడం). `default` ఉంటే non-blocking – ఏదీ ready కాకపోతే వెంటనే default.

Q: Timeout ఎలా implement చేస్తావు? A: `select` లో `case <-time.After(d)` (లేదా better: `context.WithTimeout` + `<-ctx.Done()`). Work channel మరియు timeout channel రెండింటిపై wait

– ఏది ముందు వస్తే అది. Loops లో `time.After` timer leak తప్పించడానికి context deadline లేదా reused timer.

Q: done channel ఎలా broadcast చేస్తుంది? A: `close(done)` – closed channel receive వెంటనే return అవుతుంది (zero value), కాబట్టి `<-done` మీద wait చేస్తున్న **అన్ని** goroutines ఏకకాలంలో unblock. Value పంపడం ఒక్క receiver ని మాత్రమే unblock చేస్తుంది.

---

23. sync Package

వివరణ

Channels అన్ని concurrency problems కి answer కాదు. **Shared memory ని protect చేయడానికి `sync` package** primitives ఇస్తుంది – Mutex, RWMutex, WaitGroup, Once, Cond, Map, Pool. తత్వం: **"channels for orchestration, mutexes for shared state."** ఒక counter ని protect చేయడానికి channel కంటే mutex సులభం & fast.

sync.Mutex – mutual exclusion

ఒకేసారి ఒక goroutine మాత్రమే critical section లోకి. `Lock()` / `Unlock()`.

```
type Counter struct {
	mu    sync.Mutex
	count int
}
func (c *Counter) Inc() {
	c.mu.Lock()
	defer c.mu.Unlock()   // panic-safe unlock
	c.count++             // critical section – protected
}
```

- `sync.Mutex` **zero value ready** – `var mu sync.Mutex` వెంటనే usable ("zero value useful").
- **Mutex ని copy చేయకూడదు** – `go vet` catch చేస్తుంది. Struct లో pointer receiver methods వాడు.

sync.RWMutex – reader/writer lock

Reads ఎక్కువ, writes తక్కువ అయితే. **Multiple readers ఏకకాలంలో** (RLock), కానీ writer exclusive (Lock).

```
type Cache struct {
	mu   sync.RWMutex
	data map[string]int
}
func (c *Cache) Get(k string) int {
	c.mu.RLock()         // read lock – concurrent readers OK
	defer c.mu.RUnlock()
	return c.data[k]
}
```

```
func (c *Cache) Set(k string, v int) {
        c.mu.Lock()           // write lock — exclusive
        defer c.mu.Unlock()
        c.data[k] = v
}
```

Read-heavy workloads లో RWMutex Mutex కంటే వేగం. కానీ write-heavy లో overhead ఎక్కువ (RWMutex complex) – profile చేసి నిర్ణయించు.

sync.WaitGroup – goroutines కోసం wait

అనేక goroutines పూర్తయ్యేవరకు wait చేయడానికి. `Add(n)` , `Done()` , `Wait()` .

```
func main() {
        var wg sync.WaitGroup
        for i := 0; i < 5; i++ {
                wg.Add(1)            // goroutine start ముందు Add
                go func(id int) {
                        defer wg.Done()  // పూర్తయితే Done
                        fmt.Println("worker", id)
                }(i)
        }
        wg.Wait()                 // అన్ని Done అయ్యేవరకు block
        fmt.Println("all done")
}
```

Gotcha: `wg.Add` ని goroutine లోపల కాదు, **ముందు** (goroutine start చేయకముందు) call చేయాలి – race తప్పించడానికి.

sync.Once – ఒకేసారి execution

Initialization ని ఖచ్చితంగా ఒక్కసారి (అనేక goroutines call చేసినా). Singletons, lazy init కి.

```
var (
        once     sync.Once
        instance *DB
)
func GetDB() *DB {
        once.Do(func() {        // ఒక్కసారే execute (thread-safe)
                instance = connectDB()
        })
        return instance
}
```

sync.Cond – condition variable

ఒక condition true అయ్యేవరకు goroutines wait చేసి, signal వచ్చాక wake. Producer-consumer లాంటి complex sync లో (అరుదు – channels ఎక్కువ వాడతారు).

```
var mu sync.Mutex
cond := sync.NewCond(&mu)
ready := false
```

```
// waiter
mu.Lock()
for !ready {
	cond.Wait()   // unlocks mu, waits, re-locks on wake
}
mu.Unlock()

// signaler
mu.Lock()
ready = true
cond.Broadcast()  // (లేదా Signal() — ఒక్కరిని)
mu.Unlock()
```

sync.Map — concurrent map (special cases)

Built-in map concurrency-safe కాదు. `sync.Map` lock-free-ish concurrent map — కానీ **specific patterns కి మాత్రమే optimized:** (1) keys ఒకసారి write, చాలాసార్లు read; (2) disjoint keys per goroutine. General cases కి `map + RWMutex` ఎక్కువ మంచిది.

```
var m sync.Map
m.Store("key", 42)
v, ok := m.Load("key")     // 42, true
m.LoadOrStore("k2", 100)   // atomic get-or-set
m.Delete("key")
m.Range(func(k, v any) bool { fmt.Println(k, v); return true })
```

sync.Pool — object reuse (GC pressure తగ్గించడం)

Temporary objects ని reuse చేయడానికి — allocations, GC pressure తగ్గిస్తుంది. Buffers, large structs కి. **Pool contents ఎప్పుడైనా GC అవ్వవచ్చు** — persistent storage కి కాదు.

```
var bufPool = sync.Pool{
	New: func() any { return new(bytes.Buffer) },
}
func process() {
	buf := bufPool.Get().(*bytes.Buffer)
	defer func() {
		buf.Reset()
		bufPool.Put(buf)   // తిరిగి pool కి
	}()
	// buf వాడు
}
```

sync primitives table

Primitive	ఎప్పుడు వాడాలి
`Mutex`	Shared state ని protect (single writer/reader at a time)
`RWMutex`	Read-heavy shared state (concurrent reads)

Primitive	ఎప్పుడు వాడాలి
`WaitGroup`	అనేక goroutines completion కోసం wait
`Once`	ఖచ్చితంగా-ఒకసారి init (singleton, lazy)
`Cond`	Complex condition-based wait/signal (అరుదు)
`Map`	Concurrent map – write-once-read-many, disjoint keys
`Pool`	Temporary object reuse – GC pressure తగ్గించడం

Real-life Scenario

> **Mutex = బాత్రూమ్ తలుపు తాళం.** ఒకేసారి ఒకరు మాత్రమే. **RWMutex = లైబ్రరీ పుస్తకం** – అనేకమంది ఏకకాలంలో చదవగలరు (RLock), కానీ ఎవరైనా edit చేస్తుంటే (Lock) మిగతా అందరూ wait.
>
> **WaitGroup = school trip లో teacher.** "5 మంది pichadu వెళ్ళారు (Add), ప్రతివాడు తిరిగొచ్చాక చెప్పు (Done), అందరూ వచ్చేవరకు bus బయలుదేరదు (Wait)."
>
> **Once = రాకెట్ ignition button** – ఎంతమంది నొక్కినా ఒక్కసారే మంట. **Pool = tiffin box reuse** – ప్రతిసారి కొత్తది కొనకుండా, వాడి కడిగి తిరిగి పెట్టడం (GC తగ్గింపు).

Gotchas (సాధారణ తప్పులు)

- **Unlock మర్చిపోవడం** → deadlock. `defer mu.Unlock()` ఎప్పుడూ.
- **Mutex copy** (value pass) → protection విఫలం. Pointer receiver, `go vet`.
- **WaitGroup: `Add` ని goroutine లోపల** → race (Wait ముందే return కావచ్చు). Goroutine ముందు Add.
- **RWMutex write-heavy లో నెమ్మది** – Mutex కంటే complex; profile.
- **sync.Map ని general map గా వాడటం** – తరచుగా RWMutex+map వేగం. Specific patterns కే.
- **Pool objects persistent అనుకోవడం** – ఎప్పుడైనా GC అవుతాయి; state assume చేయకు.
- **RLock లో upgrade to Lock** = deadlock (reentrant కాదు).

Key Points

- `Mutex` / `RWMutex` = shared state protection; **zero value ready**, **copy చేయకు**, **defer Unlock**.
- `RWMutex` = concurrent readers (read-heavy); write-heavy లో profile.
- `WaitGroup` = wait for goroutines; **Add before goroutine**.
- `Once` = exactly-once init.
- `Cond` = condition wait/signal (rare; channels ప్రాధాన్యం).
- `sync.Map` = special concurrent map (write-once/disjoint); else RWMutex+map.
- `sync.Pool` = object reuse → GC pressure↓; contents ephemeral.

Interview దృష్టి

Q: Mutex vs channel – ఎప్పుడు ఏది? A: Mutex – shared state ని protect చేయడం (counter, cache, map). సులభం, fast, low overhead. Channels – ownership transfer, orchestration, pipelines, signaling. Rob Pike: "Use whichever is most expressive." Simple shared state → mutex; data flow/coordination → channels.

Q: sync.Once ఎలా thread-safe గా ఒకేసారి guarantee ఇస్తుంది? A: Internally atomic flag + mutex. `Do` మొదట atomic గా "done" flag check; not done అయితే mutex తీసుకుని, double-check, function run, flag set. అనేక goroutines ఏకకాలంలో call చేసినా function ఒక్కసారే, మిగతావి పూర్తయ్యేవరకు wait.

Q: sync.Pool ఎందుకు, ఎప్పుడు? A: Frequently allocated temporary objects (buffers) ని reuse చేసి allocations + GC pressure తగ్గిస్తుంది. Get/Put. Contents ఎప్పుడైనా GC అవ్వగలవు కాబట్టి persistent cache కాదు, purely reuse. High-throughput hot paths లో (JSON encoders, byte buffers) విలువైనది.

---

24. sync/atomic

వివరణ

`sync/atomic` **= lock లేకుండా, hardware-level atomic operations.** ఒక int/pointer ని increment/swap/compare చేయడానికి mutex overhead అవసరం లేదు – CPU యొక్క atomic instructions (LOCK prefix, LL/SC) నేరుగా వాడతాయి. చాలా fast, కానీ single variable operations కే.

Atomic operations

```
import "sync/atomic"

var counter int64

// Go 1.19+ typed atomics (recommended — safer API)
var c atomic.Int64
c.Add(1)              // atomic increment
c.Load()              // atomic read
c.Store(100)          // atomic write
old := c.Swap(50)     // set & return old

// పాత function-based API (ఇంకా వాడుకలో)
atomic.AddInt64(&counter, 1)
val := atomic.LoadInt64(&counter)
atomic.StoreInt64(&counter, 100)
```

CAS – Compare-And-Swap (lock-free algorithms కి పునాది)

`CompareAndSwap(old, new)` – value ఇప్పుడు `old` అయితేనే `new` కి set చేసి `true` return; లేకపోతే ఏమీ చేయకుండా `false`. ఇదే lock-free data structures, spinlocks, optimistic concurrency కి foundation.

```
var value atomic.Int64
value.Store(10)

// CAS loop — lock-free update
for {
	old := value.Load()
	newVal := old * 2
	if value.CompareAndSwap(old, newVal) {
		break   // success — ఈ లోపు ఎవరూ మార్చలేదు
	}
	// మరొక goroutine మధ్యలో మార్చింది → retry
}
```

CAS ఎలా పని చేస్తుంది: "value ఇప్పటికీ 10 అయితే, 20 చేయి." మధ్యలో వేరే goroutine మారిస్తే CAS fail అవుతుంది → loop retry. Locks లేకుండా correctness.

atomic.Value — arbitrary value ని atomically swap

ఒక పెద్ద struct/config ని atomically replace చేయడానికి (read-heavy config reload లాంటివి).

```
var config atomic.Value   // holds any type consistently

config.Store(&Config{Timeout: 30})   // atomic write
cfg := config.Load().(*Config)       // atomic read

// hot reload — readers ఏ lock లేకుండా చదువుతారు
config.Store(&Config{Timeout: 60})   // కొత్త config atomically
```

`atomic.Pointer[T]` (1.19+) = type-safe version.

atomic vs mutex

అంశం	atomic	mutex
Scope	Single variable (int, pointer)	Multiple ops / complex critical section
Speed	చాలా fast (hardware instruction)	Slower (lock/unlock, contention)
Blocking	Non-blocking (lock-free)	Blocking (goroutines wait)
Use case	Counters, flags, single-value swap	Maps, multi-field structs, invariants
Complexity	Simple ops మాత్రమే	ఏ critical section అయినా

రూల్: ఒక int counter / flag → atomic. అనేక fields కలిపి consistent గా update చేయాలి, లేదా complex logic → mutex.

Real-life Scenario

> **Mutex = ఒక గదికి తాళం** – లోపలికి వెళ్ళి, ఎన్ని పనులైనా చేసి, బయటకొచ్చి తాళం తీయడం (multiple ops protected).
>
> **Atomic = టర్న్‌స్టైల్ (turnstile) counter** metro station దగ్గర. ఒక్కో వ్యక్తి దాటగానే counter +1 – ఇది hardware లోనే atomic, ఎవరూ wait చేయరు, తాళం లేదు. కానీ ఇది ఒకే counter కి మాత్రమే; రెండు వేర్వేరు counters ని "ఒకేసారి consistent గా" మార్చాలంటే turnstile సరిపోదు, గదికి తాళం (mutex) కావాలి.
>
> **CAS = "ధర ఇప్పటికీ ₹100 అయితేనే కొంటా."** వెళ్ళేసరికి ధర మారితే (ఎవరో కొన్నారు), deal cancel → మళ్ళీ కొత్త ధర చూసి retry.

Gotchas (సాధారణ తప్పులు)

- **atomic + non-atomic ఒకే variable మీద mix చేయడం** → race. అన్ని accesses atomic గా ఉండాలి.
- **32-bit platforms లో 64-bit atomic alignment** – పాత function API లో `int64` field struct లో మొదట ఉండాలి (8-byte align), లేకపోతే panic. Typed `atomic.Int64` (1.19+) దీన్ని handle చేస్తుంది – దాన్ని వాడు.
- **Multiple variables కి atomic** – ప్రతి variable atomic అయినా, కలిపి consistent కాదు (torn reads across vars). అలాంటిది mutex.
- **atomic.Value లో type consistency** – ఒకసారి `*Config` store చేస్తే, తర్వాత అదే type. వేరే type panic.
- **CAS loop starvation** – high contention లో retry loop CPU burn. Mutex కొన్నిసార్లు మంచిది.

Key Points

- `sync/atomic` = lock-free single-variable ops (hardware atomic instructions).
- **Typed atomics** (`atomic.Int64`, `atomic.Pointer[T]`, 1.19+) – safer, alignment handled.
- **CAS** (CompareAndSwap) = lock-free algorithms, optimistic concurrency foundation.
- **atomic.Value/Pointer** = atomically swap whole struct (config hot-reload).
- **atomic** = single var, fast, non-blocking; **mutex** = multi-op, complex sections.
- Mix atomic + plain access = **race**.

Interview దృష్టి

Q: atomic vs mutex – ఎప్పుడు atomic? A: Single variable మీద simple op (counter increment, flag set, pointer swap) అయితే atomic – hardware instruction, non-blocking, mutex కంటే fast. అనేక variables/fields ని కలిపి consistent గా update, లేదా complex critical section అయితే mutex. Atomic = narrow but fast; mutex = general.

Q: CAS అంటే? ఎందుకు ముఖ్యం? A: Compare-And-Swap – "value ఇప్పటికీ expected అయితేనే కొత్తది set చేయి, లేకపోతే fail." Atomic hardware op. Lock-free data structures, optimistic concurrency, spinlocks కి foundation. CAS fail అయితే retry (someone else changed) – locks లేకుండా correctness. ABA problem గమనించాలి.

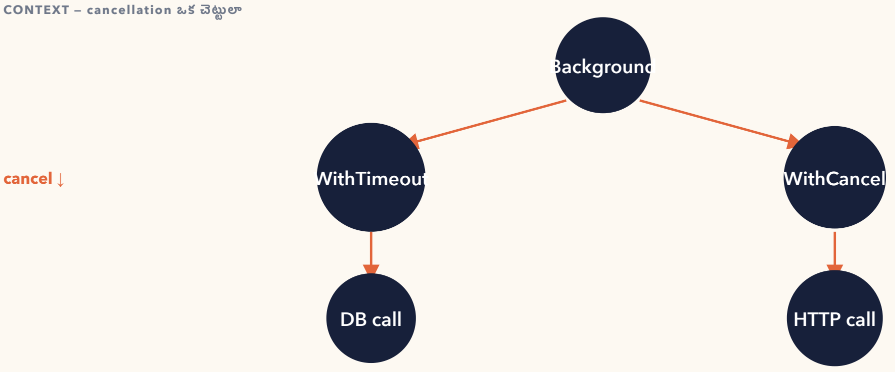
Q: 64-bit atomic alignment issue? A: 32-bit platforms లో పాత `atomic.AddInt64` కి argument 8-byte aligned ఉండాలి (struct లో మొదటి field). లేకపోతే runtime panic. Go 1.19+ typed atomics (`atomic.Int64`) ఈ alignment ని automatically guarantee చేస్తాయి – వాటిని వాడటం recommended.

---

25. context

CONTEXT · CANCELLATION చెట్టు

CONTEXT – cancellation ఒక చెట్టులా



నియమాలు

ctx ఎప్పుడూ **మొదటి parameter** – func F(ctx context.Context, ...)
Struct lo ఎప్పుడూ store చేయకూడదు · nil ctx పంపకూడదు (context.TODO() వాడాలి)
cancel() ని ఎప్పుడూ defer చేయాలి – లేకపోతే context leak.

Values ని జాగ్రత్తగా

context.WithValue ని request-scoped metadata కి మాత్రమే (trace id, user id).
దాన్ని optional parameters పంపడానికి వాడితే – type safety పోతుంది, code చదవలేం.

వివరణ

`context` = goroutines మధ్య cancellation, deadlines, request-scoped values ని propagate చేసే mechanism. Server లో ఒక request ని handle చేసేటప్పుడు అనేక goroutines (DB call, RPC, cache) spawn అవుతాయి – request cancel అయితే (client disconnect, timeout), అన్నిటినీ ఆపాలి. Context ఇదే చేస్తుంది. Standard: **HTTP handlers, DB calls, RPCs అన్నీ మొదటి argument గా `ctx context.Context` తీసుకుంటాయి.**

```
func fetchData(ctx context.Context, url string) error {
        req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)
        resp, err := http.DefaultClient.Do(req)  // ctx cancel అయితే request abort
        // ...
        return err
}
```

Context creation

```
ctx := context.Background()   // root context (main, init, tests) — never cancels
ctx := context.TODO()         // "ఇక్కడ context ఏది వాడాలో ఇంకా తెలియదు" placeholder
```

```
// derived contexts (cancellation తో)
ctx, cancel := context.WithCancel(parent)
ctx, cancel := context.WithTimeout(parent, 5*time.Second)
ctx, cancel := context.WithDeadline(parent, time.Now().Add(time.Minute))
ctx := context.WithValue(parent, key, value)
```

WithCancel – manual cancellation

```
func main() {
	ctx, cancel := context.WithCancel(context.Background())

	go worker(ctx)

	time.Sleep(2 * time.Second)
	cancel()   // worker కి "ఆగు" signal — ctx.Done() closes
	time.Sleep(time.Second)
}
func worker(ctx context.Context) {
	for {
		select {
		case <-ctx.Done():                    // cancel triggered
			fmt.Println("stopping:", ctx.Err()) // context.Canceled
			return
		default:
			// పని చేయి
			time.Sleep(500 * time.Millisecond)
		}
	}
}
```

`defer cancel()` **ఎప్పుడూ** – cancel call చేయకపోతే context resources (timer, goroutine) leak అవుతాయి. `go vet` warns.

WithTimeout / WithDeadline – time-bound

```
ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
defer cancel()   // తప్పనిసరి — timer cleanup

result, err := slowOperation(ctx)
if err != nil {
	if errors.Is(err, context.DeadlineExceeded) {
		fmt.Println("timed out!")
	}
}
```

`WithTimeout(parent, d)` = `WithDeadline(parent, now+d)`. Timeout దాటితే `ctx.Done()` closes, `ctx.Err()` = `context.DeadlineExceeded`.

WithValue – request-scoped data

Request ID, auth user వంటి request-scoped values ని propagate చేయడానికి. **అతిగా వాడకూడదు** – function parameters కి బదులు కాదు.

```
type ctxKey string
const userIDKey ctxKey = "userID"

ctx := context.WithValue(context.Background(), userIDKey, "user-42")

func handler(ctx context.Context) {
        if uid, ok := ctx.Value(userIDKey).(string); ok {
                fmt.Println("user:", uid)
        }
}
```

రూల్: WithValue ని request-scoped metadata (trace IDs, auth) కే; business logic parameters ని function args గా pass చేయి. Key custom type వాడాలి (string కాదు – collision తప్పించడానికి).

Propagation – context tree

Contexts **tree** గా ఏర్పడతాయి. Parent cancel అయితే, **అన్ని children cancel** అవుతాయి (cascade). ఒక request యొక్క మొత్తం goroutine tree ఒకేసారి ఆగుతుంది.

```
Background
   └── WithTimeout(5s)           ← request root
         ├── WithCancel          ← DB query
         └── WithValue(traceID)  ← RPC call
   // root timeout → అన్ని descendants cancel
```

Real-life Scenario

> **Context = ఒక పెద్ద mission లో command center నుండి radio.** Commander (request) అనేక teams (goroutines) ని పంపుతాడు – recon, supply, backup. Mission abort అయితే (client disconnect / timeout), commander radio లో ఒక్క "mission abort" చెప్తే, **అన్ని teams ఏకకాలంలో ఆగుతాయి** (ctx.Done cascade). ప్రతి team ఎప్పటికప్పుడు radio వింటూ (`<-ctx.Done()`) ఉంటుంది.
>
> **WithTimeout = "30 నిమిషాల్లో పూర్తి చేయకపోతే auto-abort." WithValue = mission ID ని ప్రతి team కి attach చేయడం** (tracking కి) – కానీ దీన్ని actual orders (function params) కి బదులు వాడకూడదు.

Gotchas (సాధారణ తప్పులు)

- `cancel()` **call చేయకపోవడం** → context leak (timer/goroutine). **ఎప్పుడూ** `defer cancel()` **.**
- **Context ని struct లో store చేయడం** → anti-pattern. **మొదటి function argument** గా pass చేయి.
- **WithValue ని function params కి బదులు వాడటం** → hidden dependencies, type-unsafe. Request-scoped metadata కే.
- **string key in WithValue** → collision risk. Custom unexported type key వాడు.
- `ctx.Done()` **ని select లో check చేయకపోవడం** → cancellation పని చేయదు (goroutine ఆగదు).
- **nil context pass చేయడం** → panic. `context.TODO()` వాడు placeholder గా.

Key Points

- `context` = cancellation + deadlines + request-scoped values propagation.
- **First arg** convention: `func(ctx context.Context, ...)`.
- `Background()` (root), `TODO()` (placeholder); derive with `WithCancel/Timeout/Deadline/Value`.
- **`defer cancel()` always** – leak తప్పించడం.
- `ctx.Done()` (closed channel) + `ctx.Err()` (`Canceled` / `DeadlineExceeded`).
- **Cascade** – parent cancel → all children cancel.
- `WithValue` = request metadata only; custom key type; not for params.

Interview దృష్టి

Q: context దేనికి? ఎందుకు అవసరం? A: Goroutine trees లో cancellation, deadlines, request-scoped values ని propagate చేయడానికి. Server request ఒక goroutine tree spawn చేస్తుంది; request cancel/timeout అయితే context ద్వారా అన్నీ ఏకకాలంలో ఆగుతాయి – goroutine leaks, wasted work తప్పించడం. HTTP/DB/RPC APIs అన్నీ ctx తీసుకుంటాయి.

Q: cancel() ఎందుకు తప్పనిసరి? A: `WithCancel/Timeout/Deadline` internal resources (timer, cancellation goroutine, parent లో child reference) allocate చేస్తాయి. `cancel()` call చేయకపోతే అవి leak అవుతాయి (context తనంతట తాను finish అయినా). `defer cancel()` idiomatic – double-call safe.

Q: WithValue ఎప్పుడు వాడకూడదు? A: Business logic parameters ని pass చేయడానికి కాదు (hidden, type-unsafe dependencies). కేవలం request-scoped cross-cutting metadata (trace/request ID, auth token) కి. Function కి అవసరమైన data explicit parameters గా pass చేయాలి. Key కి custom unexported type (collision safety).

---

26. Concurrency Patterns

వివరణ

Go concurrency primitives (goroutines, channels, select, sync) ని కలిపి కొన్ని **reusable patterns** ఏర్పడ్డాయి. వీటిని తెలుసుకుంటే real-world concurrent systems (pipelines, workers, rate limiters) సులభంగా build చేయవచ్చు. Interview లో ఒకటి రెండు implement చేయమని అడగవచ్చు.

1. Worker Pool – fixed goroutines, shared queue

అపరిమిత goroutines కాకుండా, fixed సంఖ్య workers ఒక jobs channel నుండి పని తీసుకుంటాయి. Resource bound.

```
func workerPool(jobs <-chan int, results chan<- int, wg *sync.WaitGroup) {
        defer wg.Done()
        for job := range jobs {          // channel closed అయ్యేవరకు
                results <- job * job          // process
```

```
        }
}

func main() {
        jobs := make(chan int, 100)
        results := make(chan int, 100)
        var wg sync.WaitGroup

        for w := 0; w < 3; w++ {          // 3 workers
                wg.Add(1)
                go workerPool(jobs, results, &wg)
        }
        for j := 1; j <= 9; j++ {         // jobs feed
                jobs <- j
        }
        close(jobs)                        // "ఇక jobs లేవు" — workers exit

        go func() { wg.Wait(); close(results) }()  // అన్ని workers అయ్యాక results close
        for r := range results {
                fmt.Println(r)
        }
}
```

2. Fan-out / Fan-in

Fan-out = ఒక source ని అనేక goroutines process చేయడం (parallelism). **Fan-in** = అనేక channels ని ఒక్క channel లోకి merge చేయడం.

```
// Fan-in — అనేక channels → ఒకటి
func fanIn(chans ...<-chan int) <-chan int {
        out := make(chan int)
        var wg sync.WaitGroup
        for _, ch := range chans {
                wg.Add(1)
                go func(c <-chan int) {
                        defer wg.Done()
                        for v := range c {
                                out <- v
                        }
                }(ch)
        }
        go func() { wg.Wait(); close(out) }()
        return out
}
```

3. Pipeline — stages connected by channels

ప్రతి stage ఒక goroutine, ఒక channel నుండి read చేసి, process చేసి, తర్వాతి channel కి write. Unix pipes లా.

```
func gen(nums ...int) <-chan int {         // stage 1: generate
        out := make(chan int)
        go func() {
                defer close(out)
                for _, n := range nums { out <- n }
```

```
        }()
        return out
}
func sq(in <-chan int) <-chan int {        // stage 2: square
        out := make(chan int)
        go func() {
                defer close(out)
                for n := range in { out <- n * n }
        }()
        return out
}
// వాడకం: gen → sq → print
for n := range sq(gen(2, 3, 4)) {
        fmt.Println(n)   // 4 9 16
}
```

4. Generator – lazy value stream

Function ఒక channel return చేసి, values ని on-demand produce చేస్తుంది (infinite sequences కూడా).

```
func fibonacci() <-chan int {
        out := make(chan int)
        go func() {
                a, b := 0, 1
                for {
                        out <- a
                        a, b = b, a+b   // ప్రతి receive కి తర్వాతి fib
                }
        }()
        return out
}
fib := fibonacci()
for i := 0; i < 5; i++ { fmt.Println(<-fib) } // 0 1 1 2 3
```

5. Semaphore – buffered channel తో concurrency limit

Buffered channel = "n permits" – max n goroutines ఏకకాలంలో.

```
sem := make(chan struct{}, 3)   // max 3 concurrent
for _, task := range tasks {
        sem <- struct{}{}            // acquire (buffer full → block)
        go func(t Task) {
                defer func() { <-sem }() // release
                process(t)
        }(task)
}
```

6. Rate limiting – time.Ticker

```
limiter := time.Tick(200 * time.Millisecond)  // 5 req/sec
for req := range requests {
        <-limiter                                   // ప్రతి request ముందు wait
```

```
        go handle(req)
}
```

7. errgroup – goroutines + error propagation

`golang.org/x/sync/errgroup` – WaitGroup + first error capture + context cancellation. Production లో అత్యంత useful.

```
import "golang.org/x/sync/errgroup"

func fetchAll(ctx context.Context, urls []string) error {
        g, ctx := errgroup.WithContext(ctx)
        for _, url := range urls {
                url := url                      // (pre-1.22 capture)
                g.Go(func() error {
                        return fetch(ctx, url)     // ఏదైనా error → ctx cancel, first
error return
                })
        }
        return g.Wait()                     // అన్నీ పూర్తి / మొదటి error
}
```

8. or-done & tee

or-done – messy cancellation ని clean గా wrap చేయడం. **tee** – ఒక channel ని రెండు channels గా split (broadcast).

```
// or-done: ctx cancel అయినా channel drain చేయకుండా clean exit
func orDone(ctx context.Context, in <-chan int) <-chan int {
        out := make(chan int)
        go func() {
                defer close(out)
                for {
                        select {
                        case <-ctx.Done():
                                return
                        case v, ok := <-in:
                                if !ok { return }
                                select {
                                case out <- v:
                                case <-ctx.Done():
                                        return
                                }
                        }
                }
        }()
        return out
}
```

Patterns table

Pattern	ఏం చేస్తుంది	ఎప్పుడు
Worker Pool	Fixed workers, shared job queue	Bounded parallelism, resource limits
Fan-out/Fan-in	Distribute + merge	Parallel processing + aggregate
Pipeline	Chained stages via channels	Stream data transformation
Generator	Lazy value stream	Infinite/on-demand sequences
Semaphore	Buffered channel permits	Concurrency cap
Rate limiting	Ticker	API throttling
errgroup	Goroutines + error + cancel	Parallel tasks with error handling
or-done/tee	Cancellation wrap / split	Clean shutdown, broadcast

Real-life Scenario

> **Worker Pool = ఒక bank లో 3 tellers (workers), ఒక queue (jobs channel).** Customers (jobs) queue లో నిలబడతారు; ఏ teller free అయితే తర్వాతి customer ని తీసుకుంటాడు. Tellers సంఖ్య fixed (bounded), infinite tellers hire చేయరు.
>
> **Pipeline = ఒక car assembly line.** Station 1 (chassis) → Station 2 (engine) → Station 3 (paint). ప్రతి station ఒక worker, conveyor belts (channels) కలుపుతాయి. **Semaphore = parking lot తో "3 slots only" board** – 3 cars ఉంటే, ఒకటి బయటకొచ్చేవరకు కొత్తది wait.

Gotchas (సాధారణ తప్పులు)

- **Channels close చేయకపోవడం** → workers/range forever block (leak). Producer close చేయాలి.
- **WaitGroup + close(results) ordering** – results ని అన్ని workers write చేసేముందు close చేయకూడదు; `go func(){ wg.Wait(); close(results) }()`.
- **errgroup లేకుండా manual error handling** – first error, cancellation miss అవుతాయి. errgroup వాడు.
- **Unbounded goroutines** (loop లో `go` without pool/semaphore) → resource exhaustion.
- **Pipeline లో ctx cancellation లేకపోవడం** → stages leak. or-done pattern.

Key Points

- **Worker pool** = bounded parallelism (fixed workers + job channel).
- **Fan-out/fan-in** = distribute + merge; **pipeline** = chained channel stages.
- **Generator** = lazy stream; **semaphore** (buffered chan) = concurrency cap.
- **Rate limit** = ticker; **errgroup** = parallel + error + context (production).
- ఎప్పుడూ **producer closes channel**, cancellation (context) wire చేయి.
- **Unbounded goroutines తప్పించు** – pool/semaphore.

Interview దృష్టి

Q: Worker pool ఎందుకు, ఎలా? A: ప్రతి job కి కొత్త goroutine spawn చేస్తే unbounded resource వాడకం (memory, DB connections). Worker pool = fixed goroutines ఒక shared jobs channel నుండి read చేస్తాయి – bounded concurrency, predictable resource use. Producer jobs channel close చేస్తే workers `range` ముగించి exit; WaitGroup తో completion.

Q: errgroup vs WaitGroup? A: WaitGroup కేవలం completion కోసం wait; errors, cancellation handle చేయదు. errgroup = WaitGroup + first-error capture + shared context cancellation (`errgroup.WithContext`) – ఒక goroutine fail అయితే context cancel అయి మిగతావి ఆగుతాయి, `Wait()` first error return. Parallel tasks with error handling కి production standard.

Q: Semaphore ని Go లో ఎలా చేస్తావు? A: Buffered channel `make(chan struct{}, n)` – n permits. Goroutine start ముందు `sem <- struct{}{}` (acquire; buffer full అయితే block), పూర్తయితే `<-sem` (release, defer). ఇది ఏకకాలంలో run అయ్యే goroutines ని n కి cap చేస్తుంది.

---

27. Go Memory Model

వివరణ

Go Memory Model = ఒక goroutine లో చేసిన memory writes ని మరో goroutine ఎప్పుడు, ఎలా చూడగలదో నిర్వచించే rules. కీలక concept: **happens-before.** Compiler + CPU instructions ని reorder చేస్తాయి (optimization) – synchronization లేకపోతే, ఒక goroutine లో write మరో goroutine కి ఏ order లో కనిపిస్తుందో guarantee లేదు.

happens-before relationship

Event A "happens-before" event B అంటే – A యొక్క effects (memory writes) B కి guaranteed visible. Synchronization primitives ఈ ordering ని establish చేస్తాయి:

- **ఒక goroutine లోపల** – program order (statements వరుసగా).
- **Channel send happens-before ఆ send యొక్క receive complete** – channel ద్వారా data pass చేస్తే, sender యొక్క అన్ని ముందటి writes receiver కి visible.
- **Channel receive from unbuffered happens-before send completes** (rendezvous).
- **Mutex Unlock happens-before తర్వాతి Lock** – critical section writes తర్వాతి lock holder కి visible.
- `sync.Once.Do` **completion happens-before ఏ ఇతర Do return.**
- **Goroutine creation** – `go f()` statement happens-before f execution. **Goroutine exit ఏదీ guarantee ఇవ్వదు** (join లేకుండా).

Data race – undefined behavior

Data race = రెండు goroutines ఒకే memory location ని ఏకకాలంలో access చేయడం, కనీసం ఒకటి write, synchronization లేకుండా. Go లో data race = **undefined behavior** (garbage values, crashes, torn reads).

```
// ❌ DATA RACE
var counter int
func main() {
        for i := 0; i < 1000; i++ {
                go func() { counter++ }()   // అనేక goroutines unsync write
        }
        // counter విలువ unpredictable — race
}
```

`counter++` atomic కాదు (read-modify-write మూడు steps) — రెండు goroutines interleave అయితే increments lost.

-race detector — bug hunting superpower

Go లో built-in **race detector** — `-race` flag తో compile/run చేస్తే, runtime లో data races detect చేసి report చేస్తుంది. Development, CI లో invaluable.

```
go run -race main.go
go test -race ./...
go build -race
```

```
==================
WARNING: DATA RACE
Write at 0x00c0000b4008 by goroutine 7:
  main.main.func1()
      main.go:8 +0x30
Previous write at 0x00c0000b4008 by goroutine 6:
  ...
==================
```

Race detector shadow memory వాడి ప్రతి memory access ని track చేస్తుంది — 5-10× slower, 5-10× more memory, కాబట్టి production లో కాదు, testing లో. **False negatives సాధ్యం** (race trigger అవ్వకపోతే detect కాదు) కానీ false positives లేవు (report అయితే నిజంగా race).

Fixes — synchronization

```
// Fix 1: mutex
var mu sync.Mutex
mu.Lock(); counter++; mu.Unlock()

// Fix 2: atomic
var counter atomic.Int64
counter.Add(1)

// Fix 3: channel (ownership)
ch := make(chan int)
// ఒక goroutine మాత్రమే counter own చేస్తుంది, మిగతావి channel ద్వారా
```

ఎందుకు channels/mutex ordering establish చేస్తాయి

Channel/mutex operations లోపల **memory barriers** (fences) ఉంటాయి – CPU/compiler ని reordering నుండి ఆపి, synchronization point ముందటి అన్ని writes ని flush చేసి, తర్వాతి goroutine కి visible చేస్తాయి. అందుకే mutex-protected write మరో goroutine కి (lock తీసుకున్నప్పుడు) సరిగ్గా కనిపిస్తుంది. Synchronization లేకపోతే ఈ barrier లేదు → stale/reordered reads.

Real-life Scenario

> **happens-before = "నేను ఉత్తరం పోస్ట్ చేశాకే నువ్వు చదవగలవు."** నేను ఉత్తరంలో "meeting 5కి" అని రాసి (write), post చేస్తే (channel send), నువ్వు అందుకుని చదివేసరికి (receive) ఖచ్చితంగా "5కి" కనిపిస్తుంది – post/receive ఒక ordering (happens-before) establish చేసింది.
>
> **Data race = ఇద్దరు ఒకే whiteboard మీద ఏకకాలంలో రాయడం** – అక్షరాలు కలిసిపోయి అర్థంలేని గజిబిజి. **Race detector = CCTV** – ఎవరు ఎప్పుడు board touch చేశారో record చేసి, "వీళ్ళిద్దరూ ఒకేసారి రాశారు!" అని పట్టిస్తుంది.

Gotchas (సాధారణ తప్పులు)

- **"నా code లో race లేదు, పని చేస్తోంది"** – race non-deterministic; ఇప్పుడు పని చేసినా production heavy load లో fail. `-race` **తో test చేయి.**
- `counter++` **atomic అనుకోవడం** – read-modify-write, atomic కాదు. Mutex/atomic కావాలి.
- **Goroutine exit మీద ఆధారపడటం** – join (WaitGroup/channel) లేకుండా goroutine writes visible అవుతాయని guarantee లేదు.
- **Reads కి కూడా sync అవసరం** – ఒకరు write చేస్తుంటే, reader కూడా lock/atomic వాడాలి (read-only అనుకుని skip చేయకు).
- **Race detector CI లో run చేయకపోవడం** – చాలా races missed.

Key Points

- Memory model = cross-goroutine memory visibility rules; **happens-before**.
- Channel send/receive, mutex lock/unlock, Once, goroutine start → happens-before edges.
- **Data race** = concurrent access, ≥1 write, no sync = **undefined behavior**.
- `-race` **detector** – runtime race detection (dev/CI, not prod; 5-10× overhead).
- Sync primitives = memory barriers → visibility + ordering.
- **Reads కూడా sync** అవసరం (concurrent write ఉంటే).

Interview దృష్టి

Q: Data race అంటే? Race condition తో తేడా? A: Data race = రెండు+ goroutines ఒకే memory ని concurrent access, ≥1 write, synchronization లేకుండా – memory-level, undefined behavior. Race condition = program correctness timing మీద ఆధారపడటం (broader, races లేకున్నా కావచ్చు – ఉదా. check-then-act). Data race ఒక race condition, కానీ అన్ని race conditions data races కావు.

Q: happens-before ఎందుకు ముఖ్యం? A: Compiler + CPU instructions reorder చేస్తాయి. Happens-before ఒక write మరో goroutine కి guaranteed visible అయ్యే ordering ను నిర్వచిస్తుంది. Channel ops, mutex, sync primitives ఈ edges create చేస్తాయి (memory barriers). Synchronization లేకపోతే write visibility guarantee లేదు → bugs.

Q: Race detector ఎలా పని చేస్తుంది? Limitations? A: `-race` shadow memory వాడి ప్రతి memory access + synchronization event ను track చేసి, happens-before violate అయిన concurrent accesses report చేస్తుంది. 5-10× CPU/memory overhead (prod కాదు). **Report అయితే నిజమైన race** (no false positives), కానీ race path execute అవ్వకపోతే miss (false negatives) – కాబట్టి concurrent tests, load తో run చేయాలి.

---

28. Common Concurrency Bugs

CONCURRENCY BUGS · నాలుగు క్లాసిక్ ఉచ్చులు

సాధారణ CONCURRENCY BUGS

Loop variable capture	`for _, v := range s { go f(v) }`	Go 1.22 కి ముందు – అందరూ చివరి v
WaitGroup.Add లోపల	`go func(){ wg.Add(1) … }`	Add ని ఎప్పుడూ goroutine బయట
Unbuffered channel deadlock	`ch <- 1 (receiver లేదు)`	main goroutine శాశ్వతంగా block
Mutex copy	`struct ని value గా పంపడం`	Mutex కూడా copy → రక్షణ పోయింది

ఒకే సాధనం ఇవన్నీ పట్టుకుంటుంది

`go test -race` – race detector. CI lo ఎప్పుడూ నడపాలి.ఇది runtime lo నిజంగా జరిగిన races ని పట్టుకుంటుంది; static analysis కాదు.

వివరణ

Concurrency లో అత్యంత తరచుగా వచ్చే bugs – వీటిని గుర్తుపట్టడం, తప్పించడం senior engineer signature. ఇవి interview favorites కూడా. ఒక్కొక్కటి cause + fix తో.

Bug 1: Deadlock – అందరూ wait, ఎవరూ progress

అన్ని goroutines ఒకదానికోసం ఒకటి wait చేస్తూ ఆగిపోవడం. Go runtime detect చేస్తే crash ("fatal error: all goroutines are asleep - deadlock!").

```
// Deadlock 1: unbuffered channel, no receiver
func main() {
        ch := make(chan int)
        ch <- 1   // ❌ receiver లేదు → forever block → deadlock
}

// Deadlock 2: lock ordering (classic)
```

```
// goroutine A: lock(mu1) then lock(mu2)
// goroutine B: lock(mu2) then lock(mu1)  ← reverse order → deadlock
```

Fix: Buffered channel లేదా separate goroutine receiver; locks **ఎప్పుడూ ఒకే order** లో acquire; `defer Unlock`.

Bug 2: Goroutine Leak – ఎప్పటికీ ముగియని goroutine

Blocked forever goroutine – memory/resource వృథా, silent (crash కాదు).

```
// ❌ LEAK: caller early return అయితే goroutine channel మీద forever block
func leak() int {
	ch := make(chan int)
	go func() { ch <- expensiveComputation() }()  // unbuffered
	return 0   // ch నుండి ఎవరూ receive చేయరు → goroutine leak forever
}
```

Fix: buffered channel (`make(chan int, 1)`), లేదా context cancellation, లేదా receive guarantee. `runtime.NumGoroutine()` / pprof goroutine profile తో detect.

Bug 3: Loop Variable Capture (pre-Go 1.22 vs post)

Go 1.21 & earlier లో loop variable అన్ని iterations share చేసేవి – closures/goroutines చివరి value చూసేవి.

```
// Go 1.21 & earlier – BUG:
for i := 0; i < 3; i++ {
	go func() { fmt.Println(i) }()   // అన్నీ 3 print (shared i)
}

// Fixes (pre-1.22):
for i := 0; i < 3; i++ {
	i := i                              // shadow – per-iteration copy
	go func() { fmt.Println(i) }()
}
// లేదా
for i := 0; i < 3; i++ {
	go func(i int) { fmt.Println(i) }(i)  // pass as arg
}

// Go 1.22+ : loop variable per-iteration – పైన fix అవసరం లేదు, 0 1 2
```

Go 1.22 లో ఈ behavior మారింది – loop variable ప్రతి iteration కి కొత్తది. కానీ interview లో "పాత Go లో ఏం జరుగుతుంది, ఎందుకు?" అడగవచ్చు.

Bug 4: Race Condition – unsynchronized shared state

```
// ❌ RACE: concurrent map write (runtime panic) లేదా counter race
var count int
for i := 0; i < 100; i++ {
```

```
        go func() { count++ }()   // data race
}
```

Fix: mutex, atomic, లేదా channel ownership. `-race` తో catch.

Bug 5: Closing a closed channel / send on closed → panic

```
// ❌ panic: send on closed channel
ch := make(chan int)
close(ch)
ch <- 1        // panic!

// ❌ panic: double close
close(ch)      // panic!
```

Fix: ఒక్క owner (sender) మాత్రమే close చేయాలి, ఒకసారి. అనేక senders ఉంటే, separate "done" signal లేదా `sync.Once` తో close guard.

```
var once sync.Once
closeCh := func() { once.Do(func() { close(ch) }) }  // safe multi-call
```

Bug 6: WaitGroup misuse

```
// ❌ Add inside goroutine — race, Wait might return early
for _, item := range items {
        go func() {
                wg.Add(1)   // ❌ too late — Wait already might have passed
                defer wg.Done()
        }()
}
// ✅ Add BEFORE goroutine
for _, item := range items {
        wg.Add(1)
        go func() { defer wg.Done() }()
}
```

Bug 7: Forgetting select default / ctx.Done → goroutine won't stop

```
// ❌ ctx cancel ని check చేయకపోతే goroutine ఆగదు (leak)
for {
        doWork()   // forever — cancellation లేదు
}
// ✅
for {
        select {
        case <-ctx.Done():
                return
        default:
                doWork()
```

```
        }
}
```

Bugs cheat table

Bug	Cause	Fix
Deadlock	Circular wait / no receiver	Buffered chan, consistent lock order, defer Unlock
Goroutine leak	Blocked forever, no cancel	context/done, buffered chan, pprof monitor
Loop var capture	Shared var (pre-1.22)	`v:=v` / pass arg / Go 1.22+
Race condition	Unsync shared state	mutex/atomic/channel, `-race`
Closed channel panic	Send/close on closed	Single owner closes once, sync.Once
WaitGroup misuse	Add inside goroutine	Add before goroutine
No cancellation	Missing ctx.Done check	select with `<-ctx.Done()`

Real-life Scenario

> **Deadlock = ఇరుకు వంతెనపై ఎదురెదురు రెండు కార్లు** – ఇద్దరూ "నువ్వు వెనక్కి వెళ్ళు" అని wait, ఎవరూ కదలరు. Fix: traffic rule (consistent order – ఎప్పుడూ ఎడమవైపు వాడు వెనక్కి).
>
> **Goroutine leak = ఆఫీస్‌లో ఒక ఉద్యోగి "మేనేజర్ చెప్పేదాకా wait చేయమన్నారు"** అని కూర్చున్నాడు, మేనేజర్ మర్చిపోయాడు – ఆ ఉద్యోగి జీవితాంతం wait, జీతం మాత్రం వృథా (memory).
>
> **Closed channel send = మూసేసిన postbox లో ఉత్తరం వేయడం** – postbox చిరిగిపోతుంది (panic).

Gotchas summary

- **Deadlocks compile-time లో కనిపించవు** – runtime. `-race`, careful lock ordering.
- **Leaks silent** – crash కాదు, memory పెరుగుతూ. pprof goroutine profile monitor.
- **"పని చేస్తోంది" ≠ correct** – races non-deterministic; load testing + `-race`.
- **Single channel owner** principle – sender creates, sends, closes.

Key Points

- **Deadlock** – circular/no-receiver wait; consistent lock order, buffered channels.
- **Goroutine leak** – blocked forever; cancellation (context/done), pprof.
- **Loop var capture** – pre-1.22 shared; `v:=v` or Go 1.22+.
- **Race condition** – unsync shared state; mutex/atomic/channel + `-race`.
- **Closed channel** – single owner closes once; `sync.Once` guard.
- **WaitGroup Add before goroutine**; **select** `<-ctx.Done()` for cancellation.

Interview దృష్టి

Q: ఈ code output? (pre-Go 1.22)

```
for i := 0; i < 3; i++ { go func(){ fmt.Print(i) }() }
```

A: పాత Go లో బహుశా `333` (అన్ని closures ఒకే shared `i` ని capture, goroutines run అయ్యేసరికి i=3). Go 1.22+ లో per-iteration variable – `012` (any order). Fix (పాత Go): `i:=i` shadow లేదా arg pass.

Q: Goroutine leak ఎలా detect చేస్తావు? A: `runtime.NumGoroutine()` monitor (steadily పెరిగితే leak), `net/http/pprof` goroutine profile (`/debug/pprof/goroutine`) – blocked goroutines stack traces చూపిస్తుంది. Prevention: ప్రతి goroutine కి clear termination path (context cancellation, channel close, timeout).

Q: అనేక goroutines ఒకే channel కి send చేస్తుంటే ఎవరు close చేయాలి? A: Senders ఎవరూ నేరుగా close చేయకూడదు (double close/send-on-closed panic). ఒక coordinator goroutine `sync.WaitGroup` తో అన్ని senders పూర్తయ్యేవరకు wait చేసి, తర్వాత close చేయాలి (`go func(){ wg.Wait(); close(ch) }()`). లేదా separate done channel.

Part 4 — Memory & Runtime Internals

> ఇక్కడ Go యొక్క "under the hood" — stack vs heap, escape analysis, garbage collector, memory allocator, runtime internals. SSE interviews లో ఇవి seniority ని వేరు చేసే topics. వీటిని అర్థం చేసుకుంటే, performance-critical Go రాయగలవు, allocations తగ్గించగలవు, GC tuning చేయగలవు.

---

29. Stack vs Heap + Escape Analysis

STACK VS HEAP · ESCAPE ANALYSIS

STACK vs HEAP — escape analysis నిర్ణయిస్తుంది

Stack

Function ముగిస్తే ఆటోమేటిక్ గా పోతుంది
GC పని లేదు — చాలా వేగం
Compiler "ఇది బయటికి పోదు" అని నిర్ధారిస్తే

Heap

GC నిర్వహిస్తుంది
Function ముగిసినా బతికే ఉండగలదు
Pointer బయటికి వెళ్తే తప్పనిసరి

ఎప్పుడు heap కి escape అవుతుంది

Pointer ని return చేస్తే · interface lo పెడితే · closure lo capture అయితే
Size compile time lo తెలియకపోతే · goroutine lo వాడితే

`go build -gcflags="-m"` — compiler ఏం నిర్ణయించిందో చూపిస్తుంది.**ఆచరణాత్మక సలహా**ముందు clear code రాయండి — తర్వాత profile చేసి, hot path lo మాత్రమే allocations తగ్గించండి."Pointer వాడితే వేగం" అనేది అపోహ — చిన్న structs కి copy చేయడమే చౌక.

వివరణ

ప్రతి program కి రెండు memory regions:

- **Stack** — per-goroutine, function calls కి. Fast (pointer bump), automatic cleanup (function return → freed). No GC involvement.
- **Heap** — shared, dynamically allocated (long-lived objects). **GC managed** — slower allocation, cleanup కి GC అవసరం.

Go లో మనం `new` / `&` వాడినంత మాత్రాన heap కి వెళ్తుందని కాదు. **Compiler నిర్ణయిస్తుంది** — ఇదే **escape analysis.** ఒక value function scope దాటి "escape" అవుతుందా? అవుతే heap, లేకపోతే stack. ఇది Go యొక్క ఒక key optimization — programmer manually decide చేయనవసరం లేదు (C లా), కానీ safe (C లా dangling కాదు).

Escape Analysis – compiler ఎలా నిర్ణయిస్తుంది

Compiler ఒక variable యొక్క lifetime ని analyze చేస్తుంది. Function return అయ్యాక కూడా reference మిగిలితే (escape), heap కి. Value function లోపలే die అయితే, stack.

```
go build -gcflags='-m' main.go   # escape analysis decisions చూపిస్తుంది
# "moved to heap: x"  → escaped
# "does not escape"   → stack
```

Common escape triggers

```
// 1. Local variable యొక్క pointer ని return → ESCAPES to heap
func newUser() *User {
        u := User{Name: "x"}   // "moved to heap: u"
        return &u              // pointer escapes function scope
}

// 2. Interface లో value పెట్టడం → తరచుగా escapes
func print(v any) { fmt.Println(v) }  // v boxed → heap
print(42)

// 3. Closure లో capture (by reference) → escapes
func counter() func() int {
        n := 0             // n escapes (closure captures)
        return func() int { n++; return n }
}

// 4. Slice/map కి pointer store, unknown size → heap
// 5. Value too large for stack → heap
// 6. Compiler analyze చేయలేని dynamic cases → conservatively heap
```

Stack-friendly (does NOT escape)

```
// value return — copy, no escape
func makeUser() User {
        u := User{Name: "x"}
        return u              // "does not escape" — copied to caller stack
}

// local computation, pointer function దాటదు
func sum(nums []int) int {
        total := 0            // stack — pure local
        for _, n := range nums { total += n }
        return total
}
```

ఎందుకు ముఖ్యం – performance

- **Stack allocation ~ free** (pointer increment), no GC.
- **Heap allocation** – allocator work + **GC pressure** (ప్రతి heap object GC scan చేయాలి).
- Hot loops లో అనవసర escapes → millions of heap allocs → GC overhead → latency spikes.

Optimization strategies:

- Value semantics (చిన్న small structs) – copy stack లో ఉంటుంది.
- పెద్ద unnecessary pointers తప్పించు.
- `sync.Pool` reuse (hot allocations).
- Slices ని pre-allocate (`make([]T, 0, n)`) – append escapes తగ్గించు.
- Benchmarks లో `-benchmem` allocations count చూడు; `pprof` heap profile.

```
// Benchmark లో allocations కనిపిస్తాయి
// go test -bench=. -benchmem
// BenchmarkX  1000000  120 ns/op  16 B/op  1 allocs/op
//                                          ↑ allocation escape సూచిక
```

Real-life Scenario

> **Stack = నీ desk మీద temporary sticky notes** – పని అయ్యాక చింపేస్తావు, cleanup automatic, ఖరీదు లేదు. **Heap = office central filing cabinet** – ఎక్కువ కాలం ఉండేవి; ఒక file నీకు అవసరం లేకపోయాక, ఒక janitor (GC) వచ్చి పరిశీలించి తీసేయాలి – time + effort.
>
> **Escape analysis = office assistant** ఒక పత్రం "నువ్వు వెళ్ళిపోయాక కూడా ఎవరైనా చూస్తారా?" అని ఆలోచించి – అవును అయితే filing cabinet (heap) లో, లేదంటే desk sticky note (stack). నువ్వు `&` రాసినా, అది నిజంగా escape అవ్వకపోతే stack లోనే ఉంచుతాడు.

Gotchas (సాధారణ తప్పులు)

- **`&` / `new` = heap అనుకోవడం** – తప్పు. Escape analysis నిర్ణయిస్తుంది; escape అవ్వకపోతే stack.
- **Value return = copy కాబట్టి slow అనుకోవడం** – small structs copy stack లో pointer కంటే fast (no GC).
- **Interface boxing hidden allocations** – `fmt.Println(intVal)` int ని heap కి box చేస్తుంది. Hot paths లో గమనించు.
- **పెద్ద struct value return** – copy ఖరీదు; balance escape vs copy cost.
- **Escape analysis అనవసరంగా trust చేయడం** – verify `-gcflags='-m'` , benchmarks తో.

Key Points

- **Stack** = fast, auto-cleanup, no GC (function-local). **Heap** = GC-managed, slower.
- **Escape analysis** (compiler) నిర్ణయిస్తుంది – `&` / `new` heap కాదని కాదు.
- Escapes: pointer-return, interface boxing, closures, large values.
- Heap allocs = **GC pressure** → latency. Stack allocs ~ free.
- Verify: `go build -gcflags='-m'` , `go test -benchmem` , pprof heap.
- Optimize: value semantics (small), pre-alloc slices, `sync.Pool` .

Interview దృష్టి

Q: Go లో value stack కి వెళ్తుందా heap కి – ఎవరు నిర్ణయిస్తారు? A: Compiler, **escape analysis** ద్వారా. Variable యొక్క reference function scope దాటి బతికితే (escapes) heap కి, లేకపోతే stack. `&` / `new` heap ని guarantee చేయవు – escape అవ్వకపోతే stack. `-gcflags='-m'` తో decisions చూడవచ్చు.

Q: Escape analysis ఎందుకు ముఖ్యం performance కి? A: Stack allocation ~ free (no GC), heap allocation GC pressure సృష్టిస్తుంది (scan అవసరం). Hot paths లో అనవసర escapes → millions heap allocs → GC overhead → latency. Escapes తగ్గిస్తే throughput పెరుగుతుంది, GC pauses తగ్గుతాయి.

Q: Local variable యొక్క address return చేయడం safe ఎందుకు (C లో కాదు)? A: C లో అది stack unwind అయ్యాక dangling pointer. Go escape analysis "ఈ variable escapes" అని గుర్తించి heap లో allocate చేస్తుంది; GC pointer ఉన్నంతవరకు ఉంచుతుంది. కాబట్టి safe + idiomatic.

---

30. Garbage Collector (GC)

వివరణ

Go automatic **garbage collection** ఇస్తుంది – heap లో ఇక reference లేని objects ని automatically free చేస్తుంది. Go GC = **concurrent, tri-color, mark-and-sweep collector** with a strong focus on **low latency** (short pauses), not maximum throughput. Design goal: **STW (stop-the-world) pauses < 1ms** typically, even large heaps తో.

Tri-color mark-and-sweep – ఎలా పని చేస్తుంది

Objects ని మూడు రంగుల్లో classify చేస్తుంది:

- **White** – ఇంకా visit కాలేదు (candidate for collection). ప్రారంభంలో అన్ని objects white.
- **Grey** – reachable అని తెలిసింది కానీ దాని children ఇంకా scan కాలేదు (queue లో).
- **Black** – reachable + children అన్నీ scan అయ్యాయి (safe, keep).

Algorithm:

1. **Roots** (stacks, globals) నుండి directly reachable objects ని grey చేయి.
2. ఒక grey object తీసుకో → దాని pointers ని follow చేసి referenced objects ని grey చేయి → ఈ object ని black చేయి.
3. Grey set ఖాళీ అయ్యేవరకు repeat.
4. చివర్లో **white objects = unreachable = garbage** → **sweep** (free).

Invariant: black object ఎప్పుడూ white object ని నేరుగా point చేయకూడదు (tri-color invariant). ఇది broken అయితే live object ని పొరపాటున collect చేయవచ్చు.

Write Barrier – concurrent marking ని safe చేయడం

GC **application తో concurrent గా** run అవుతుంది (mutator = నీ program). Marking జరుగుతున్నప్పుడు program pointers ని మారిస్తే? – ఒక black object కి కొత్త white object attach అయితే (tri-color invariant break), ఆ white object miss అయి collect అవుతుంది (bug!). దీన్ని ఆపడానికి **write barrier** – pointer write జరిగినప్పుడు runtime code inject అవుతుంది, కొత్తగా pointed object ని grey చేస్తుంది (Go Dijkstra-style + hybrid barrier వాడుతుంది). ఇది correctness ని guarantee చేస్తుంది concurrent marking లో.

GC phases & STW

```
1. Sweep Termination  — [STW, tiny] previous sweep finish
2. Mark Setup         — [STW, tiny] write barrier ON, roots scan prep
3. Marking            — [CONCURRENT] tri-color mark, program కూడా run
4. Mark Termination   — [STW, tiny] finish marking, write barrier OFF
5. Sweeping           — [CONCURRENT] free white objects (lazy, on-demand)
```

రెండు చిన్న STW pauses మాత్రమే (సాధారణంగా microseconds); heavy work (marking, sweeping) concurrent. అందుకే Go GC latency-friendly.

GC Pacing – ఎప్పుడు trigger అవుతుంది

GC ప్రతిసారి కాదు – **GOGC** (default 100) heap growth ఆధారంగా trigger చేస్తుంది. GOGC=100 అంటే: **heap ప్రస్తుత live size కంటే 100% (రెట్టింపు) పెరిగినప్పుడు GC.** పేసర్ (pacer) ఒక feedback loop – GC ఎప్పుడు మొదలుపెడితే ఆ target hit అవుతుందో అంచనా వేస్తుంది.

```
GOGC=100   # default — live heap 2× అయ్యేసరికి GC
GOGC=200   # తక్కువ తరచు GC (ఎక్కువ memory, తక్కువ CPU)
GOGC=50    # ఎక్కువ తరచు GC (తక్కువ memory, ఎక్కువ CPU)
GOGC=off   # GC disable (special cases — jobs)
```

GOMEMLIMIT (Go 1.19+) – soft memory limit

GOGC ఒక్కటే memory spikes ని ఆపదు. **GOMEMLIMIT** ఒక soft memory ceiling – heap ఆ limit దగ్గరకొస్తే GC ఎక్కువ aggressive గా run అవుతుంది (GOGC ఎక్కువున్నా). Containers (K8s pods) లో OOM kills తప్పించడానికి invaluable.

```
GOMEMLIMIT=512MiB   # heap ఈ limit ని దాటకుండా GC aggressive
# GOGC=off + GOMEMLIMIT=... → "limit దగ్గరే GC" pattern (memory-bound)
```

Tuning

```
import "runtime/debug"
debug.SetGCPercent(200)         // GOGC programmatically
debug.SetMemoryLimit(512<<20)   // GOMEMLIMIT programmatically
runtime.GC()                    // manual GC (అరుదు — testing)

// GC stats
var m runtime.MemStats
```

```
runtime.ReadMemStats(&m)
fmt.Println(m.NumGC, m.PauseTotalNs)
// GODEBUG=gctrace=1 — ప్రతి GC cycle log
```

Real-life Scenario

> **Tri-color = ఒక పెద్ద library లో "ఏ పుస్తకాలు వాడుకలో ఉన్నాయి" audit.** White = ఇంకా చూడని పుస్తకాలు (బహుశా పనికిరానివి). Grey = "ఇది వాడుకలో ఉంది, కానీ ఇది reference చేసే పుస్తకాలు ఇంకా చూడలేదు." Black = "పూర్తిగా verify అయింది, ఉంచాలి." Audit అయ్యాక ఇంకా white గా మిగిలినవి = ఎవరూ వాడనివి → తీసేయి (sweep).
>
> **Write barrier = audit జరుగుతున్నప్పుడు ఎవరైనా కొత్త పుస్తకం అరలో పెడితే**, security "ఆగు, ఈ కొత్త పుస్తకాన్ని కూడా audit list లో పెట్టు" (grey) అని ఖచ్చితం చేస్తుంది – లేకపోతే వాడుకలో ఉన్న పుస్తకం పొరపాటున పారేయబడుతుంది.
>
> **GOGC = "library ఎంత నిండాక audit చేయాలి" rule.** GOMEMLIMIT = "shelf capacity hard limit – దగ్గరకొస్తే వెంటనే audit."

GC tuning table

Setting	Effect	ఎప్పుడు
`GOGC=100` (default)	2× heap growth → GC	సాధారణ
`GOGC` పెంచడం	తక్కువ GC, ఎక్కువ memory	Memory ఉంది, CPU కావాలి
`GOGC` తగ్గించడం	ఎక్కువ GC, తక్కువ memory	Memory-constrained
`GOMEMLIMIT`	Soft ceiling, aggressive GC near limit	Containers (OOM తప్పించడం)
`GOGC=off` +GOMEMLIMIT	Limit-driven GC	Predictable memory-bound

Gotchas (సాధారణ తప్పులు)

- **`runtime.GC()` production లో అతిగా వాడటం** → unnecessary STW. GC ని runtime handle చేయనివ్వు.
- **GOMEMLIMIT లేకుండా containers** → GOGC memory spikes OOM kill (K8s). GOMEMLIMIT set చేయి.
- **GC ని disable (GOGC=off) చేసి memory leak** – short-lived batch jobs కే.
- **అధిక allocations = GC pressure** – root cause escape analysis (Topic 29), pooling.
- **"GC pause ఎక్కువ" అనుకుని GOGC tweak** – నిజ కారణం తరచుగా allocation rate; profile ముందు.

Key Points

- Go GC = **concurrent, tri-color mark-sweep**, low-latency (sub-ms STW).
- White (garbage candidate) / grey (reachable, unscanned) / black (scanned, keep).
- **Write barrier** – concurrent marking లో tri-color invariant preserve.

- Two tiny **STW** pauses (setup, mark termination); mark/sweep concurrent.
- **GOGC** (default 100) = heap growth trigger (2×); **GOMEMLIMIT** (1.19+) = soft ceiling.
- Tuning via env vars / `runtime/debug`; profile before tuning.

Interview దృష్టి

Q: Go GC ఎలా పని చేస్తుంది? A: Concurrent tri-color mark-and-sweep. Roots నుండి reachable objects ని grey→black గా mark చేస్తుంది (mostly application తో concurrent), unreachable (white) objects ని sweep చేస్తుంది. Correctness కి write barrier (concurrent pointer writes ని handle). Design goal = low latency – రెండు tiny STW pauses (setup, mark termination) మాత్రమే, మిగతా concurrent.

Q: Write barrier ఎందుకు అవసరం? A: GC application తో concurrent గా mark చేస్తుంది. Program ఒక black object కి కొత్త (white) object ని point చేస్తే tri-color invariant break అవుతుంది – ఆ live white object miss అయి collect అవుతుంది. Write barrier ప్రతి pointer write మీద runtime code inject చేసి, కొత్త object ని grey చేస్తుంది – correctness guarantee.

Q: GOGC vs GOMEMLIMIT? A: GOGC (default 100) = heap ప్రస్తుత live size కంటే X% పెరిగినప్పుడు GC trigger (100 = 2×). GOMEMLIMIT (1.19+) = soft memory ceiling – heap ఆ limit దగ్గరకొస్తే GC aggressive అవుతుంది, GOGC ఏమైనా. Containers లో OOM kills తప్పించడానికి GOMEMLIMIT set చేయాలి. రెండూ కలిపి memory-bound behavior.

---

31. Memory Allocator

వివరణ

Heap కి object allocate చేయడం ఖరీదైన operation కావచ్చు (contention, fragmentation). Go allocator దీన్ని fast, low-contention గా చేయడానికి **Google's TCMalloc (Thread-Caching Malloc)** heritage నుండి inspired **multi-tier caching allocator** వాడుతుంది. కీలక idea: **per-P (per-thread) caches → lock-free fast path.**

మూడు tiers: mcache → mcentral → mheap

```
┌─────────────────────────────────────────────────┐
│  mheap (global heap)                             │
│  - OS నుండి memory (arenas), pages manage        │
│  - largest allocations నేరుగా ఇక్కడి నుండి        │
└─────────────────────────────────────────────────┘
                  │ spans supply
┌─────────────────────────────────────────────────┐
│  mcentral (per size-class, global, lock protected) │
│  - ఒక్కో size class కి spans list                │
│  - mcache empty అయితే ఇక్కడి నుండి refill         │
└─────────────────────────────────────────────────┘
                  │ span assign
┌─────────────────────────────────────────────────┐
│  mcache (per-P, LOCK-FREE fast path!)            │
```

```
│  - ఒక్కో P కి own cache — no contention        │
│  - చిన్న allocations ఇక్కడి నుండి directly        │
└────────────────────────────────────────────────┘
```

- `mcache` – **per-P** (per logical processor). Lock-free! చిన్న allocations (< 32KB) ఇక్కడి నుండి – ఒక P కి own cache కాబట్టి contention లేదు (fast path). ఇది TCMalloc యొక్క thread-caching idea.
- `mcentral` – ఒక్కో **size class** కి global list of spans. mcache ఖాళీ అయితే, mcentral నుండి కొత్త span తీసుకుంటుంది (lock తీసుకుని – కానీ అరుదు).
- `mheap` – global heap; OS నుండి memory (arenas – 64MB chunks), pages manage చేస్తుంది. mcentral కి spans supply. **పెద్ద objects (> 32KB) నేరుగా mheap నుండి.**

Size classes – fragmentation తగ్గించడం

Go ~70 predefined **size classes** (8B, 16B, 24B, 32B, ... up to 32KB) వాడుతుంది. ఒక allocation ని దగ్గరి పెద్ద size class కి round up చేస్తుంది. ఉదా. 17-byte request → 24-byte class. ఇది:

- **Fragmentation తగ్గిస్తుంది** – same-size objects ఒకే span లో, reuse సులభం.
- **Allocation fast** – size class నిర్ణయించి, ఆ class యొక్క free list నుండి తీసుకోవడం O(1).
- చిన్న internal waste (17→24) కానీ overall efficient.

Spans – memory యొక్క unit

Span (`mspan`) = ఒకటి లేదా అంతకంటే ఎక్కువ contiguous **pages** (8KB each) యొక్క block, ఒక నిర్దిష్ట size class కి divide చేయబడింది. ఉదా. ఒక span 8KB (1 page) ని 512 × 16-byte slots గా divide చేయవచ్చు. Allocator span యొక్క free slots నుండి objects ఇస్తుంది. Span యొక్క అన్ని objects free అయితే, span ని mcentral/mheap కి తిరిగి ఇవ్వవచ్చు.

Allocation fast path

```
1. Object size → size class నిర్ణయించు
2. Current P యొక్క mcache లో ఆ class యొక్క span చూడు
3. Free slot ఉంటే → return (LOCK-FREE, few instructions) ✅ fast path
4. mcache span full → mcentral నుండి refill (lock, rare)
5. mcentral empty → mheap నుండి కొత్త span (rare)
6. mheap short → OS నుండి arena (rarest, syscall)
```

అత్యధిక allocations step 3 లోనే ముగుస్తాయి – lock-free, చాలా fast. అందుకే Go allocations చౌక (కానీ stack ఇంకా చౌక – Topic 29).

Tiny allocator

Very small objects (< 16 bytes, no pointers – ఉదా. small strings, single bytes) ని ఒకే 16-byte block లో pack చేస్తుంది (tiny allocator) – memory efficiency.

Real-life Scenario

> **mcache = ప్రతి chef దగ్గర own small ingredient tray** (per-P). ఏదైనా కావాలంటే tray నుండి వెంటనే తీసుకుంటాడు – వేరే chef తో పోటీ (lock) లేదు, super fast.
>
> **mcentral = kitchen యొక్క central pantry** (size class వారీగా అరలు). Chef tray ఖాళీ అయితే pantry కి వెళ్తాడు (queue/lock, కానీ అరుదు).
>
> **mheap = వెనుక warehouse** – OS (సప్లయర్) నుండి bulk stock. Pantry అయిపోతే ఇక్కడి నుండి.
>
> **Size classes = standard container sizes** (small/medium/large boxes). 17 items ఉంటే medium (24) box లో పెడతారు – ప్రతిదానికి custom box చేయకుండా, standard sizes తో fast + తక్కువ వృథా.

Allocator tiers table

Tier	Scope	Lock	ఏమిటి
`mcache`	Per-P	Lock-free	Fast path, small allocs (<32KB)
`mcentral`	Global, per size-class	Locked	mcache refill source
`mheap`	Global	Locked	OS memory, large allocs, arenas
Size classes	~70 (8B–32KB)	–	Round-up, fragmentation↓
Span	Pages block	–	Size-class-divided allocation unit

Gotchas (సాధారణ తప్పులు)

- **అనేక చిన్న allocations = mcache refills + GC pressure** – batch/pool చేయి.
- **32KB పైన objects నేరుగా mheap (locked, slow)** – పెద్ద buffers reuse (sync.Pool).
- **Size class rounding waste** – 33-byte struct → 48-byte class. Struct size ని class boundaries కి tune చేయవచ్చు (extreme optimization).
- **Fragmentation** – different-size long-lived objects. Pooling, arena patterns.
- Allocator internals ని micro-optimize చేయడం అరుదుగా worth – ముందు allocations తగ్గించు (escape analysis).

Key Points

- Go allocator = **TCMalloc-inspired**, multi-tier: **mcache (per-P, lock-free) → mcentral (per size-class) → mheap (global/OS)**.
- **~70 size classes** (8B–32KB) – round-up → fragmentation↓, O(1) alloc.
- **Span** = pages divided into size-class slots.
- Fast path = **lock-free mcache** (most allocs); refill rare.
- Large (>32KB) → mheap directly; tiny (<16B) → tiny allocator packs.
- Reduce allocs (stack/pool) beats micro-tuning the allocator.

Interview దృష్టి

Q: Go memory allocator ఎలా పని చేస్తుంది? ఎందుకు fast? A: TCMalloc-inspired multi-tier. **mcache** per-P (per logical processor) – small allocations lock-free fast path (contention లేదు). ఖాళీ అయితే **mcentral** (per size-class, global, locked) నుండి span refill; అది ఖాళీ అయితే **mheap** (OS arenas). Per-P caching వల్ల అత్యధిక allocations lock-free – అందుకే fast + scalable.

Q: Size classes ఎందుకు? A: ~70 predefined sizes (8B–32KB). Allocation ని దగ్గరి పెద్ద class కి round up – same-size objects ఒకే span లో pack అవుతాయి → external fragmentation తగ్గుతుంది, free-list allocation O(1). చిన్న internal waste (round-up) కానీ overall efficiency + speed.

Q: Span అంటే? A: `mspan` = contiguous pages (8KB each) block, ఒక size class కి slots గా divided. Allocator span యొక్క free slots నుండి objects ఇస్తుంది. Span యొక్క అన్ని objects free అయితే mcentral/mheap కి తిరిగి. Allocator memory management యొక్క unit.

---

32. Runtime Deep – defer/panic/recover Internals, Stack Growth, GODEBUG

వివరణ

Go runtime అనేది program తో పాటు compile అయ్యే code – goroutines, GC, allocator, scheduler, defer/panic mechanics అన్నీ ఇక్కడ. కొన్ని deep mechanics ని చూద్దాం – ఇవి "ఎలా magic జరుగుతుంది" అనే curiosity ని satisfy చేస్తాయి, performance debugging కి సహాయపడతాయి.

defer internals – evolution

`defer` implementation Go versions లో గణనీయంగా optimize అయింది:

- **Old (pre-1.13):** ప్రతి `defer` heap-allocated `_defer` struct, goroutine యొక్క linked list కి push. Function return లో list ని walk చేసి execute. **Slow** (~50ns per defer, allocation).
- **Go 1.13:** stack-allocated `_defer` (heap కాదు) – allocation తగ్గింది.
- **Go 1.14: Open-coded defers** – compiler simple cases (fixed defers, no loops) లో defer calls ని నేరుగా function epilogue లో **inline** చేస్తుంది. ఒక bitmask ఏ defers run చేయాలో track చేస్తుంది. **దాదాపు zero overhead** – normal function call అంత fast. Loops లోని defers ఇంకా slower path.

అందుకే modern Go లో `defer` ని free గా వాడవచ్చు (hot loops లో తప్ప – అక్కడ open-coding apply అవ్వదు).

panic/recover internals

- **panic** – runtime `_panic` struct create చేసి goroutine కి link చేస్తుంది, తర్వాత **deferred functions ని unwind order (LIFO)** లో execute చేస్తూ stack ని పైకి walk చేస్తుంది.
- **recover** – deferred function లోపల call అయితే, current goroutine యొక్క `_panic` ని "recovered" గా mark చేస్తుంది. Unwinding ఆగి, recovering deferred function నుండి normal గా return అవుతుంది (ఆ function యొక్క caller కి control).

- recover deferred function లో **నేరుగా** call అవ్వాలి – nested helper లో పని చేయదు.
- recovered కాకపోతే – stack పూర్తిగా unwind అయి, program crash (goroutine stack trace print).

Stack growth – contiguous copying stacks

Goroutine 2KB stack తో మొదలవుతుంది. Function prologue లో compiler ఒక **stack check** inject చేస్తుంది – "ఈ function కి కావలసిన space ఉందా?" లేకపోతే `morestack` trigger:

1. కొత్త, **రెట్టింపు పెద్ద** stack allocate చేస్తుంది.
2. పాత stack యొక్క contents ని కొత్తదానికి **copy** చేస్తుంది.
3. **Pointers ని adjust చేస్తుంది** – stack లోని values కి point చేసే pointers ని కొత్త addresses కి update (GC + runtime దీన్ని precisely track చేస్తుంది).
4. పాత stack free.

ఇది **contiguous copying stacks** approach (పాత segmented stacks కాదు – "hot split" problem తప్పించడానికి 1.3 లో మారారు). Stack shrink కూడా అవుతుంది (GC సమయంలో, usage తక్కువ అయితే). Max stack size default 1GB (`debug.SetMaxStack`).

అందుకే deep recursion crash కాదు (limit వరకు), మరియు goroutine 2KB తో మొదలైనా అవసరమైతే grow అవుతుంది.

GODEBUG – runtime introspection knobs

`GODEBUG` environment variable = runtime behavior ని observe/tweak చేసే knobs. Debugging, performance analysis కి invaluable.

```
GODEBUG=gctrace=1 ./app       # ప్రతి GC cycle stats (pause, heap size)
GODEBUG=schedtrace=1000 ./app # ప్రతి 1000ms scheduler state (Ps, Gs, queues)
GODEBUG=scheddetail=1 ...      # detailed per-P/M/G state
GODEBUG=inittrace=1 ./app      # package init timing
GODEBUG=allocfreetrace=1 ...   # ప్రతి alloc/free (చాలా verbose)
GODEBUG=madvdontneed=1 ...     # memory ని OS కి eager return
```

`gctrace=1` output ఉదాహరణ:

```
gc 1 @0.012s 2%: 0.018+1.2+0.003 ms clock, ... 4->5->2 MB, 5 MB goal
        |         |     └ mark/sweep timings    └ heap: before->after->live
        └ GC number, elapsed time, % CPU on GC
```

runtime package – introspection

```
runtime.NumGoroutine()       // live goroutines
runtime.NumCPU()             // cores
runtime.GOMAXPROCS(0)        // current P count (0 = query)
runtime.Gosched()            // yield current goroutine (scheduler point)
runtime.GC()                 // force GC
runtime.ReadMemStats(&m)     // detailed memory stats
runtime.Stack(buf, all)      // goroutine stack traces (debugging)
```

Real-life Scenario

> **Stack growth = ఒక చిన్న notebook తో మొదలుపెట్టడం.** పేజీలు నిండితే, ఒక పెద్ద notebook తీసుకుని, పాత notebook అంతా కొత్తదానికి copy చేసి (stack copy), అన్ని bookmark references ని update చేసి (pointer adjust), పాతది పారేస్తావు. నువ్వు ఎప్పుడూ "notebook నిండిపోతుందా?" అని బెంగపడనవసరం లేదు – runtime automatic గా పెంచుతుంది.
>
> **GODEBUG = car dashboard యొక్క hidden diagnostic mode.** సాధారణంగా కనిపించదు, కానీ ఒక secret code (env var) enter చేస్తే engine RPM, fuel flow, GC cycles అన్నీ live గా చూపిస్తుంది – mechanic (నీవు) debugging కి.

Gotchas (సాధారణ తప్పులు)

- **defer in tight hot loop** – open-coding apply అవ్వదు (loop defers slower path). Manual cleanup కొన్నిసార్లు fast.
- **recover ని deferred function బయట / nested helper లో** call చేయడం → పని చేయదు (nil). Deferred function లో నేరుగా.
- **Deep recursion 1GB దాటితే** – stack overflow crash. Iterative గా rewrite.
- **GODEBUG production లో verbose knobs** (allocfreetrace) → huge overhead. Targeted debugging కే.
- **Stack copy వల్ల interior pointers move** – `unsafe` తో stack addresses hold చేస్తే invalid అవుతాయి.

Key Points

- **defer** – pre-1.13 heap linked-list → 1.14 **open-coded** (near-zero cost simple cases).
- **panic** unwinds LIFO deferred calls; **recover** (deferred function లోనే) stops unwinding.
- **Stack growth** = contiguous **copying** (2× + copy + pointer adjust); shrinks too; max ~1GB.
- Deep recursion safe up to limit; 2KB start grows on demand.
- **GODEBUG** knobs – gctrace, schedtrace, inittrace (observe runtime).
- `runtime` package – NumGoroutine, Gosched, MemStats introspection.

Interview దృష్టి

Q: Goroutine stack ఎలా grow అవుతుంది? A: 2KB తో మొదలు. Function prologue లో stack-space check; చాలకపోతే `morestack` – రెట్టింపు పెద్ద stack allocate, పాత contents copy, stack-లోకి point చేసే pointers adjust (runtime/GC precisely track), పాతది free. Contiguous copying stacks (segmented కాదు). Shrink కూడా అవుతుంది. అందుకే lakhs of goroutines cheap గా మొదలై, అవసరమైతే grow.

Q: defer overhead ఇప్పుడు ఎందుకు తక్కువ? A: Go 1.14 open-coded defers – compiler simple defer cases ని function epilogue లో inline చేస్తుంది (bitmask తో ఏవి run చేయాలో track), heap-allocated `_defer` list అవసరం లేదు. దాదాపు normal call అంత fast. Loops లోని defers ఇంకా slower path.

Q: GODEBUG ఎప్పుడు వాడతావు? A: Runtime behavior observe చేయడానికి – `gctrace=1` (GC pauses/heap), `schedtrace` (scheduler contention), `inittrace` (slow package init). Production issue diagnose చేసేటప్పుడు (GC latency, goroutine buildup) targeted గా enable చేసి, తర్వాత తీసేస్తాం (కొన్ని verbose knobs overhead).

Part 5 – Standard Library & Tooling

> Go యొక్క "batteries included" తత్వం – గొప్ప standard library + first-class tooling. ఇక్కడ modules deep, essential stdlib packages, testing (Go యొక్క బలం), tooling (pprof, vet, lint), reflection, unsafe. Production Go engineer కి ఇవి రోజువారీ tools.

---

33. Packages & Modules Deep

వివరణ

Topic 2 లో modules basics చూశాం. ఇక్కడ deeper – semantic import versioning, go.sum internals, vendoring, workspaces, replace/exclude, internal packages, init() order. ఇవి large codebases, monorepos, dependency management లో కీలకం.

Semantic Import Versioning (SIV)

Go modules **semantic versioning** (semver: MAJOR.MINOR.PATCH) follow అవుతాయి. కీలక rule: **major version 2+ import path లోనే ఉండాలి.**

```
import "github.com/foo/bar"      // v0/v1
import "github.com/foo/bar/v2"   // v2 — path లో /v2!
import "github.com/foo/bar/v3"   // v3
```

ఎందుకు? v2 breaking changes కలిగి ఉంటుంది; v1, v2 ఒకే program లో coexist చేయగలవు (వేర్వేరు import paths). ఇది "diamond dependency" hell ని తప్పిస్తుంది – రెండు deps వేర్వేరు major versions కావాలంటే, రెండూ import అవుతాయి.

Minimal Version Selection (MVS)

Go dependency resolution **MVS** వాడుతుంది – npm/maven కంటే వేరు. ప్రతి dependency కి, అవసరమైన **అత్యల్ప version** ఎంచుకుంటుంది (అన్ని requirements satisfy చేసే minimum). "Latest" కాదు – reproducibility కి. Upgrade explicit (`go get -u`).

go.sum – verification internals

`go.sum` ప్రతి module version కి **రెండు hashes:** (1) module content యొక్క hash, (2) దాని go.mod యొక్క hash. Download అయిన module ఈ hashes తో verify అవుతుంది – mismatch అయితే build fail (tampered/corrupted). Go **checksum database** (sum.golang.org) తో cross-verify (public transparency log).

go.work – workspaces (Go 1.18+)

అనేక local modules ని ఒకేసారి develop చేయడానికి (monorepo, module + దాని dependency ని simultaneously edit). `go.work` file multiple modules ని link చేస్తుంది.

```
go work init ./service ./shared   # go.work create
go work use ./newmodule           # module add
```

```
// go.work
go 1.22
use (
	./service
	./shared   // service ఇక్కడ నుండి shared ని local గా వాడుతుంది (published కాదు)
)
```

`go.work` ని సాధారణంగా commit చేయరు (local dev tool). Published `replace` కంటే cleaner.

replace & exclude

```
// go.mod
require github.com/foo/bar v1.2.0

// replace — local fork / patched version / local path
replace github.com/foo/bar => ../my-fork-of-bar
replace github.com/foo/bar v1.2.0 => github.com/me/bar v1.2.1-patch

// exclude — నిర్దిష్ట buggy version ని skip
exclude github.com/foo/bar v1.1.0
```

`replace` – buggy dependency ని local fix, లేదా private fork కి point చేయడానికి. `go.work` రాకముందు local multi-module dev కి వాడేవారు.

Vendoring

`go mod vendor` – అన్ని dependencies ని project లోని `vendor/` folder కి copy చేస్తుంది. Build ఆ vendored copies వాడుతుంది (network అవసరం లేదు). Air-gapped builds, dependency snapshot, supply-chain audit కి.

```
go mod vendor        # vendor/ create
go build -mod=vendor # (vendor/ ఉంటే auto-detect Go 1.14+)
```

internal packages – compiler-enforced privacy

`internal/` directory లోని packages ని **ఆ internal యొక్క parent directory subtree మాత్రమే** import చేయగలదు – బయటి modules చేయలేవు. Encapsulation ని compiler enforce చేస్తుంది (capitalization కంటే బలమైన boundary).

```
myapp/
├── internal/
│   └── auth/       # myapp/... మాత్రమే import చేయగలదు
└── pkg/
    └── api/        # ఎవరైనా import చేయగలరు
```

init() function & order

`init()` = package initialize అయ్యేటప్పుడు automatically call అయ్యే function. Arguments/return లేవు. Package-level state setup కి.

```
var config Config
func init() {          // main కంటే ముందు auto-run
        config = loadConfig()
}
// ఒక file లో అనేక init() ఉండవచ్చు; అన్నీ run అవుతాయి
```

Order (guaranteed):

1. Imported packages ముందు initialize (depth-first, dependency order).
2. ఒక package లో: package-level variables (dependency order) → `init()` functions (file order, declaration order).
3. అన్నీ అయ్యాక `main.main()`.

Real-life Scenario

> **Semantic import versioning = ఒక పుస్తకం యొక్క 1st edition, 2nd edition ని వేర్వేరు shelf codes తో ఉంచడం.** "Physics v2" breaking changes కలిగి ఉంటుంది; ఒకరు v1 reference కావాలంటే, మరొకరు v2 – రెండూ library లో వేర్వేరు codes తో coexist. Confusion లేదు.
>
> **go.work = ఒక architect తన project + అతను design చేస్తున్న furniture ని ఒకేసారి edit చేయడం** – furniture ని shop కి పంపకుండా (publish), local గా try చేసి adjust. **internal/ = "staff only" గది** – building లోని staff (parent module) మాత్రమే, బయటి visitors కాదు.

Gotchas (సాధారణ తప్పులు)

- **v2+ import path లో /vN మర్చిపోవడం** → wrong version / build error.
- `replace` **ని production go.mod లో commit చేయడం** → consumers కి break. Local dev కి go.work.
- `init()` **overuse** – hidden side effects, testing కష్టం. Explicit initialization ప్రాధాన్యం.
- **init() order మీద fragile dependency** – cross-package init ordering assume చేయకు.
- **go.sum ని ignore/delete చేయడం** → verification loss (security).
- **Blank import** `_ "pkg"` – init() side effects కోసం (drivers) – ఉద్దేశపూర్వకం, కానీ గుర్తుంచుకో.

Key Points

- **Semantic import versioning** – major v2+ import path లో (`/v2`); versions coexist.
- **MVS** – minimal version selection (reproducible), not latest.
- **go.sum** – content + go.mod hashes; checksum DB verify (security).
- **go.work** (1.18+) – multi-module local dev; `replace` / `exclude` – fork/skip.
- **Vendoring** – `vendor/` snapshot (air-gapped, audit).
- **internal/** – compiler-enforced privacy (subtree only).
- **init()** – auto pre-main; order: imports → vars → init → main.

Interview దృష్టి

Q: v2 major version ఎందుకు import path లో ఉంటుంది? A: Breaking changes కలిగిన major versions ని ఒకే build లో coexist చేయడానికి. `github.com/foo/bar/v2` – v1, v2 వేర్వేరు import paths కాబట్టి రెండు dependencies వేర్వేరు major versions కావాలంటే రెండూ import అవుతాయి, diamond dependency conflict లేదు.

Q: internal/ package ఎలా పని చేస్తుంది? A: `internal/` directory లోని packages ని దాని immediate parent directory subtree లోని code మాత్రమే import చేయగలదు; బయటి modules compile error పొందుతాయి. Capitalization (unexported) కంటే బలమైన, compiler-enforced module boundary – public API surface ని control చేయడానికి.

Q: init() functions order ఏమిటి? A: మొదట imported packages initialize (dependency/depth-first order), ప్రతి package లో package-level variables → init() functions (file & declaration order), అన్నీ అయ్యాక main(). ఒక package కి అనేక init() ఉండవచ్చు. Cross-package fine-grained ordering మీద ఆధారపడకూడదు.

---

34. Essential stdlib Tour

వివరణ

Go's "batteries included" – standard library చాలా powerful, third-party dependencies అవసరం తరచుగా ఉండదు. ఒక senior Go engineer ఈ core packages ని బాగా తెలిసి ఉండాలి. వేగంగా tour చేద్దాం.

fmt – formatting & printing

```
fmt.Println("hello", 42)              // hello 42\n
fmt.Printf("%d %s %v %+v %#v\n",       // format verbs
        42, "hi", user, user, user)
s := fmt.Sprintf("id=%d", 42)         // string return
fmt.Errorf("failed: %w", err)         // wrapped error

// Key verbs:
```

```
// %v (default), %+v (fields+names), %#v (Go syntax), %T (type)
// %d (int), %s (string), %f (float), %q (quoted), %p (pointer), %x (hex)
```

io & bufio – streaming I/O

`io.Reader` / `io.Writer` = Go I/O యొక్క universal interfaces (అన్ని I/O ఇవే). `bufio` buffering జోడిస్తుంది (syscalls తగ్గించడం).

```
// io.Copy – Reader నుండి Writer కి stream (memory-efficient)
io.Copy(dst, src)   // మొత్తం memory లోకి load చేయకుండా

// bufio.Scanner – line-by-line reading (large files)
scanner := bufio.NewScanner(file)
for scanner.Scan() {
        line := scanner.Text()
        // process line – మొత్తం file memory లో లేదు
}

// bufio.Writer – batched writes
w := bufio.NewWriter(file)
defer w.Flush()   // buffer ని flush చేయడం మర్చిపోకు!
w.WriteString("data")
```

os – operating system

```
os.Args                          // command-line args ([0] = program)
os.Getenv("PATH")                // env variable
os.Setenv("KEY", "val")
file, err := os.Open("x.txt")   // read-only
file, err := os.Create("x.txt")// create/truncate
data, err := os.ReadFile("x")   // మొత్తం file → []byte (చిన్న files)
os.WriteFile("x", data, 0644)
os.Exit(1)                       // (defers run అవ్వవు!)
```

time – time & duration

```
now := time.Now()
d := 5 * time.Second           // Duration
time.Sleep(d)
later := now.Add(time.Hour)
elapsed := time.Since(start)   // Duration since start

// Formatting – reference time: Mon Jan 2 15:04:05 MST 2006!
now.Format("2006-01-02 15:04:05")  // ఈ నిర్దిష్ట numbers layout
t, _ := time.Parse("2006-01-02", "2026-07-15")

// Timers & Tickers
timer := time.NewTimer(2 * time.Second)
ticker := time.NewTicker(time.Second)  // periodic
defer ticker.Stop()                     // leak తప్పించు
```

Go's time formatting reference: `Mon Jan 2 15:04:05 MST 2006` (= 01/02 03:04:05PM '06 -0700) — mnemonic 1234567.

sort & slices/maps (Go 1.21+)

```
sort.Ints([]int{3, 1, 2})
sort.Strings(names)
sort.Slice(people, func(i, j int) bool {  // custom
        return people[i].Age < people[j].Age
})

// slices package (1.21+) — generic, ergonomic
slices.Sort(nums)
slices.Contains(nums, 5)
idx, found := slices.BinarySearch(sortedNums, 7)
slices.Max(nums); slices.Min(nums)
slices.Reverse(nums)
slices.Index(nums, 3)

// maps package (1.21+)
keys := maps.Keys(m)     // iterator (1.23+) / slice
maps.Clone(m)
```

encoding/json — marshal/unmarshal

```
type User struct {
        ID    int    `json:"id"`
        Name  string `json:"name"`
        Email string `json:"email,omitempty"`  // empty → skip
        pass  string `json:"-"`                // never serialize
}

// Marshal — struct → JSON
data, err := json.Marshal(user)           // {"id":1,"name":"x"}
pretty, _ := json.MarshalIndent(user, "", "  ")

// Unmarshal — JSON → struct
var u User
err = json.Unmarshal(data, &u)            // pointer అవసరం!

// Streaming (large data — memory-efficient)
decoder := json.NewDecoder(reader)
decoder.Decode(&u)
encoder := json.NewEncoder(writer)
encoder.Encode(u)
```

Gotcha: unexported fields serialize అవ్వవు; unmarshal target pointer అవ్వాలి; unknown JSON fields default గా ignore అవుతాయి.

regexp, container/list, container/heap

```
re := regexp.MustCompile(`\d+`)          // compile once (expensive)
re.FindString("abc123")                  // "123"
```

```
re.MatchString("a1b2")              // true
re.ReplaceAllString("a1b2", "X")      // "aXbX"

// container/heap — priority queue (interface implement చేయాలి)
// container/list — doubly linked list (అరుదు; slices ఎక్కువ)
```

stdlib table

Package	పని
`fmt`	Formatting, printing (verbs)
`io` / `bufio`	Streaming I/O, Reader/Writer, buffering
`os`	Files, args, env, process
`time`	Time, Duration, timers, tickers
`sort` / `slices` / `maps`	Sorting, generic slice/map ops (1.21+)
`encoding/json`	JSON marshal/unmarshal, streaming
`strings` / `strconv`	String ops, conversions
`regexp`	Regular expressions
`context`	Cancellation, deadlines
`sync` / `atomic`	Concurrency primitives
`net/http`	HTTP client/server
`errors`	Error wrapping, Is/As

Real-life Scenario

> **io.Reader/Writer = universal USB port.** File అయినా, network అయినా, memory buffer అయినా, compression stream అయినా — అన్నీ ఒకే "port" (interface) కి plug అవుతాయి. `io.Copy(dst, src)` ఏ source నుండి ఏ destination కి అయినా data pump చేస్తుంది — cable ఒకటే. అందుకే Go లో gzip → file → network chain trivial.
>
> **bufio = నీళ్ళు bucket తో మోయడం vs pipe.** ప్రతి glass కి బావి దగ్గరకి వెళ్ళడం (unbuffered, ప్రతి byte కి syscall) నెమ్మది; bucket నింపుకుని ఒకేసారి తేవడం (buffered) fast.

Gotchas (సాధారణ తప్పులు)

- **bufio.Writer `Flush()` మర్చిపోవడం** → buffered data write అవ్వదు (lost). `defer w.Flush()`.
- **json.Unmarshal కి non-pointer** → పని చేయదు (silently). `&target`.
- **Unexported fields JSON లో serialize అవ్వవు** — exported (capital) fields + tags.
- **time.Format layout** — arbitrary format కాదు; reference time `2006-01-02 15:04:05` numbers.

- **regexp.Compile ని hot loop లో** → recompile ఖరీదు. `MustCompile` once (package level).
- **os.Exit defers run చేయదు** – cleanup skip.
- **os.ReadFile పెద్ద files కి** → మొత్తం memory. Streaming (bufio) వాడు.

Key Points

- `io.Reader` / `io.Writer` = universal I/O interfaces; `io.Copy`, `bufio` (buffering).
- `fmt` verbs: `%v/%+v/%#v/%T/%w`.
- `time` – reference layout `2006-01-02 15:04:05`; Stop timers/tickers.
- `slices` / `maps` (1.21+) – generic ergonomic ops.
- `encoding/json` – tags (`omitempty`, `-`), pointer for Unmarshal, streaming Decoder/Encoder.
- `regexp.MustCompile` once; `bufio.Writer.Flush()` always.

Interview దృష్టి

Q: io.Reader/io.Writer ఎందుకు అంత powerful? A: ఒక్క method interfaces (`Read` / `Write`) – files, network, buffers, compression, crypto అన్నీ implement చేస్తాయి. దీనివల్ల composable I/O: `io.Copy`, pipelines (gzip.NewWriter wrapping file), decorators అన్నీ ఒకే abstraction మీద పని చేస్తాయి. Small interface + wide implementation = Go design excellence.

Q: JSON marshaling లో common gotchas? A: (1) Unexported (lowercase) fields serialize అవ్వవు. (2) Unmarshal target pointer అవ్వాలి. (3) `omitempty` empty values skip, `-` ఎప్పటికీ serialize కాదు. (4) Struct tags key names control చేస్తాయి. (5) Unknown JSON fields default ignore. (6) పెద్ద data కి streaming Decoder/Encoder (memory).

Q: time.Format ఎందుకు వింతగా ఉంటుంది? A: Go reference time `Mon Jan 2 15:04:05 MST 2006` (mnemonic 1 2 3 4 5 6 7) వాడుతుంది – నువ్వు కావలసిన format ని ఈ నిర్దిష్ట numbers తో రాస్తావు (ఉదా. `2006-01-02`). strftime `%Y-%m-%d` కాదు. మొదట గందరగోళం, కానీ నేర్చుకుంటే readable.

---

35. Testing

వివరణ

Testing Go యొక్క **first-class citizen** – `testing` package + `go test` built-in, external framework అవసరం లేదు. Convention: `*_test.go` files, `TestXxx(t *testing.T)` functions. Go testing philosophy: simple, table-driven, fast.

Basic test

```
// file: math.go
func Add(a, b int) int { return a + b }

// file: math_test.go
package math
```

```
import "testing"

func TestAdd(t *testing.T) {
	got := Add(2, 3)
	want := 5
	if got != want {
		t.Errorf("Add(2,3) = %d; want %d", got, want)
	}
}
```

```
go test ./...          # అన్ని tests
go test -v ./...       # verbose
go test -run TestAdd   # నిర్దిష్ట test
```

`t.Error/Errorf` – fail కానీ continue; `t.Fatal/Fatalf` – fail + ఆగిపో; `t.Helper()` – helper function mark.

Table-driven tests – Go idiom

అనేక cases ని ఒక slice of structs తో test చేయడం – Go లో అత్యంత common, idiomatic pattern.

```
func TestAdd(t *testing.T) {
	tests := []struct {
		name string
		a, b int
		want int
	}{
		{"positives", 2, 3, 5},
		{"negatives", -1, -1, -2},
		{"zero", 0, 0, 0},
		{"mixed", -5, 5, 0},
	}
	for _, tt := range tests {
		t.Run(tt.name, func(t *testing.T) {  // subtest
			if got := Add(tt.a, tt.b); got != tt.want {
				t.Errorf("Add(%d,%d) = %d; want %d", tt.a, tt.b, got,
tt.want)
			}
		})
	}
}
```

`t.Run(name, fn)` = **subtest** – ప్రతి case విడిగా report, `go test -run TestAdd/zero` తో individual run.

Benchmarks

Performance measure చేయడానికి. `BenchmarkXxx(b *testing.B)`, loop `b.N` (runtime chooses).

```
func BenchmarkAdd(b *testing.B) {
	for i := 0; i < b.N; i++ {
```

```
                Add(2, 3)
        }
}
```

```
go test -bench=. -benchmem   # benchmarks + allocation stats
# BenchmarkAdd-8  1000000000  0.30 ns/op  0 B/op  0 allocs/op
```

`-benchmem` = per-op bytes + allocations (escape analysis insight). **benchstat** tool multiple runs compare చేసి statistically significant differences చూపిస్తుంది.

Fuzzing (Go 1.18+)

Random/mutated inputs automatically generate చేసి edge cases, crashes కనుగొంటుంది. `FuzzXxx(f *testing.F)`.

```
func FuzzReverse(f *testing.F) {
        f.Add("hello")                          // seed corpus
        f.Fuzz(func(t *testing.T, s string) {   // random strings
                rev := Reverse(s)
                if Reverse(rev) != s {              // property: reverse twice =
original
                        t.Errorf("double reverse mismatch: %q", s)
                }
        })
}
```

```
go test -fuzz=FuzzReverse   # continuous fuzzing until crash/stop
```

Examples – testable documentation

`ExampleXxx` functions – documentation + tests (output comment verify అవుతుంది). godoc లో కనిపిస్తాయి.

```
func ExampleAdd() {
        fmt.Println(Add(2, 3))
        // Output: 5    ← ఈ output verify అవుతుంది
}
```

Coverage

```
go test -cover ./...                    # coverage %
go test -coverprofile=cov.out ./...
go tool cover -html=cov.out             # visual HTML report
```

TestMain – setup/teardown

Package tests కి global setup/teardown (DB connect, temp dirs).

```
func TestMain(m *testing.M) {
	setup()                  // DB, fixtures
	code := m.Run()          // అన్ని tests run
	teardown()               // cleanup
	os.Exit(code)
}
```

Mocking via interfaces

Go లో mocking frameworks అవసరం తక్కువ – **interfaces** ద్వారా dependencies inject చేసి, tests లో fake implementations.

```
type Store interface { Get(id int) (User, error) }

type mockStore struct{ users map[int]User }
func (m *mockStore) Get(id int) (User, error) {
	u, ok := m.users[id]
	if !ok { return User{}, ErrNotFound }
	return u, nil
}
// test: real DB కి బదులు mockStore inject
svc := NewService(&mockStore{users: map[int]User{1: {Name: "x"}}})
```

testify – popular assertion library

`stretchr/testify` – assertions (`assert`, `require`), mocks (`mock`). Boilerplate తగ్గిస్తుంది (కానీ pure stdlib purists దీన్ని avoid చేస్తారు).

```
import "github.com/stretchr/testify/assert"
assert.Equal(t, 5, Add(2, 3))
assert.NoError(t, err)
require.NotNil(t, result)   // require = fail + stop
```

Testing table

Tool	పని
`TestXxx` + table-driven	Unit tests (idiomatic)
`t.Run` subtests	Isolated cases, selective run
`BenchmarkXxx` + `-benchmem`	Performance + allocations
`FuzzXxx` (1.18+)	Auto edge-case/crash discovery
`ExampleXxx`	Testable docs
`-cover` / `-coverprofile`	Coverage
`TestMain`	Setup/teardown

Tool	పని
Interfaces	Mocking (DI)

Real-life Scenario

> **Table-driven tests = ఒక recipe ని అనేక ingredients తో test చేయడం** – same code, వేర్వేరు inputs/outputs ఒక list లో. కొత్త case add చేయడం = list కి ఒక row.
>
> **Fuzzing = ఒక కోతికి keyboard ఇచ్చి, app crash అవుతుందేమో చూడటం** – నువ్వు ఊహించని weird inputs (empty, huge, unicode, null bytes) automatically try చేసి, hidden bugs పట్టిస్తుంది. **Mocking via interface = crash test dummy** – నిజమైన DB కి బదులు, predictable fake తో test.

Gotchas (సాధారణ తప్పులు)

- **Table test loop var capture** (pre-1.22) – `tt` ని closure/parallel లో capture. `tt := tt` (Go 1.22+ fixed).
- `t.Parallel()` **shared state** – parallel subtests race. Isolate state.
- **Benchmark లో compiler optimizing away** – result ని sink variable కి assign (dead-code elimination తప్పించు).
- **Time-dependent tests flaky** – `time.Sleep` కి బదులు synchronization; inject clock.
- **Over-mocking** – implementation ని test చేస్తావు behavior కాదు. Real deps ఎక్కడ సాధ్యమో.
- **TestMain లో os.Exit మర్చిపోవడం** – teardown skip.

Key Points

- Built-in `testing` + `go test`; `*_test.go`, `TestXxx(t *testing.T)`.
- **Table-driven + `t.Run` subtests** = idiomatic pattern.
- **Benchmarks** `-benchmem` (allocations); **benchstat** compare.
- **Fuzzing** (1.18+) – auto edge-case discovery; **Examples** = testable docs.
- **Coverage** (`-cover`, HTML); **TestMain** = setup/teardown.
- **Mocking via interfaces** (DI) – frameworks అవసరం తక్కువ; testify optional.

Interview దృష్టి

Q: Go లో mocking ఎలా, frameworks లేకుండా? A: Interfaces + dependency injection. Production dependency ని interface గా define చేసి, function/struct ఆ interface accept చేస్తుంది. Test లో ఆ interface యొక్క fake implementation (predictable) inject చేస్తాం. Go small interfaces + implicit satisfaction దీన్ని natural చేస్తాయి – mockgen/testify optional.

Q: Table-driven tests ఎందుకు idiomatic? A: అనేక cases ని ఒక slice of structs (input, expected, name) గా, ఒక loop + `t.Run` subtest తో test చేయడం. కొత్త case = ఒక row (DRY), ప్రతి case isolated report, selective run (`-run Test/case`). Readable, maintainable – Go community standard.

Q: Fuzzing vs table tests? A: Table tests = నువ్వు ఊహించిన cases (deterministic). Fuzzing (1.18+) = seed corpus నుండి random/mutated inputs auto-generate చేసి, నువ్వు ఊహించని edge cases (empty, huge, malformed) తో crashes/property violations కనుగొంటుంది. Parsers, encoders, input handling కి invaluable. రెండూ complementary.

---

36. Tooling

వివరణ

Go's tooling ecosystem = దాని biggest strengths లో ఒకటి. Formatting, linting, profiling, tracing, race detection – అన్నీ first-class. ఒక senior Go engineer ఈ tools ని daily వాడతాడు, ముఖ్యంగా performance debugging కి pprof/trace.

gofmt / goimports – canonical formatting

`gofmt` = ఒకటే official formatting style – **no arguments, no config.** Style wars లేవు. `goimports` = gofmt + imports auto-add/remove/sort.

```
gofmt -w .        # అన్ని files format (in-place)
goimports -w .    # + imports manage
```

అన్ని Go code ఒకేలా కనిపిస్తుంది – reviews లో style discussions లేవు, tooling handle చేస్తుంది.

go vet – static analysis

Compile అయ్యే కానీ suspicious code ని catch చేస్తుంది – printf format mismatches, unreachable code, struct tag errors, lock copies, loop var issues.

```
go vet ./...
# ఉదా: "Printf format %d has arg of wrong type string"
```

golangci-lint – meta-linter

అనేక linters (staticcheck, gosimple, ineffassign, errcheck, gosec ...) ని ఒక్క fast tool గా aggregate. Production standard.

```
golangci-lint run
# staticcheck: unused code, gosec: security, errcheck: ignored errors, ...
```

pprof – profiling (SSE must-know)

`runtime/pprof` + `net/http/pprof` – CPU, memory (heap), goroutine, block, mutex profiling. Performance bottlenecks కనుగొనడానికి essential.

```
import _ "net/http/pprof"   // /debug/pprof/ endpoints register (blank import)
go func() { http.ListenAndServe("localhost:6060", nil) }()
```

```
# CPU profile — 30 సెకన్లు sample
go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30
# Heap (memory) profile
go tool pprof http://localhost:6060/debug/pprof/heap
# Goroutine profile (leaks!)
go tool pprof http://localhost:6060/debug/pprof/goroutine

# pprof interactive:
(pprof) top       # hottest functions
(pprof) list Foo  # line-by-line
(pprof) web       # visual graph (svg)

# benchmark profiling
go test -bench=. -cpuprofile=cpu.out -memprofile=mem.out
go tool pprof cpu.out
```

go tool trace — execution tracer

pprof "ఎక్కడ time spent" చెప్తే, trace "ఎప్పుడు, ఎలా goroutines scheduled, blocked, GC ran" timeline చూపిస్తుంది. Concurrency issues, latency spikes, scheduler behavior కి.

```
import "runtime/trace"
f, _ := os.Create("trace.out")
trace.Start(f)
defer trace.Stop()
// ... workload
```

```
go tool trace trace.out   # browser timeline: goroutines, GC, syscalls, scheduler
```

race detector

```
go test -race ./...       # tests తో races
go build -race && ./app   # runtime race detection (dev/staging)
```

(Topic 27 లో deep — 5-10× overhead, dev/CI only.)

build tags / constraints — conditional compilation

Platform/feature-specific code. File పైన `//go:build` line.

```
//go:build linux && amd64
package main
// ఈ file Linux amd64 లోనే compile
```

```
//go:build integration
// integration tag ఉంటేనే: go test -tags=integration
```

go generate – code generation

`//go:generate` comments → `go generate` run చేసినప్పుడు execute (mocks, stringers, protobuf).

```
//go:generate stringer -type=Weekday
//go:generate mockgen -source=store.go -destination=mock_store.go
```

```
go generate ./...   # అన్ని //go:generate directives run
```

Tooling table

Tool	పని
`gofmt` / `goimports`	Canonical formatting + imports
`go vet`	Static analysis (suspicious code)
`golangci-lint`	Meta-linter (staticcheck, gosec, ...)
`pprof`	CPU/heap/goroutine/mutex profiling
`go tool trace`	Execution timeline (scheduler, GC)
`-race`	Data race detection
build tags	Conditional compilation
`go generate`	Code generation

Real-life Scenario

> **pprof = hospital లో full-body scan.** "నొప్పి ఎక్కడ?" – scan (CPU profile) చూపిస్తుంది ఏ function 80% time తింటోందో (hotspot). Heap profile = "ఏ organ నీళ్ళు నిలుపుకుంటోంది" (memory bloat). Goroutine profile = "ఎన్ని goroutines బతికున్నాయి, ఎక్కడ stuck" (leaks).
>
> **go tool trace = flight recorder (black box).** ప్రతి goroutine ఎప్పుడు run అయింది, ఎప్పుడు blocked, GC ఎప్పుడు వచ్చింది – millisecond-level timeline. "ఎందుకు latency spike?" కి answer.
>
> **gofmt = ఒకటే official uniform** – ఎవరి style ఏదైనా, తుది code ఒకేలా. Reviews లో "spaces vs tabs" గొడవలు లేవు.

Gotchas (సాధారణ తప్పులు)

- **net/http/pprof ని public port లో expose** → security risk (memory/goroutine dumps). localhost/authenticated only.
- **pprof profile duration తక్కువ** → misleading (rare hotspots miss). Representative load + duration.
- **`-race` production లో** → 5-10× overhead. Dev/CI/staging.
- **build tag syntax** – `//go:build` (1.17+); blank line తర్వాత package. పాత `// +build` deprecated.
- **go generate ని CI లో run + commit మర్చిపోవడం** → stale generated code. Verify in CI.
- **Profiling ముందు micro-optimize** – "measure, don't guess." pprof చూపించని చోట optimize చేయకు.

Key Points

- `gofmt` / `goimports` – one canonical style (no config, no debates).
- `go vet` (suspicious code), `golangci-lint` (meta-linter, production).
- **pprof** – CPU/heap/goroutine/mutex profiling (`net/http/pprof` , `top` / `list` / `web`).
- **go tool trace** – scheduler/GC/goroutine timeline (concurrency/latency).
- `-race` – data races (dev/CI).
- build tags (`//go:build`) – conditional compilation; `go generate` – codegen.

Interview దృష్టి

Q: Production లో performance issue ఎలా debug చేస్తావు? A: `net/http/pprof` enable చేసి – CPU profile (hottest functions, `top` / `list`), heap profile (memory bloat, allocations), goroutine profile (leaks, blocked goroutines). Latency/scheduler issues కి `go tool trace` (timeline). "Measure first" – pprof చూపించిన hotspot ని మాత్రమే optimize. Allocations కి `-benchmem` + escape analysis.

Q: pprof vs trace తేడా? A: pprof = **statistical sampling** – "aggregate గా ఎక్కడ time/memory spent" (which functions). trace = **event timeline** – "ప్రతి goroutine ఎప్పుడు scheduled/blocked, GC ఎప్పుడు, syscalls" (temporal, concurrency behavior). Hotspot → pprof; latency spike / scheduler contention / GC impact → trace.

Q: gofmt ఎందుకు configurable కాదు? A: ఉద్దేశపూర్వకం. ఒకటే style → style debates లేవు, ఏ Go code అయినా familiar, tooling (diff, refactor) consistent. "Gofmt's style is no one's favorite, yet gofmt is everyone's favorite" – uniformity యొక్క విలువ personal preference కంటే ఎక్కువ.

---

37. Reflection

వివరణ

Reflection = runtime లో types, values ని inspect & manipulate చేయడం – compile time లో తెలియని types తో పని చేయడం. `reflect` package ద్వారా. JSON encoding, ORMs, dependency injection, validation libraries ఇది వాడతాయి. కానీ: **slow, complex, type-unsafe** – చివరి ఉపాయం మాత్రమే.

Rob Pike's law: **"Clear is better than clever. Reflection is never clear."**

reflect.Type & reflect.Value

రెండు core concepts: **Type** (type information) మరియు **Value** (actual value + manipulation).

```
import "reflect"

x := 42
t := reflect.TypeOf(x)    // int (reflect.Type)
v := reflect.ValueOf(x)   // 42 (reflect.Value)

fmt.Println(t.Kind())     // int (underlying kind)
fmt.Println(v.Int())      // 42 (typed extract)

// struct inspection
type User struct {
	Name string `json:"name" validate:"required"`
	Age  int    `json:"age"`
}
u := User{Name: "Surya", Age: 30}
tp := reflect.TypeOf(u)
for i := 0; i < tp.NumField(); i++ {
	field := tp.Field(i)
	fmt.Printf("%s: %s (tag: %s)\n",
		field.Name, field.Type, field.Tag.Get("json"))
}
// Name: string (tag: name)
// Age: int (tag: age)
```

Struct tags via reflection

Struct tags (Topic 11) ని reflection చదువుతుంది – ఇదే JSON/DB/validation libraries ఎలా tags ని interpret చేస్తాయో.

```
field.Tag.Get("json")      // "name"
field.Tag.Get("validate")  // "required"
// validation library: ప్రతి field తీసి, validate tag చూసి rule apply
```

Modifying values – settability

Reflection తో values modify చేయాలంటే, value **addressable + settable** అవ్వాలి – pointer తో ValueOf.

```
x := 42
v := reflect.ValueOf(&x).Elem()   // pointer → Elem() = addressable
if v.CanSet() {
        v.SetInt(100)
}
fmt.Println(x)   // 100

// reflect.ValueOf(x) (pointer కాదు) → CanSet() false (copy)
```

The three laws of reflection

1. **Reflection interface value నుండి reflection object కి వెళ్తుంది** (`reflect.TypeOf/ValueOf`).
2. **Reflection object నుండి interface value కి తిరిగి** (`v.Interface()`).
3. **Value ని modify చేయాలంటే, అది settable అవ్వాలి** (addressable – pointer's Elem).

Cost – ఎందుకు avoid చేయాలి

- **Slow** – direct code కంటే 10-100× (type checks runtime లో, no compiler optimizations, allocations).
- **Type-unsafe** – errors runtime లో (panic), compile time లో కాదు.
- **Unreadable** – complex, error-prone.
- **No escape analysis benefits** – తరచుగా heap allocations.

ఎప్పుడు వాడాలి: truly generic infrastructure (serialization, ORM) ఎక్కడ compile-time types తెలియవు, మరియు **generics సరిపోవు.** ఇప్పుడు generics (Topic 17) చాలా reflection use cases ని replace చేస్తున్నాయి – type-safe + fast.

Real-life Scenario

> **Reflection = ఒక universal translator ఏ భాష అయినా runtime లో అర్థం చేసుకోవడం.** ముందుగా భాష తెలియకపోయినా (compile-time type unknown), person మాట్లాడుతున్నప్పుడే (runtime) analyze చేసి translate చేస్తుంది. కానీ ఇది నెమ్మది (ప్రతి పదం analyze), తప్పులు జరగవచ్చు (misinterpret = runtime panic), మరియు complex. తెలిసిన భాష (concrete type / generics) అయితే direct మాట్లాడటమే fast + safe.
>
> **JSON library = ఈ translator ని వాడి** ఏ struct అయినా (ముందు తెలియకపోయినా) fields, tags చదివి JSON కి translate చేస్తుంది – అందుకే `json.Marshal(anyStruct)` పని చేస్తుంది.

Gotchas (సాధారణ తప్పులు)

- **Non-pointer value modify చేయడం** → `CanSet() false`, panic. `reflect.ValueOf(&x).Elem()`.
- **Wrong Kind method** → `v.Int()` on string → panic. Kind check ముందు.
- **Performance-critical paths లో reflection** → bottleneck. Cache reflect.Type, లేదా codegen/generics.
- **Reflection overuse** – simple problem కి. Interfaces/generics ముందు try చేయి.

- **Unexported fields** – reflection చదవగలదు కానీ `Set` చేయలేదు (panic). Exported fields కే settable.
- **v.Interface() on unexported field** → panic.

Key Points

- Reflection = runtime type/value inspection & manipulation (`reflect`).
- `reflect.TypeOf` (type), `reflect.ValueOf` (value); `Kind()` = underlying category.
- **Struct tags** via reflection → JSON/ORM/validation libraries.
- Modify: value **settable** అవ్వాలి (pointer's `Elem()` , `CanSet()`).
- **Slow (10-100×), type-unsafe (runtime panics), complex** – last resort.
- **Generics** ఇప్పుడు చాలా reflection use cases ని replace చేస్తున్నాయి (type-safe + fast).

Interview దృష్టి

Q: Reflection ఎప్పుడు వాడాలి, ఎప్పుడు avoid? A: Truly generic infrastructure – serialization (JSON), ORMs, DI, validation – ఎక్కడ compile-time types తెలియవు మరియు generics సరిపోవు. Avoid: performance-critical paths (10-100× slow), simple problems (interfaces/generics చాలు), type safety అవసరమైన చోట (runtime panics). "Reflection is never clear" – చివరి ఉపాయం.

Q: JSON marshaling reflection ఎలా వాడుతుంది? A: `json.Marshal(v)` reflect.TypeOf/ValueOf తో struct fields ని iterate చేసి, ప్రతి field యొక్క `json` tag చదివి (key name, omitempty), value ని extract చేసి JSON build చేస్తుంది. Compile-time లో struct type తెలియదు కాబట్టి reflection తప్పనిసరి (generics పూర్తిగా replace చేయలేవు – dynamic field enumeration).

Q: Reflection ఎందుకు slow? A: Type checks/dispatch runtime లో (compile-time optimizations లేవు), values ని interface{} గా box చేయడం (allocations, heap escapes), method/field lookups dynamic. Direct typed code కంటే 10-100×. reflect.Type ని cache చేయడం, లేదా codegen/generics తో reflection తప్పించడం helps.

---

38. unsafe

వివరణ

`unsafe` **package = Go యొక్క type safety & memory safety ని bypass చేయడం.** Pointer arithmetic, type reinterpretation, memory layout hacks – ఇవన్నీ సాధ్యం, కానీ **GC guarantees కోల్పోతాం, portability పోతుంది, program crash కావచ్చు.** పేరులోనే warning – "unsafe." 99.9% Go code దీన్ని touch చేయదు. కానీ high-performance libraries (serialization, zero-copy conversions) కొన్నిసార్లు వాడతాయి.

Core primitives

```
import "unsafe"
```

```
// unsafe.Pointer — ఏ pointer type కి అయినా convert
var f float64 = 3.14
bits := *(*uint64)(unsafe.Pointer(&f))  // float64 bits ని uint64 గా చూడటం

// unsafe.Sizeof / Alignof / Offsetof — memory layout inspection
type S struct { a bool; b int64; c bool }
unsafe.Sizeof(S{})       // 24 (padding తో — Topic 11)
unsafe.Alignof(int64(0)) // 8
unsafe.Offsetof(S{}.b)   // 8 (padding తర్వాత)
```

unsafe.Pointer – the bridge

`unsafe.Pointer` = ఏ type యొక్క pointer తో అయినా convert చేయగల special pointer. `*T ↔ unsafe.Pointer ↔ *U` conversions allow చేస్తుంది (type system bypass).

```
// Zero-copy []byte → string (allocation తప్పించడం — advanced!)
func bytesToString(b []byte) string {
        return *(*string)(unsafe.Pointer(&b))
        // (Go 1.20+ లో unsafe.String/unsafe.Slice safer)
}
```

uintptr rules – the dangerous part

`uintptr` = integer that holds a pointer's bits. **కానీ uintptr GC కి pointer కాదు** – GC దాన్ని track చేయదు! ఒక object యొక్క address ని uintptr గా store చేస్తే, GC ఆ object ని move (stack growth) లేదా collect చేయవచ్చు → uintptr dangling.

కీలక rule: `unsafe.Pointer → uintptr → unsafe.Pointer` conversion **ఒకే expression లో** జరగాలి (atomic, GC gap లేకుండా):

```
// ✅ SAFE — ఒకే expression, GC gap లేదు
p := unsafe.Pointer(uintptr(unsafe.Pointer(&arr[0])) + offset)

// ❌ DANGEROUS — uintptr ని variable లో store చేయడం
addr := uintptr(unsafe.Pointer(&arr[0]))  // GC arr ని move చేయవచ్చు!
// ... ఈ మధ్యలో GC → addr invalid
p := unsafe.Pointer(addr + offset)         // dangling — crash/corruption
```

Go 1.17+/1.20+ safer helpers

```
// unsafe.Slice — pointer + len → slice (bounds-aware)
s := unsafe.Slice(ptr, length)

// unsafe.String / unsafe.StringData (1.20+) — safer string↔bytes
str := unsafe.String(&b[0], len(b))
data := unsafe.StringData(str)
```

ఇవి raw uintptr arithmetic కంటే safer – వీటిని ప్రాధాన్యం.

ఎప్పుడు justified

- **Zero-copy conversions** (`[]byte ↔ string`) – hot paths లో allocation తప్పించడం (high-perf serialization).
- **Struct field manipulation** at low level (rare – cgo, syscalls).
- **Interfacing with C / syscalls** (cgo, os packages internals).
- **Performance-critical libraries** (encoding, protobuf) – measured, isolated, well-tested.

ఎప్పుడూ కాదు: general application code. Type safety, portability, maintainability ఇచ్చే safety worth చాలా ఎక్కువ. `go vet` unsafe misuse ని కొంత catch చేస్తుంది.

Real-life Scenario

> **unsafe = ఒక car యొక్క safety features (seatbelt, airbag) ని బలవంతంగా disable చేయడం** కొంచెం extra speed కోసం. Professional race track (isolated high-perf library) లో, expert driver (careful developer), controlled conditions లో కొన్నిసార్లు worth. కానీ రోజువారీ రోడ్లు (application code) మీద = ప్రమాదం, crash వస్తే మిగిలేది ఏమీ ఉండదు.
>
> **uintptr rule = "seatbelt తీసిన క్షణంలోనే గమ్యం చేరాలి."** Pointer ని uintptr చేసి, integer math చేసి, తిరిగి pointer చేయడం ఒకే క్షణంలో (expression) – మధ్యలో ఆగితే (variable store), GC "car" ని move చేసి నీ address పనికిరాకుండా చేస్తుంది.

Gotchas (సాధారణ తప్పులు)

- **uintptr ని variable లో store చేయడం** → GC object move/collect → dangling. ఒకే expression లో convert.
- **unsafe.Pointer ని GC-invisible చేయడం** – object ని artificially keep-alive చేయాలంటే `runtime.KeepAlive`.
- **Portability loss** – struct layouts platform-specific (padding, endianness). `unsafe` code non-portable.
- **Type reinterpretation bugs** – wrong size/layout → memory corruption (silent).
- **Version fragility** – internal layouts Go versions మధ్య మారవచ్చు (unsafe assumptions break).
- **Raw uintptr arithmetic** – 1.17+ `unsafe.Slice` / `unsafe.String` safer alternatives.

Key Points

- `unsafe` = bypass type/memory safety – pointer arithmetic, type reinterpretation.
- `unsafe.Pointer` = universal pointer bridge (`*T ↔ *U`); `Sizeof/Alignof/Offsetof` = layout.
- **`uintptr` GC-invisible** – `Pointer→uintptr→Pointer` ఒకే expression (GC gap లేదు).
- Prefer **`unsafe.Slice` / `unsafe.String`** (1.17+/1.20+) over raw uintptr.
- Justified: zero-copy conversions, cgo/syscalls, high-perf libraries (isolated, tested).
- **Never in general app code** – loses GC guarantees, portability, safety.

Interview దృష్టి

Q: unsafe.Pointer vs uintptr – తేడా, danger? A: `unsafe.Pointer` = GC-tracked pointer (object keep-alive, move-aware). `uintptr` = plain integer holding an address – **GC దీన్ని track చేయదు.** uintptr ని variable లో store చేస్తే, GC ఆ object ని move (stack copy) లేదా collect చేయవచ్చు → uintptr dangling → corruption. అందుకే `Pointer→uintptr→Pointer` ఒకే expression లో (GC gap లేకుండా) చేయాలి.

Q: unsafe ఎప్పుడు justified? A: Zero-copy conversions (`[]byte↔string` allocations తప్పించడం), cgo/syscall interfacing, performance-critical libraries (serialization) – isolated, benchmarked, well-tested contexts లో. General application code లో ఎప్పటికీ కాదు – type safety, portability, maintainability worth much more. Go 1.17+/1.20+ `unsafe.Slice` / `String` safer helpers ప్రాధాన్యం.

Q: []byte → string zero-copy ఎందుకు కావచ్చు? A: `string(b)` bytes ని copy చేస్తుంది (immutability కోసం allocation). High-throughput paths (parsers, serializers) లో ఈ copy overhead గణనీయం. `unsafe` (లేదా 1.20+ `unsafe.String`) copy లేకుండా reinterpret చేస్తుంది – కానీ resulting string ని mutate చేయకూడదు (undefined behavior), original bytes alive ఉండాలి. Careful గా, isolated గా మాత్రమే.

Part 6 — Applied Go & Idioms

> ఇక్కడ Go ని real-world లో ఎలా వాడతారో — HTTP servers/clients, JSON/config/database, idiomatic Go (Effective Go), gotchas cheat-sheet, మరియు interview cheat-sheet. ఇవి production Go engineer యొక్క daily bread. చివరి రెండు topics మొత్తం guide ని reference tables గా condense చేస్తాయి.

---

39. HTTP — net/http Server & Client

వివరణ

Go's `net/http` = production-grade HTTP server + client **standard library లోనే** — Express/Flask లాంటి frameworks అవసరం లేదు (Docker, K8s, Kubernetes API server అన్నీ దీన్నే వాడతాయి, భారీ scale లో). Goroutine-per-request model (netpoller వల్ల lakhs of connections handle చేస్తుంది, Topic 20).

Basic HTTP server

```
package main

import (
	"encoding/json"
	"log"
	"net/http"
)

func main() {
	mux := http.NewServeMux()   // router (ServeMux)

	mux.HandleFunc("GET /users/{id}", getUser)   // Go 1.22+ method+wildcard
	mux.HandleFunc("POST /users", createUser)
	mux.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request) {
		w.WriteHeader(http.StatusOK)
		w.Write([]byte("ok"))
	})

	log.Fatal(http.ListenAndServe(":8080", mux))
}

func getUser(w http.ResponseWriter, r *http.Request) {
	id := r.PathValue("id")   // Go 1.22+ path wildcard extract
	user := User{ID: id, Name: "Surya"}
	w.Header().Set("Content-Type", "application/json")
	json.NewEncoder(w).Encode(user)   // struct → JSON response
}

// createUser — POST body ని JSON గా decode చేసి కొత్త user return చేస్తుంది
```

```
func createUser(w http.ResponseWriter, r *http.Request) {
        var u User
        if err := json.NewDecoder(r.Body).Decode(&u); err != nil {
                http.Error(w, err.Error(), http.StatusBadRequest) // invalid JSON →
400
                return
        }
        w.Header().Set("Content-Type", "application/json")
        w.WriteHeader(http.StatusCreated) // 201
        json.NewEncoder(w).Encode(u)
}

// User — JSON tags response/request shape ని define చేస్తాయి
type User struct {
        ID   string `json:"id"`
        Name string `json:"name"`
}
```

Handler & HandlerFunc

Core interface: **`http.Handler`** — ఒక్క method `ServeHTTP(w, r)`. `http.HandlerFunc` = ఒక function ని Handler గా adapt చేసే type (Go's function-as-interface idiom).

```
type Handler interface {
        ServeHTTP(w http.ResponseWriter, r *http.Request)
}
// HandlerFunc — plain function ని Handler గా
type HandlerFunc func(w http.ResponseWriter, r *http.Request)
func (f HandlerFunc) ServeHTTP(w http.ResponseWriter, r *http.Request) { f(w, r) }
```

ServeMux (routing) — Go 1.22+ enhancements

Go 1.22 లో `ServeMux` కి **method matching + path wildcards** వచ్చాయి — third-party router అవసరం తగ్గింది.

```
mux.HandleFunc("GET /items/{id}", ...)        // method + wildcard
mux.HandleFunc("GET /files/{path...}", ...)  // trailing wildcard (rest)
r.PathValue("id")                             // wildcard value
```

Middleware — composition via wrapping

Middleware = Handler ని wrap చేసే Handler (logging, auth, recovery). Go లో decorators గా compose.

```
func Logging(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
                start := time.Now()
                next.ServeHTTP(w, r)   // inner handler call
                log.Printf("%s %s %v", r.Method, r.URL.Path, time.Since(start))
        })
}

func Recover(next http.Handler) http.Handler {
```

```
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
                defer func() {
                        if err := recover(); err != nil {
                                http.Error(w, "internal error",
http.StatusInternalServerError)
                        }
                }()
                next.ServeHTTP(w, r)
        })
}
// chain: Logging(Recover(mux))
handler := Logging(Recover(mux))
http.ListenAndServe(":8080", handler)
```

HTTP Client – with timeout & context

```
// ❌ http.Get / DefaultClient — timeout లేదు (hangs forever risk)
// ✅ custom client with timeout
client := &http.Client{Timeout: 10 * time.Second}

ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)
resp, err := client.Do(req)
if err != nil {
        return err
}
defer resp.Body.Close()   // తప్పనిసరి — leak తప్పించడం!

body, _ := io.ReadAll(resp.Body)
```

Graceful shutdown – production essential

Server ని in-flight requests పూర్తయ్యేవరకు wait చేసి, తర్వాత shutdown (deploy/SIGTERM లో).

```
srv := &http.Server{Addr: ":8080", Handler: mux}
go func() {
        if err := srv.ListenAndServe(); err != http.ErrServerClosed {
                log.Fatal(err)
        }
}()

// SIGTERM/SIGINT కోసం wait
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()
srv.Shutdown(ctx)   // in-flight requests drain, కొత్తవి reject
log.Println("server stopped gracefully")
```

Real-life Scenario

> **HTTP server = ఒక restaurant.** ServeMux = host/receptionist (ఏ table ఏ waiter కి route). Handler = waiter (ఒక request serve). Middleware = kitchen quality checks — ప్రతి dish ముందు "hygiene check (auth), log (order record), taste test (recovery)" wrapped layers.
>
> **Graceful shutdown = "ఇక కొత్త customers రాకండి (కొత్త requests reject), కానీ లోపల తింటున్నవాళ్ళు (in-flight) పూర్తి చేయనివ్వండి, తర్వాత మూసేద్దాం."** హఠాత్తుగా light ఆర్పి అందరినీ బయటకి తోయడం (hard kill) = అర్ధాంతర requests, data loss.

Gotchas (సాధారణ తప్పులు)

- **`resp.Body.Close()` మర్చిపోవడం** → connection/goroutine leak. `defer resp.Body.Close()` ఎప్పుడూ.
- **`http.DefaultClient` / `http.Get` timeout లేదు** → hung requests. Custom `http.Client{Timeout}`.
- **Handler లో panic → ఆ goroutine crash** (default recover ఉంది కానీ connection closes). Recovery middleware.
- **Response write తర్వాత WriteHeader** — order తప్పు (WriteHeader ముందు). Double WriteHeader warning.
- **Blocking work in handler** — goroutine hog; context timeout, async offload.
- **Shared handler state without sync** → races (handlers concurrent).

Key Points

- `net/http` = production server + client (stdlib); goroutine-per-request + netpoller.
- **`http.Handler`** (`ServeHTTP`) core; `HandlerFunc` = function adapter.
- **ServeMux** (1.22+): method matching + path wildcards (`{id}`, `PathValue`).
- **Middleware** = handler-wrapping decorators (logging, auth, recovery).
- **Client**: custom `http.Client{Timeout}` + context; **`defer resp.Body.Close()`**.
- **Graceful shutdown** — `srv.Shutdown(ctx)` drains in-flight (SIGTERM).

Interview దృష్టి

Q: Go HTTP server లక్షల connections ఎలా handle చేస్తుంది? A: Goroutine-per-request model + netpoller (Topic 20). ప్రతి connection ఒక goroutine (2KB, cheap), కానీ blocked network I/O లో goroutine netpoller (epoll/kqueue) కి park అవుతుంది, OS thread free. కొన్ని threads లక్షల goroutines ని multiplex చేస్తాయి — thread-per-connection (Java classic) లా కాదు, C10K problem solved natively.

Q: Middleware ఎలా implement చేస్తావు? A: Handler ని accept చేసి, wrapped Handler return చేసే function — `func(next http.Handler) http.Handler`. Inner `ServeHTTP(w, r)` ముందు/తర్వాత logic (logging, auth, recover). Chain: `Logging(Auth(Recover(mux)))`. Composition via wrapping — Go's decorator pattern, frameworks అవసరం లేదు.

Q: Graceful shutdown ఎందుకు, ఎలా? A: Deploy/scale-down లో server ని kill చేస్తే in-flight requests fail (data loss, bad UX). `srv.Shutdown(ctx)` – కొత్త connections reject చేసి, in-flight requests context deadline వరకు drain అయ్యేవరకు wait. SIGTERM/SIGINT signal catch చేసి Shutdown call చేయాలి. Zero-downtime deploys కి essential.

Raw TCP / UDP – `net` package (HTTP కి కింద ఉన్నది)

`net/http` అనేది `net` package మీద కట్టబడింది. Custom protocols (game servers, proxies, DB drivers, gRPC transport) కోసం నేరుగా TCP/UDP sockets వాడతాం. ఇక్కడ కూడా **goroutine-per-connection** idiom – netpoller వల్ల లక్షల connections scale అవుతాయి.

```
// TCP echo server — ప్రతి connection ఒక goroutine
func main() {
        ln, err := net.Listen("tcp", ":9000")   // listening socket
        if err != nil {
                log.Fatal(err)
        }
        defer ln.Close()
        for {
                conn, err := ln.Accept()   // కొత్త client connection కోసం block
                if err != nil {
                        continue
                }
                go handle(conn)            // ప్రతి client ని concurrent గా serve
        }
}

func handle(conn net.Conn) {
        defer conn.Close()
        conn.SetReadDeadline(time.Now().Add(30 * time.Second)) // idle timeout
        scanner := bufio.NewScanner(conn)   // bufio = efficient buffered read
        for scanner.Scan() {
                line := scanner.Text()
                conn.Write([]byte("echo: " + line + "\n"))
        }
}
```

```
// TCP client
conn, err := net.Dial("tcp", "localhost:9000")
if err != nil { log.Fatal(err) }
defer conn.Close()
fmt.Fprintln(conn, "hello")
resp, _ := bufio.NewReader(conn).ReadString('\n')
fmt.Print(resp)   // echo: hello

// UDP — connectionless (packets, no handshake, no delivery guarantee)
udpAddr, _ := net.ResolveUDPAddr("udp", ":9001")
udpConn, _ := net.ListenUDP("udp", udpAddr)
buf := make([]byte, 1024)
n, clientAddr, _ := udpConn.ReadFromUDP(buf)   // ఏ client నుండైనా packet
udpConn.WriteToUDP([]byte("ack"), clientAddr)
_ = n
```

అంశం	TCP (`net.Listen` / `Dial`)	UDP (`net.ListenUDP`)
Connection	Stream, handshake, ordered, reliable	Connectionless, packets, unordered
API	`Accept` / `Read` / `Write` (stream)	`ReadFromUDP` / `WriteToUDP` (datagram)
వాడకం	HTTP, gRPC, DB, chat	DNS, video/voice, gaming, metrics
Go idiom	goroutine-per-conn	ఒక goroutine loop (packets)

> **Key idea:** `net.Conn` అనేది `io.Reader` + `io.Writer` interface ని satisfy చేస్తుంది – కాబట్టి `bufio` , `io.Copy` , `encoding/json` డైరెక్ట్‌గా socket మీద పనిచేస్తాయి. TCP server లో ఎప్పుడూ read/write **deadlines** (`SetReadDeadline`) పెట్టు – లేకపోతే slow/dead clients goroutines ని లీక్ చేస్తాయి.

---

40. JSON & Config, database/sql + Connection Pool, Context in Requests

వివరణ

Production Go service = HTTP handler → config → database, request-scoped context తో wired. ఈ topic ఆ backend plumbing – JSON/config handling, `database/sql` (connection pooling తో), request context propagation.

Config – env, flags, files

```
// 1. Environment variables (12-factor apps)
port := os.Getenv("PORT")
if port == "" { port = "8080" }

// 2. Flags
var debug = flag.Bool("debug", false, "enable debug")
flag.Parse()

// 3. Struct + JSON/YAML config file
type Config struct {
	Port     int    `json:"port"`
	Database struct {
		DSN          string `json:"dsn"`
		MaxOpenConns int    `json:"max_open_conns"`
	} `json:"database"`
}
func loadConfig(path string) (*Config, error) {
	data, err := os.ReadFile(path)
	if err != nil {
		return nil, fmt.Errorf("read config: %w", err)
	}
	var cfg Config
	if err := json.Unmarshal(data, &cfg); err != nil {
```

```
                return nil, fmt.Errorf("parse config: %w", err)
        }
        return &cfg, nil
}
```

database/sql – standard DB interface

`database/sql` = database-agnostic interface; ఒక **driver** (postgres, mysql, sqlite) blank-import చేస్తాం (init() registers). `*sql.DB` = **connection pool** (single connection కాదు!).

```
import (
        "database/sql"
        _ "github.com/lib/pq"   // postgres driver — blank import (init registers)
)

db, err := sql.Open("postgres", dsn)   // pool create (connection ఇంకా open కాదు!)
if err != nil { return err }
defer db.Close()

if err := db.Ping(); err != nil {       // నిజంగా connect verify
        return err
}
```

Connection Pool tuning – production critical

`*sql.DB` internally connection pool ని manage చేస్తుంది. Defaults production కి సరిపోవు – tune చేయాలి.

```
db.SetMaxOpenConns(25)                  // max concurrent connections
db.SetMaxIdleConns(25)                  // idle pool size (reuse)
db.SetConnMaxLifetime(5 * time.Minute)  // connection recycle (stale తప్పించడం)
db.SetConnMaxIdleTime(1 * time.Minute)  // idle connection timeout
```

ఎందుకు? DB కి limited connections (ఉదా. Postgres default 100). MaxOpenConns ఎక్కువ → DB overwhelm; తక్కువ → app throughput bottleneck. Load ఆధారంగా tune. ConnMaxLifetime – load balancer/DB failover behind stale connections తప్పించడం.

Queries with context

ఎప్పుడూ `...Context` variants – request cancel/timeout అయితే query కూడా cancel. Slow queries hang తప్పించడం.

```
func getUser(ctx context.Context, db *sql.DB, id int) (User, error) {
        var u User
        err := db.QueryRowContext(ctx,
                "SELECT id, name, email FROM users WHERE id = $1", id).
                Scan(&u.ID, &u.Name, &u.Email)
        if err == sql.ErrNoRows {
                return User{}, ErrNotFound
        }
        if err != nil {
                return User{}, fmt.Errorf("query user %d: %w", id, err)
```

```
        }
        return u, nil
}

// Multiple rows
rows, err := db.QueryContext(ctx, "SELECT id, name FROM users")
if err != nil { return err }
defer rows.Close()   // తప్పనిసరి!
for rows.Next() {
        var u User
        if err := rows.Scan(&u.ID, &u.Name); err != nil { return err }
        users = append(users, u)
}
return rows.Err()    // iteration errors check
```

Transactions

```
tx, err := db.BeginTx(ctx, nil)
if err != nil { return err }
defer tx.Rollback()   // commit అయితే no-op; error path లో rollback

_, err = tx.ExecContext(ctx, "UPDATE accounts SET balance = balance - $1 WHERE id =
$2", amt, from)
if err != nil { return err }
_, err = tx.ExecContext(ctx, "UPDATE accounts SET balance = balance + $1 WHERE id =
$2", amt, to)
if err != nil { return err }

return tx.Commit()    // అన్నీ OK అయితే commit
```

Context propagation – end-to-end

HTTP request → handler → service → DB – ఒక్క context అంతా propagate. Request cancel → అన్నీ downstream ops cancel.

```
func handler(w http.ResponseWriter, r *http.Request) {
        ctx := r.Context()              // request-scoped context (client disconnect
→ cancel)
        user, err := getUser(ctx, db, 42) // ctx DB query వరకు propagate
        // ...
}
```

Real-life Scenario

> **Connection pool = ఒక call center లో fixed phone lines (25).** ప్రతి customer కి కొత్త line వేయరు (connection open ఖరీదు); ఉన్న lines ని reuse. అన్ని busy అయితే కొత్త calls wait. లైన్లు ఎక్కువ → exchange (DB) overload; తక్కువ → customers wait. Balance కీలకం. ConnMaxLifetime = "ప్రతి line ని 5 నిమిషాలకోసారి refresh" (పాత crackly lines తప్పించడం).
>
> **Context propagation = ఒక order tracking number** – customer request నుండి, warehouse, delivery వరకు అదే number. Customer "cancel" చెప్తే, ఆ number ఉన్న అన్ని stages ఆగిపోతాయి (DB query కూడా).

Gotchas (సాధారణ తప్పులు)

- `sql.Open` **connection open చేయదు** – lazy pool. `db.Ping()` తో verify.
- `rows.Close()` **/** `resp.Body.Close()` **మర్చిపోవడం** → connection leak, pool exhaustion.
- **Non-context queries** → request cancel అయినా query hang. `...Context` variants.
- `*sql.DB` **per-request create** – anti-pattern! ఒక్క shared pool (app lifetime).
- **String concat SQL** → SQL injection. **Parameterized** (`$1` , `?`) ఎప్పుడూ.
- **Default pool settings production లో** – MaxOpenConns unlimited → DB overwhelm. Tune.
- `rows.Err()` **check మర్చిపోవడం** – iteration errors silent.
- **Transaction rollback మర్చిపోవడం** – `defer tx.Rollback()` (commit తర్వాత no-op).

Key Points

- Config: env vars (12-factor) + flags + struct/JSON files; wrap errors with context.
- `database/sql` = driver-agnostic; `*sql.DB` **= connection pool** (not one conn).
- **Tune pool**: `SetMaxOpenConns/MaxIdleConns/ConnMaxLifetime` (production critical).
- `...Context` **query variants** – request cancel → query cancel.
- **Parameterized queries** (SQL injection); `defer rows.Close()` , `rows.Err()` .
- **Context propagation** end-to-end (handler → service → DB).
- One shared `*sql.DB` (app lifetime), not per-request.

Interview దృష్టి

Q: `*sql.DB` **connection నా pool నా?** A: **Connection pool** – single connection కాదు. `sql.Open` lazily pool create చేస్తుంది (ఇంకా connect చేయదు – `Ping` verify). Concurrent goroutines దీన్ని safely share చేస్తాయి; pool connections lend/reuse చేస్తుంది. అందుకే app lifetime కి ఒక్క shared `*sql.DB` – per-request create చేయకూడదు.

Q: Connection pool ఎందుకు tune చేయాలి? A: Default settings production కి సరిపోవు. MaxOpenConns ఎక్కువ → DB (limited connections, ఉదా. Postgres 100) overwhelm; తక్కువ → app throughput bottleneck (goroutines wait). ConnMaxLifetime → load balancer/DB failover behind stale connections తప్పించడం. Load testing తో tune.

Q: Query లో context ఎందుకు? A: `QueryContext` / `ExecContext` — request cancel (client disconnect) లేదా timeout అయితే running query కూడా cancel అవుతుంది (DB కి cancellation signal). లేకపోతే slow query request పోయినా hang అవుతుంది, connection pool exhaust చేస్తుంది. Request context ని handler నుండి DB వరకు propagate చేయాలి.

---

41. Idiomatic Go (Effective Go Highlights)

వివరణ

"Idiomatic Go" = Go community accepted conventions & patterns. Syntactically correct కంటే **idiomatic** గా రాయడం seniority signal. Reference: **Effective Go** + **Go Code Review Comments** + **Google Go Style Guide.** కీలక principles ని చూద్దాం.

1. Naming — short, clear, MixedCase

```
// ✅ Short in small scope (idiomatic)
for i, v := range items { ... }   // i, v — not index, value
c := make(chan int)                // c, not theChannel

// ✅ MixedCase, not snake_case
var userID int                     // not user_id
func GetUserByID() {}              // not Get_User_By_ID

// ✅ Getters: no "Get" prefix
func (u User) Name() string {}     // not GetName()
func (u *User) SetName(n string){} // Setters: Set prefix OK

// ✅ Interfaces: -er suffix
type Reader interface { Read(...) }
type Stringer interface { String() string }

// ✅ Package names: short, lowercase, no underscores
package httputil    // not http_util or httpUtils
// Avoid stutter: http.HTTPServer → http.Server
```

2. Error handling — explicit, wrapped, early return

```
// ✅ Handle immediately, early return (no deep nesting)
func process() error {
        data, err := fetch()
        if err != nil {
                return fmt.Errorf("fetch: %w", err)   // wrap with context
        }
        result, err := transform(data)
        if err != nil {
                return fmt.Errorf("transform: %w", err)
        }
        return save(result)
```

```
}
// ❌ Don't: log AND return (double handling); don't ignore errors
```

3. Accept interfaces, return structs

```
// ✅ Accept interface (flexible input) — io.Reader ఏదైనా
func Process(r io.Reader) (*Result, error) { ... }
// ✅ Return concrete struct (caller కి full type)
func NewService() *Service { ... }   // interface కాదు
```

Consumer తన అవసరమైన interface ని define చేసుకుంటుంది (producer package లో కాదు).

4. Zero value useful

```
// ✅ Type ని initialize లేకుండా usable గా design
var mu sync.Mutex        // వెంటనే usable (zero value ready)
var buf bytes.Buffer     // వెంటనే usable
var wg sync.WaitGroup    // వెంటనే
// Constructors అవసరం లేకుండా zero value meaningful
```

5. Composition over inheritance

```
// ✅ Embed, don't inherit (Go కి inheritance లేదు)
type Server struct {
        *log.Logger   // embed — logging methods promoted
        handler http.Handler
}
```

6. Small interfaces

```
// ✅ చిన్న interfaces (1-3 methods) — powerful, composable
type Reader interface { Read(p []byte) (int, error) }   // ఒక్క method!
// "The bigger the interface, the weaker the abstraction." — Rob Pike
// అవసరమైన చోట compose: io.ReadWriteCloser
```

7. Other key idioms

```
// defer for cleanup (close, unlock)
defer f.Close()
defer mu.Unlock()

// comma-ok patterns
val, ok := m[key]
v, ok := i.(string)

// Named return కి restraint (చిన్న functions కే)
// Blank import for side effects: _ "github.com/lib/pq"
// Blank identifier: _, err := f()
```

```
// Guard clauses / early returns (happy path unindented)
if err != nil { return err }
// ... happy path ఇక్కడ, deeply nested కాదు

// Prefer channels/mutex based on expressiveness (dogma కాదు)
```

Idioms summary table

Idiom	Rule
Naming	Short (small scope), MixedCase, no `Get` prefix, `-er` interfaces
Errors	Explicit, wrap `%w`, early return, no double-handle
Interfaces	Accept interfaces, return structs; keep small
Zero value	Make it useful (no constructor needed)
Composition	Embed, don't inherit
Cleanup	`defer` (close, unlock)
Control flow	Guard clauses, early return (happy path flat)
Concurrency	Channels for orchestration, mutex for state

Real-life Scenario

> **Idiomatic Go = ఒక ఊరిలో అందరూ మాట్లాడే యాస (dialect).** వ్యాకరణబద్ధమైన తెలుగు (syntactically correct) రాయగలవు, కానీ locals మాట్లాడే యాస (idioms) రాస్తే నువ్వు "వాళ్ళలో ఒకడివి" (senior Go dev) అని తెలుస్తుంది. `GetName()` కి బదులు `Name()`, deep nesting కి బదులు early return — ఇవి native Go speakers కి natural.
>
> **Small interfaces = ఒక్క పని బాగా చేసే tool.** Swiss-army knife (పెద్ద interface) కంటే, ఒక్క పదునైన కత్తి (io.Reader) ఎక్కువ చోట్ల plug అవుతుంది.

Gotchas (సాధారణ తప్పులు)

- **Java/C# habits తెచ్చుకోవడం** – getters (`GetX`), పెద్ద interfaces, deep inheritance thinking. Go-native ఆలోచించు.
- **Premature interfaces** – ఒక్క implementation ఉంటే interface అవసరం లేదు. అవసరమైనప్పుడు extract.
- **Returning interfaces from constructors** – concrete struct return చేయి (flexibility caller కి).
- **Ignoring gofmt/golangci-lint** – idiom violations. Tooling trust చేయి.
- **Over-commenting obvious code** – idiomatic Go self-documenting; comments why, not what.
- **Panic ని control flow కి** – errors return (Topic 18).

Key Points

- **Naming**: short (small scope), MixedCase, no `Get` , `-er` interfaces, no stutter.
- **Errors**: explicit, wrap `%w` , early return, no double-handle/ignore.
- **Accept interfaces, return structs**; consumer defines interface.
- **Zero value useful** (no mandatory constructors).
- **Composition (embed) over inheritance**; **small interfaces**.
- **Guard clauses / early return** (flat happy path); `defer` cleanup.
- Trust `gofmt` / `golangci-lint` ; self-documenting code.

Interview దృష్టి

Q: "Accept interfaces, return structs" ఎందుకు? A: Interface accept – caller ఏ implementation అయినా pass చేయగలడు (flexible, testable, mockable). Concrete struct return – caller కి full type, అన్ని methods/fields అందుబాటులో (interface return చేస్తే artificial restriction). Consumer తన అవసరమైన interface ని define చేసుకుంటుంది – loose coupling.

Q: "Zero value useful" idiom? A: Type ని initialize/constructor లేకుండా directly usable గా design చేయడం. `sync.Mutex` , `bytes.Buffer` , `sync.WaitGroup` – `var x T` వెంటనే ready. Boilerplate తగ్గుతుంది, APIs cleaner. Design లో zero values ni meaningful గా చేయాలి (nil slice appendable, nil map readable).

Q: Small interfaces ఎందుకు మంచివి? A: "The bigger the interface, the weaker the abstraction." చిన్న interfaces (io.Reader – ఒక్క method) ఎక్కువ types implement చేస్తాయి (wide adoption), compose అవుతాయి (io.ReadWriter), mock సులభం, single responsibility. పెద్ద interfaces tight coupling + tough to satisfy. Go stdlib చిన్న interfaces (Reader, Writer, Stringer) మీద నిర్మించబడింది.

---

42. Gotchas & Pitfalls Cheat-sheet

వివరణ

Go సులభమైన భాష కానీ కొన్ని పదునైన మూలలు (sharp edges) ఉన్నాయి – వీటిని ఒకచోట consolidate చేద్దాం. ఇవి production bugs + interview traps. ప్రతి ఒక్కటి **bug + fix** తో. ఈ list ని memorize చేస్తే చాలా production incidents తప్పించవచ్చు.

1. nil map write → panic

```
var m map[string]int   // nil
m["x"] = 1             // ❌ panic: assignment to entry in nil map
// ✅ Fix:
m := make(map[string]int)   // లేదా map literal
```

2. Slice aliasing / append corruption

```
a := []int{1, 2, 3, 4}
b := a[:2]
b = append(b, 99)   // ❌ a[2] overwritten (shared backing array)
// ✅ Fix: full slice expr — a[:2:2] (cap limit → append allocates new)
```

3. Loop variable capture (pre-Go 1.22)

```
for _, v := range items {
        go func() { use(v) }()   // ❌ pre-1.22: అన్నీ చివరి v
}
// ✅ Fix (pre-1.22): v := v  లేదా  go func(v T){...}(v)
// Go 1.22+: automatically fixed (per-iteration variable)
```

4. defer in loop → resource pile-up

```
for _, path := range paths {
        f, _ := os.Open(path)
        defer f.Close()   // ❌ function చివర్లోనే — files పేరుకుపోతాయి
}
// ✅ Fix: loop body ని function లో wrap (defer per-iteration)
```

5. Error shadowing with :=

```
x, err := f1()
if cond {
        y, err := f2()   // ❌ కొత్త err (shadow) — outer err update కాదు
        _ = y; _ = err
}
return err            // outer err (బహుశా stale)
// ✅ Fix: y, err = f2()  (= not :=)  లేదా go vet -vettool=shadow
```

6. Goroutine leak — no cancellation

```
go func() { val := <-ch }()   // ❌ ch కి ఎవరూ send చేయకపోతే leak forever
// ✅ Fix: context/done channel, select with <-ctx.Done()
```

7. Send/close on closed channel → panic

```
close(ch); ch <- 1   // ❌ panic: send on closed channel
close(ch)            // ❌ panic: close of closed channel (double)
// ✅ Fix: single owner closes once; sync.Once guard
```

8. Typed nil in interface → != nil

```
func f() error {
        var e *MyError = nil
        return e   // ❌ interface {*MyError, nil} ≠ nil
}
if f() != nil { /* wrongly true! */ }
// ✅ Fix: explicit return nil (typed nil pointer కాదు)
```

9. Float equality

```
0.1 + 0.2 == 0.3   // ❌ false (IEEE-754)
// ✅ Fix: math.Abs(a-b) < epsilon
```

10. Pointer receiver + value → interface unsatisfied

```
func (t *T) M() {}
var i I = T{}   // ❌ T value method set లో M లేదు
var i I = &T{}  // ✅
```

11. String len = bytes, not chars

```
len("నమస్తే")   // ❌ 18 (bytes), 6 కాదు
// ✅ Fix: utf8.RuneCountInString(s)
```

12. resp.Body / rows / file Close మర్చిపోవడం

```
resp, _ := http.Get(url)
// ❌ resp.Body.Close() లేదు → leak
// ✅ Fix: defer resp.Body.Close()
```

13. time.After in loop → timer leak

```
for {
        select {
        case <-time.After(time.Second):  // ❌ ప్రతి iteration కొత్త timer
        }
}
// ✅ Fix: reusable time.Timer లేదా context deadline
```

14. Map iteration order assumed

```
for k := range m { ... }   // ❌ random order — దాని మీద ఆధారపడకు
// ✅ Fix: keys collect + sort
```

15. Integer division / overflow surprises

```
5 / 2          // ❌ 2 (integer division, 2.5 కాదు)
var b uint8 = 255; b++   // ❌ 0 (silent overflow wrap)
// ✅ Fix: float64(5)/2; తగిన integer size
```

Master cheat table

#	Gotcha	Fix
1	nil map write panic	`make`/literal init
2	Slice append aliasing	Full slice expr `a[:n:n]`, `copy`
3	Loop var capture	`v:=v` / arg pass / Go 1.22+
4	defer in loop	Wrap body in function
5	Error shadow `:=`	Use `=`, `go vet` shadow
6	Goroutine leak	context/done cancellation
7	Closed channel panic	Single owner, `sync.Once`
8	Typed nil interface	Explicit `return nil`
9	Float equality	epsilon compare
10	Pointer receiver + value	`&value`
11	String len = bytes	`utf8.RuneCountInString`
12	Unclosed Body/rows/file	`defer x.Close()`
13	time.After loop leak	Reusable timer / ctx
14	Map order assumed	Sort keys
15	Int division/overflow	Convert / size

Real-life Scenario

> **ఈ gotchas = రహదారిపై hidden potholes.** రోడ్డు (Go) బాగానే ఉంది, కానీ ఈ 15 గుంతలు తెలియకపోతే production లో టైరు పగులుతుంది (incident). Senior driver ఈ గుంతల map ని మనసులో పెట్టుకుని, ప్రతి దాని దగ్గర automatic గా జాగ్రత్త పడతాడు – `defer Close`, `make(map)`, `a[:n:n]`, `-race`.

Key Points

- nil map write, slice aliasing, loop var (pre-1.22), defer-in-loop, error shadow – top 5 silent bugs.

- Typed-nil interface, pointer-receiver satisfaction – top interface traps.
- Goroutine leak, closed-channel panic – top concurrency traps.
- Always `defer x.Close()` (Body/rows/file); `-race` in CI.
- String len = bytes; float equality = epsilon; map order = random.

Interview దృష్టి

Q: Go లో అత్యంత common bugs ఏవి? A: (1) Loop variable capture in goroutines/closures (pre-1.22). (2) Slice append aliasing (shared backing array corruption). (3) Goroutine leaks (no cancellation). (4) nil map write panic. (5) Error shadowing with `:=`. (6) Typed-nil interface (`!= nil` surprise). (7) Unclosed resources (Body/rows). వీటిని `go vet`, `-race`, `golangci-lint` చాలావరకు catch చేస్తాయి.

Q: `defer` ని loop లో వాడకూడదా? A: జాగ్రత్తగా. Loop లో `defer` surrounding **function** return వరకు accumulate అవుతుంది (loop iteration కాదు) – వేలాది files/locks పేరుకుపోయి resource exhaustion. Fix: loop body ని ఒక function (closure/named) లో wrap చేసి, ఆ function scope లో defer – per-iteration cleanup.

---

43. Memory Tips + Common Mistakes + Interview Cheat-sheet

వివరణ

మొత్తం guide ని ఒక్క reference గా condense చేసే final topic. Memory tips (గుర్తుంచుకోవడానికి mnemonics + tables), common mistakes summary, మరియు top Go interview Q&A. Interview ముందు దీన్ని చదివితే చాలు.

Memory Tips Table (గుర్తుంచుకోవడానికి)

Concept	ఒక్క వాక్యంలో గుర్తు
Slice	`{ptr, len, cap}` header – window on array; append re-assign, aliasing
Map	Hash buckets (8/bucket); nil write panic; random order; not concurrent-safe
String	Immutable UTF-8 bytes; len=bytes; range=runes
Interface	`{type, value}`; typed-nil ≠ nil; implicit satisfy
Goroutine	2KB stack, grows; M:N on threads; leak = blocked forever
Channel	Unbuffered=handshake, buffered=mailbox; sender closes
GMP	G=goroutine, M=thread, P=context(GOMAXPROCS); work-stealing
GC	Tri-color concurrent mark-sweep; write barrier; GOGC=100

Concept	ఒక్క వాక్యంలో గుర్తు
Escape	Compiler decides stack/heap; `&` ≠ heap; `-gcflags='-m'`
defer	LIFO; args eval immediately; call deferred
Pointer receiver	Method set: `*T` has value+pointer, `T` value only
Value vs pointer	Mutate/large → pointer; small/immutable → value
Error	Values not exceptions; `%w` wrap; `errors.Is/As`
context	Cancellation tree; `defer cancel()`; first arg
Zero value	int=0, string="", bool=false, ref-types=nil; useful

Common Mistakes Table

Mistake	Correct approach
nil map కి write	`make`/literal init ముందు
append return ignore	`s = append(s, x)`
Slice sub-append aliasing	`a[:n:n]` full slice expr
Loop var in goroutine (pre-1.22)	`v:=v` / arg / Go 1.22+
defer in loop	Function లో wrap
Error `:=` shadow	`=` వాడు
Goroutine no cancel	context/done
Send on closed channel	Single owner close
Typed nil as error	Explicit `return nil`
Pointer receiver + value interface	`&value`
No `resp.Body.Close()`	`defer Close()`
`sql.DB` per request	One shared pool
Non-context DB/HTTP	`...Context` variants
Float `==`	epsilon
Reflection in hot path	Generics / codegen

Top Go Interview Q&A (crisp answers)

Q1. Goroutine vs thread? Goroutine = Go runtime managed, 2KB stack (grows), μs creation, user-space switch. Thread = OS managed, 1-2MB, expensive. M:N multiplexing → లక్షలు సాధ్యం.

Q2. Channel deadlock ఎప్పుడు? Unbuffered channel send with no receiver; circular channel waits; all goroutines blocked. Runtime "all goroutines asleep - deadlock!" fatal. Fix: buffered/separate goroutine, consistent ordering.

Q3. Slice internals + append growth? `{ptr, len, cap}` header on backing array. `len<cap` → in-place; `len==cap` → new array (2×/1.25×) + copy. Always `s = append(...)`. Shares backing array → aliasing.

Q4. Escape analysis? Compiler variable escape చేస్తుందా (function scope దాటి live) అని analyze – heap; లేకపోతే stack. `&` / `new` heap కాదని guarantee లేదు. `-gcflags='-m'`. Fewer escapes = less GC pressure.

Q5. GC ఎలా పని చేస్తుంది? Concurrent tri-color mark-sweep. Roots నుండి reachable objects grey→black mark (mostly concurrent), white = garbage sweep. Write barrier = concurrent correctness. Sub-ms STW. GOGC/GOMEMLIMIT tune.

Q6. GMP scheduler? G=goroutine, M=OS thread, P=logical processor + local run queue (count=GOMAXPROCS). G needs M+P to run. Work-stealing (idle P steals). Syscall handoff, netpoller for I/O.

Q7. Interface nil trap? Interface = `{type, value}`. Typed nil pointer → `{*T, nil}` → interface ≠ nil (type non-nil). Both must be nil. Fix: explicit `return nil`.

Q8. context ఎందుకు? Cancellation, deadlines, request-scoped values ని goroutine tree లో propagate. Request cancel/timeout → all downstream ops stop (leaks తప్పించడం). First arg, `defer cancel()`.

Q9. Mutex vs channel? Mutex = shared state protection (simple, fast). Channel = ownership transfer, orchestration. "Use whichever is most expressive." Simple state → mutex; data flow → channel.

Q10. Value vs pointer receiver? Mutate state / large struct → pointer. Small/immutable → value. Method set: `*T` = value+pointer methods, `T` = value only → pointer-receiver method interface ని value satisfy చేయదు. Consistency: ఒకే type కి ఒకే kind.

Q11. defer args ఎప్పుడు eval? `defer` statement execute అయిన క్షణంలో args eval; call return వరకు వాయిదా. `i:=0; defer print(i); i=10` → 0. LIFO order.

Q12. Data race detect? `go test -race` / `go run -race` – shadow memory, happens-before violations report. 5-10× overhead (dev/CI). No false positives, false negatives possible.

Q13. Generics internally? GCShape stenciling + dictionaries – same memory-layout types (all pointers) share one compiled stencil; type-specific info via runtime dictionary. C++ monomorphization కంటే తక్కువ code bloat, slight indirection.

Q14. Goroutine leak detect? `runtime.NumGoroutine()` steadily పెరిగితే; `net/http/pprof` goroutine profile (blocked stacks). Prevent: context cancellation, channel close, timeouts.

Q15. Error handling philosophy? Errors = values (not exceptions). Explicit `if err != nil`, wrap `%w` (context), `errors.Is/As` (sentinel/type). Visible error paths, no hidden control flow. panic = bugs only.

Interview Framework (Go coding round)

1. **Clarify** – inputs, outputs, edge cases, concurrency requirements.
2. **Idiomatic structure** – small functions, error returns, interfaces for deps.
3. **Concurrency** – goroutines + channels/sync; cancellation (context); no leaks.
4. **Correctness** – `-race` mental check; nil/empty/overflow edge cases.
5. **Test** – table-driven tests; error cases.
6. **Discuss** – trade-offs (mutex vs channel, value vs pointer), complexity, scaling.

Real-life Scenario

> **ఈ cheat-sheet = పరీక్ష ముందు రాత్రి చదివే ఒక్క పేజీ సారాంశం.** మొత్తం 43 topics చదివాక, interview ముందు ఈ tables + 15 Q&A ఒక్కసారి revise చేస్తే, అన్ని concepts తాజాగా మనసులో ఉంటాయి. "Goroutine vs thread, slice internals, GC, GMP, escape analysis, interface nil, context" – ఈ 7 దాదాపు ప్రతి Go interview లో వస్తాయి.

Key Points (మొత్తం guide సారాంశం)

- **Foundations** – 25 keywords, zero values, capitalization=access, UTF-8 strings.
- **Data structures** – slices (`{ptr,len,cap}`, aliasing), maps (buckets, nil panic), structs (embedding, padding), interfaces (implicit, typed-nil).
- **Concurrency** – goroutines (2KB), GMP (work-stealing, netpoller), channels (handshake/mailbox), sync/atomic/context, patterns, memory model (`-race`).
- **Runtime** – escape analysis (stack/heap), GC (tri-color, write barrier, GOGC/GOMEMLIMIT), allocator (mcache/mcentral/mheap), copying stacks.
- **Tooling** – gofmt, vet, pprof, trace, race; testing (table-driven, fuzzing).
- **Applied** – net/http (goroutine-per-request), database/sql (pool + context), idiomatic Go (accept interfaces/return structs, small interfaces, zero-value useful).

Interview దృష్టి (final meta)

Q: Go ని ఒక్క వాక్యంలో ఎందుకు ఎంచుకుంటావు? A: Simple language (fast onboarding + maintainable large teams), built-in concurrency (goroutines/channels – cloud-native workloads కి perfect), single static binary (trivial deployment), great tooling & stdlib, low-latency GC. Docker/K8s/etcd దీన్నే ఎందుకు వాడతాయో అదే కారణం – network services, infrastructure, CLIs కి ideal balance of simplicity, performance, concurrency.

> **ముగింపు:** ఈ 43 topics Go ని absolute basics నుండి deepest internals వరకు cover చేశాయి – syntax నుండి scheduler, GC, allocator, memory model వరకు. ప్రతి concept కి real-life analogy, code, trade-offs, gotchas, interview angle ఇచ్చాం. ఇప్పుడు నీవు **SDE2 & SSE Go interviews ని confident గా** ఎదుర్కోగలవు, production-grade idiomatic Go రాయగలవు. తర్వాతి మెట్టు: `LLD_Go_Telugu.md`, `HLD_Go_Telugu.md`, `SystemDesign_Go_Telugu.md` – Go తో system design.
>
> **"ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు" – ఇప్పుడు నీకు Go జీవితాంతం గుర్తుంటుంది.** 🚀